DICE Embeddings

Release 0.1.3.2

Caglar Demir

Jul 23, 2026

Contents:

DICE Embeddings[1]: Hardware-agnostic Framework for Large-scale Knowledge Graph Embeddings:

1 Dicee Manual

Version: dicee 0.3.2

GitHub repository: https://github.com/dice-group/dice-embeddings

Publisher and maintainer: Caglar Demir[2]

Contact: caglar.demir@upb.de

License: OSI Approved :: MIT License

Dicee is a hardware-agnostic framework for large-scale knowledge graph embeddings.

Knowledge graph embedding research has mainly focused on learning continuous representations of knowledge graphs towards the link prediction problem. Recently developed frameworks can be effectively applied in a wide range of research-related applications. Yet, using these frameworks in real-world applications becomes more challenging as the size of the knowledge graph grows

We developed the DICE Embeddings framework (dicee) to compute embeddings for large-scale knowledge graphs in a hardware-agnostic manner. To achieve this goal, we rely on

1. **Pandas**[3] **& Co.** to use parallelism at preprocessing a large knowledge graph,
2. **PyTorch**[4] **& Co.** to learn knowledge graph embeddings via multi-CPUs, GPUs, TPUs or computing cluster, and
3. **Huggingface**[5] to ease the deployment of pre-trained models.

Why Pandas[6] **& Co. ?** A large knowledge graph can be read and preprocessed (e.g. removing literals) by pandas, modin, or polars in parallel. Through polars, a knowledge graph having more than 1 billion triples can be read in parallel fashion. Importantly, using these frameworks allow us to perform all necessary computations on a single CPU as well as a cluster of computers.

Why PyTorch[7] **& Co. ?** PyTorch is one of the most popular machine learning frameworks available at the time of writing. PytorchLightning facilitates scaling the training procedure of PyTorch without boilerplate. In our framework, we combine PyTorch[8] & PytorchLightning[9]. Users can choose the trainer class (e.g., DDP by Pytorch) to train large knowledge graph embedding models with billions of parameters. PytorchLightning allows us to use state-of-the-art model parallelism techniques (e.g. Fully Sharded Training, FairScale, or DeepSpeed) without extra effort. With our framework, practitioners can directly use PytorchLightning for model parallelism to train gigantic embedding models.

Why Huggingface[10]**?** Seamlessly deploy and share pre-trained embedding models through the Huggingface ecosystem.

[1] https://github.com/dice-group/dice-embeddings
[2] https://github.com/Demirrr
[3] https://pandas.pydata.org/
[4] https://pytorch.org/
[5] https://huggingface.co/
[6] https://pandas.pydata.org/
[7] https://pytorch.org/
[8] https://pytorch.org/
[9] https://www.pytorchlightning.ai/
[10] https://huggingface.co/

2 Installation

2.1 Installation from Source

```
git clone https://github.com/dice-group/dice-embeddings.git
conda create -n dice python=3.10.13 --no-default-packages && conda activate dice &&␣
↪cd dice-embeddings &&
pip3 install -e .
```

or

```
pip install dicee
```

3 Download Knowledge Graphs

```
wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪certificate && unzip KGs.zip
```

To test the Installation

```
python -m pytest -p no:warnings -x # Runs >114 tests leading to > 15 mins
python -m pytest -p no:warnings --lf # run only the last failed test
python -m pytest -p no:warnings --ff # to run the failures first and then the rest of␣
↪the tests.
```

4 Knowledge Graph Embedding Models

1. TransE, DistMult, ComplEx, ConEx, QMult, OMult, ConvO, ConvQ, Keci
2. All 44 models available in https://github.com/pykeen/pykeen#models

   For more, please refer to `examples`.

5 How to Train

To Train a KGE model (KECI) and evaluate it on the train, validation, and test sets of the UMLS benchmark dataset.

```
from dicee.executer import Execute
from dicee.config import Namespace
args = Namespace()
args.model = 'Keci'
args.scoring_technique = "KvsAll"  # 1vsAll, or AllvsAll, or NegSample
args.dataset_dir = "KGs/UMLS"
args.path_to_store_single_run = "Keci_UMLS"
args.num_epochs = 100
args.embedding_dim = 32
args.batch_size = 1024
reports = Execute(args).start()
print(reports["Train"]["MRR"]) # => 0.9912
print(reports["Test"]["MRR"]) # => 0.8155
# See the Keci_UMLS folder embeddings and all other files
```

where the data is in the following form

```
$ head -3 KGs/UMLS/train.txt
acquired_abnormality    location_of     experimental_model_of_disease
anatomical_abnormality  manifestation_of        physiologic_function
alga    isa     entity
```

A KGE model can also be trained from the command line

```
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

dicee automaticaly detects available GPUs and trains a model with distributed data parallels technique. Under the hood, dicee uses lighning as a default trainer.

```
# Train a model by only using the GPU-0
CUDA_VISIBLE_DEVICES=0 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪"train_val_test"
# Train a model by only using GPU-1
CUDA_VISIBLE_DEVICES=1 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪"train_val_test"
NCCL_P2P_DISABLE=1 CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL -
↪-dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

Under the hood, dicee executes run.py script and uses lighning as a default trainer

```
# Two equivalent executions
# (1)
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪'MRR': 0.8064032293278861}

# (2)
CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL --dataset_dir "KGs/
↪UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪9753123402351737}
# Evaluate Keci on Train set: Evaluate Keci on Train set
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪'MRR': 0.8064032293278861}
```

Similarly, models can be easily trained with torchrun

```
torchrun --standalone --nnodes=1 --nproc_per_node=gpu dicee/scripts/run.py --trainer␣
↪torchDDP --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪'MRR': 0.8072499937521418}
# Evaluate Keci on Test set: Evaluate Keci on Test set
{'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪'MRR': 0.8064032293278861}
```

You can also train a model in multi-node multi-gpu setting.

```
torchrun --nnodes 2 --nproc_per_node=gpu  --node_rank 0 --rdzv_id 455 --rdzv_backend␣
↪c10d --rdzv_endpoint=nebula  dicee/scripts/run.py --trainer torchDDP --dataset_dir␣
↪KGs/UMLS
torchrun --nnodes 2 --nproc_per_node=gpu  --node_rank 1 --rdzv_id 455 --rdzv_backend␣
↪c10d --rdzv_endpoint=nebula dicee/scripts/run.py --trainer torchDDP --dataset_dir␣
↪KGs/UMLS
```

Train a KGE model by providing the path of a single file and store all parameters under newly created directory called `KeciFamilyRun`.

```
dicee --path_single_kg "KGs/Family/family-benchmark_rich_background.owl" --model Keci␣
↪--path_to_store_single_run KeciFamilyRun --backend rdflib
```

where the data is in the following form

```
$ head -3 KGs/Family/train.txt
_:1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/2002/07/owl
↪#Ontology> .
<http://www.benchmark.org/family#hasChild> <http://www.w3.org/1999/02/22-rdf-syntax-ns
↪#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .
<http://www.benchmark.org/family#hasParent> <http://www.w3.org/1999/02/22-rdf-syntax-
↪ns#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .
```

Apart from n-triples or standard link prediction dataset formats, we support ["owl", "nt", "turtle", "rdf/xml", "n3"]*. Moreover, a KGE model can be also trained by providing **an endpoint of a triple store**.

```
dicee --sparql_endpoint "http://localhost:3030/mutagenesis/" --model Keci
```

For more, please refer to `examples`.

6 Creating an Embedding Vector Database

6.1 Learning Embeddings

```
# Train an embedding model
dicee --dataset_dir KGs/Countries-S1 --path_to_store_single_run CountryEmbeddings --
↪model Keci --p 0 --q 1 --embedding_dim 32 --adaptive_swa
```

6.2 Loading Embeddings into Qdrant Vector Database

```
# Ensure that Qdrant available
# docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334      -v $(pwd)/
↪qdrant_storage:/qdrant/storage:z      qdrant/qdrant
diceeindex --path_model "CountryEmbeddings" --collection_name "dummy" --location
↪"localhost"
```

6.3 Launching Webservice

```
diceeserve --path_model "CountryEmbeddings" --collection_name "dummy" --collection_
↪location "localhost"
```

Retrieve and Search

Get embedding of germany

```
curl -X 'GET' 'http://0.0.0.0:8000/api/get?q=germany' -H 'accept: application/json'
```

Get most similar things to europe

```
curl -X 'GET' 'http://0.0.0.0:8000/api/search?q=europe' -H 'accept: application/json'
{"result":[{"hit":"europe","score":1.0},
{"hit":"northern_europe","score":0.67126536},
{"hit":"western_europe","score":0.6010134},
{"hit":"puerto_rico","score":0.5051694},
{"hit":"southern_europe","score":0.4829831}]}
```

7 Answering Complex Queries

```
# pip install dicee
# wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪certificate & unzip KGs.zip
from dicee.executer import Execute
from dicee.config import Namespace
from dicee.knowledge_graph_embeddings import KGE
# (1) Train a KGE model
args = Namespace()
args.model = 'Keci'
args.p=0
args.q=1
args.optim = 'Adam'
args.scoring_technique = "AllvsAll"
args.path_single_kg = "KGs/Family/family-benchmark_rich_background.owl"
args.backend = "rdflib"
args.num_epochs = 200
args.batch_size = 1024
args.lr = 0.1
args.embedding_dim = 512
result = Execute(args).start()
# (2) Load the pre-trained model
```

(continues on next page)

(continued from previous page)

```
pre_trained_kge = KGE(path=result['path_experiment_folder'])
# (3) Single-hop query answering
# Query: ?E : \exist E.hasSibling(E, F9M167)
# Question: Who are the siblings of F9M167?
# Answer: [F9M157, F9F141], as (F9M167, hasSibling, F9M157) and (F9M167, hasSibling,␣
↪F9F141)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="1p",
                                                     query=('http://www.benchmark.org/
↪family#F9M167',
                                                            ('http://www.benchmark.
↪org/family#hasSibling',)),
                                                     tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9F141" in top_entities
assert "http://www.benchmark.org/family#F9M157" in top_entities
# (2) Two-hop query answering
# Query: ?D : \exist E.Married(D, E) \land hasSibling(E, F9M167)
# Question: To whom a sibling of F9M167 is married to?
# Answer: [F9F158, F9M142] as (F9M157 #married F9F158) and (F9F141 #married F9M142)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="2p",
                                                     query=("http://www.benchmark.org/
↪family#F9M167",
                                                            ("http://www.benchmark.
↪org/family#hasSibling",
                                                             "http://www.benchmark.
↪org/family#married")),
                                                     tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9M142" in top_entities
assert "http://www.benchmark.org/family#F9F158" in top_entities
# (3) Three-hop query answering
# Query: ?T : \exist D.type(D,T) \land Married(D,E) \land hasSibling(E, F9M167)
# Question: What are the type of people who are married to a sibling of F9M167?
# (3) Answer: [Person, Male, Father] since  F9M157 is [Brother Father Grandfather␣
↪Male] and F9M142 is [Male Grandfather Father]

predictions = pre_trained_kge.answer_multi_hop_query(query_type="3p", query=("http://
↪www.benchmark.org/family#F9M167",
                                                                             ("http://
↪www.benchmark.org/family#hasSibling",
                                                                             "http://
↪www.benchmark.org/family#married",
                                                                             "http://
↪www.w3.org/1999/02/22-rdf-syntax-ns#type")),
                                                     tnorm="min", k=5)
top_entities = [topk_entity for topk_entity, query_score in predictions]
print(top_entities)
assert "http://www.benchmark.org/family#Person" in top_entities
assert "http://www.benchmark.org/family#Father" in top_entities
assert "http://www.benchmark.org/family#Male" in top_entities
```

For more, please refer to `examples/multi_hop_query_answering`.

8 Predicting Missing Links

```
from dicee import KGE
# (1) Train a knowledge graph embedding model..
# (2) Load a pretrained model
pre_trained_kge = KGE(path='..')
# (3) Predict missing links through head entity rankings
pre_trained_kge.predict_topk(h=[".."],r=[".."],topk=10)
# (4) Predict missing links through relation rankings
pre_trained_kge.predict_topk(h=[".."],t=[".."],topk=10)
# (5) Predict missing links through tail entity rankings
pre_trained_kge.predict_topk(r=[".."],t=[".."],topk=10)
```

9 Downloading Pretrained Models

```
from dicee import KGE
# (1) Load a pretrained ConEx on DBpedia
model = KGE(url="https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-
↪dim128-epoch256-KvsAll")
```

- For more please look at dice-research.org/projects/DiceEmbeddings/[11]

10 How to Deploy

```
from dicee import KGE
KGE(path='...').deploy(share=True,top_k=10)
```

11 Docker

To build the Docker image:

```
docker build -t dice-embeddings .
```

To test the Docker image:

```
docker run --rm -v ~/.local/share/dicee/KGs:/dicee/KGs dice-embeddings ./main.py --
↪model AConEx --embedding_dim 16
```

12 Coverage Report

The coverage report is generated using coverage.py[12]:

```
Name                                                         Stmts   Miss  Cover   Missing
------------------------------------------------------------------------------------------
dicee/__init__.py                                                7      0   100%
dicee/abstracts.py                                             338    115    66%   112-113,␣
```

(continues on next page)

[11] https://files.dice-research.org/projects/DiceEmbeddings/
[12] https://coverage.readthedocs.io/en/7.6.0/

(continued from previous page)

```
↪131, 154-155, 160, 173, 197, 240-254, 290, 303-306, 309-313, 353-364, 379-387, 402,␣
↪413-417, 427-428, 434-436, 442-445, 448-453, 576-596, 602-606, 610-612, 631, 658-696
dicee/callbacks.py                                          248    103    58%   50-55,␣
↪67-73, 76, 88-93, 98-103, 106-109, 116-133, 138-142, 146-147, 247, 281-285, 291-292,
↪ 310-316, 319, 324-325, 337-343, 349-358, 363-365, 410, 421-434, 438-473, 485-491
dicee/config.py                                              97      2    98%   146-147
dicee/dataset_classes.py                                    430    146    66%   16, 44,␣
↪57, 89-98, 104, 111-118, 121, 124, 127-151, 207-213, 216, 219-221, 324, 335-338,␣
↪354, 420-421, 439, 562-581, 583, 587-599, 606-615, 618, 622-636, 780-787, 790-794,␣
↪845, 866-878, 902-915, 937, 941-954, 964-967, 973, 985, 987, 989, 1012-1022
dicee/eval_static_funcs.py                                  256    100    61%   104, 109,
↪ 114, 261-356, 363-414, 442, 465-468
dicee/evaluator.py                                          267     48    82%   48, 53,␣
↪58, 77, 82-83, 86, 102, 119, 130, 134, 139, 173-184, 191-202, 310, 340-358, 452,␣
↪462, 480-485
dicee/executer.py                                           134     16    88%   53-57,␣
↪166-176, 235-236, 283
dicee/knowledge_graph.py                                     82     10    88%   84, 94-
↪95, 124, 128, 132-134, 137-138, 140
dicee/knowledge_graph_embeddings.py                         654    415    37%   25, 28-
↪29, 37-50, 55-88, 91-125, 129-137, 171, 173-229, 261, 265, 276-277, 301-303, 311,␣
↪339-362, 493, 497-519, 523-547, 580, 656, 665, 710-716, 748, 806-1171, 1202-1263,␣
↪1267-1295, 1326, 1332
dicee/models/__init__.py                                      9      0   100%
dicee/models/adopt.py                                       187    172     8%   50-86,␣
↪99-110, 129-185, 195-242, 266-322, 346-448, 484-517
dicee/models/base_model.py                                  240     35    85%   30-35,␣
↪64, 66, 92, 99-116, 171, 204, 244, 250, 259, 262, 266, 273, 277, 279, 294, 307-308,␣
↪362, 365, 438, 450
dicee/models/clifford.py                                    470    278    41%   10, 12,␣
↪16, 24-25, 52-56, 79-87, 101-103, 108-109, 140-160, 184, 191, 195-256, 273-277, 289,
↪ 292, 297, 302, 346-361, 377-444, 464-470, 483, 486, 491, 496, 525-531, 544, 547,␣
↪552, 557, 567-576, 592-593, 613-685, 696-699, 724-749, 773-806, 842-846, 859, 869,␣
↪872, 877, 882, 887, 891, 895, 904-905, 935, 942, 947, 975-979, 1007-1016, 1026-1034,
↪ 1052-1054, 1072-1074, 1090-1092
dicee/models/complex.py                                     162     25    85%   86-109,␣
↪273-287
dicee/models/dualE.py                                        59     10    83%   93-102,␣
↪142-156
dicee/models/ensemble.py                                     89     67    25%   7-29, 31,
↪ 34, 37, 40, 43, 46, 49, 52-54, 56-58, 64-68, 71-90, 93-94, 97-112, 131
dicee/models/function_space.py                              262    221    16%   10-23,␣
↪27-36, 39-48, 52-69, 76-87, 90-99, 102-111, 115-127, 135-157, 160-166, 169-186, 189-
↪195, 198-206, 209, 214-235, 244-247, 251-255, 259-268, 272-293, 302-308, 312-329,␣
↪333-336, 345-353, 356, 367-373, 393-407, 425-439, 444-454, 462-466, 475-479
dicee/models/literal.py                                      33      1    97%   82
dicee/models/octonion.py                                    227     83    63%   21-44,␣
↪320-329, 334-345, 348-370, 374-416, 426-474
dicee/models/pykeen_models.py                                55      5    91%   77-80,␣
↪135
dicee/models/quaternion.py                                  192     69    64%   7-21, 30-
↪55, 68-72, 107, 185, 328-342, 345-364, 368-389, 399-426
```

(continues on next page)

(continued from previous page)

```
dicee/models/real.py                                    61     12    80%   37-42,␣
↪70-73, 91, 107-110
dicee/models/static_funcs.py                            10      0   100%
dicee/models/transformers.py                           234    189    19%   20-39,␣
↪42, 56-71, 80-98, 101-112, 119-121, 124, 130-147, 151-176, 182-186, 189-193, 199-
↪203, 206-208, 225-252, 261-264, 267-272, 275-300, 306-311, 315-368, 372-394, 400-410
dicee/query_generator.py                               374    346     7%   17-51,␣
↪55, 61-64, 68-69, 77-91, 99-146, 154-187, 191-205, 211-268, 273-302, 306-442, 452-
↪471, 479-502, 509-513, 518, 523-529
dicee/read_preprocess_save_load_kg/__init__.py           3      0   100%
dicee/read_preprocess_save_load_kg/preprocess.py       243     40    84%   33, 39,␣
↪76, 100-125, 131, 136-149, 175, 205, 380-381
dicee/read_preprocess_save_load_kg/read_from_disk.py    36     11    69%   34, 38-
↪40, 47, 55, 58-72
dicee/read_preprocess_save_load_kg/save_load_disk.py    53     21    60%   29-30,␣
↪38, 47-68
dicee/read_preprocess_save_load_kg/util.py             236    125    47%   159, 173-
↪175, 179-180, 198-204, 207-209, 214-216, 230, 244-247, 252-260, 265-271, 276-281,␣
↪286-291, 303-324, 330-386, 390-394, 398-399, 403, 407-408, 436, 441, 448-449
dicee/sanity_checkers.py                                47     19    60%   8-12, 21-
↪31, 46, 51, 58, 69-79
dicee/static_funcs.py                                  483    194    60%   42, 52,␣
↪58-63, 85, 92-96, 109-119, 129-131, 136, 143, 167, 172, 184, 190, 198, 202, 229-233,
↪ 295, 303-309, 320-330, 341-361, 389, 413-414, 419-420, 437-438, 440-441, 443-444,␣
↪452, 470-474, 491-494, 498-503, 507-511, 515-516, 522-524, 539-553, 558-561, 566-
↪569, 578-629, 634-646, 663-680, 683-691, 695-713, 724
dicee/static_funcs_training.py                         155     66    57%   7-10,␣
↪222-319, 327-328
dicee/static_preprocess_funcs.py                        98     43    56%   17-25,␣
↪50, 57, 59, 70, 83-107, 112-115, 120-123, 128-131
dicee/trainer/__init__.py                                1      0   100%
dicee/trainer/dice_trainer.py                          151     18    88%   22, 30-
↪31, 33-35, 97, 104, 109-114, 152, 237, 280-283
dicee/trainer/model_parallelism.py                      99     87    12%   10-25,␣
↪30-116, 121-132, 136, 141-197
dicee/trainer/torch_trainer.py                          77      6    92%   31, 102,␣
↪168, 179-181
dicee/trainer/torch_trainer_ddp.py                      89     71    20%   11-14,␣
↪43, 47-67, 78-94, 113-122, 126-136, 151-158, 168-191
-----------------------------------------------------------------------------------
TOTAL                                                 6948   3169    54%
```

13 How to cite

Currently, we are working on our manuscript describing our framework. If you really like our work and want to cite it now, feel free to chose one :)

```
# Keci
@inproceedings{demir2023clifford,
  title={Clifford Embeddings--A Generalized Approach for Embedding in Normed Algebras}
↪,
```

(continues on next page)

(continued from previous page)

```
  author={Demir, Caglar and Ngonga Ngomo, Axel-Cyrille},
  booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in␣
↪Databases},
  pages={567--582},
  year={2023},
  organization={Springer}
}
# LitCQD
@inproceedings{demir2023litcqd,
  title={LitCQD: Multi-Hop Reasoning in Incomplete Knowledge Graphs with Numeric␣
↪Literals},
  author={Demir, Caglar and Wiebesiek, Michel and Lu, Renzhong and Ngonga Ngomo, Axel-
↪Cyrille and Heindorf, Stefan},
  booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in␣
↪Databases},
  pages={617--633},
  year={2023},
  organization={Springer}
}
# DICE Embedding Framework
@article{demir2022hardware,
  title={Hardware-agnostic computation for large-scale knowledge graph embeddings},
  author={Demir, Caglar and Ngomo, Axel-Cyrille Ngonga},
  journal={Software Impacts},
  year={2022},
  publisher={Elsevier}
}
# KronE
@inproceedings{demir2022kronecker,
  title={Kronecker decomposition for knowledge graph embeddings},
  author={Demir, Caglar and Lienen, Julian and Ngonga Ngomo, Axel-Cyrille},
  booktitle={Proceedings of the 33rd ACM Conference on Hypertext and Social Media},
  pages={1--10},
  year={2022}
}
# QMult, OMult, ConvQ, ConvO
@InProceedings{pmlr-v157-demir21a,
  title =          {Convolutional Hypercomplex Embeddings for Link Prediction},
  author =       {Demir, Caglar and Moussallem, Diego and Heindorf, Stefan and Ngonga␣
↪Ngomo, Axel-Cyrille},
  booktitle =          {Proceedings of The 13th Asian Conference on Machine Learning},
  pages =          {656--671},
  year =          {2021},
  editor =          {Balasubramanian, Vineeth N. and Tsang, Ivor},
  volume =          {157},
  series =          {Proceedings of Machine Learning Research},
  month =          {17--19 Nov},
  publisher =    {PMLR},
  pdf =          {https://proceedings.mlr.press/v157/demir21a/demir21a.pdf},
  url =          {https://proceedings.mlr.press/v157/demir21a.html},
}
# ConEx
```

(continues on next page)

(continued from previous page)

```
@inproceedings{demir2021convolutional,
title={Convolutional Complex Knowledge Graph Embeddings},
author={Caglar Demir and Axel-Cyrille Ngonga Ngomo},
booktitle={Eighteenth Extended Semantic Web Conference - Research Track},
year={2021},
url={https://openreview.net/forum?id=6T45-4TFqaX}}
# Shallom
@inproceedings{demir2021shallow,
  title={A shallow neural model for relation prediction},
  author={Demir, Caglar and Moussallem, Diego and Ngomo, Axel-Cyrille Ngonga},
  booktitle={2021 IEEE 15th International Conference on Semantic Computing (ICSC)},
  pages={179--182},
  year={2021},
  organization={IEEE}
```

For any questions or wishes, please contact: `caglar.demir@upb.de`

14 dicee

DICE Embeddings - Knowledge Graph Embedding Library.

A library for training and using knowledge graph embedding models with support for various scoring techniques and training strategies.

Submodules:

evaluation: Model evaluation functions and Evaluator class models: KGE model implementations trainer: Training orchestration scripts: Utility scripts

14.1 Submodules

dicee.__main__

dicee.abstracts

Attributes

logger

Classes

AbstractTrainer	Abstract base class for KGE model trainers.
BaseInteractiveKGE	Base class for interactive, post-training use of KGE models.
InteractiveQueryDecomposition	Mixin that provides fuzzy-logic operators for multi-hop EPFO query answering.
AbstractCallback	Abstract base class for KGE training lifecycle callbacks.
AbstractPPECallback	Abstract base class for Post-training Parameter Ensembling (PPE) callbacks.
BaseInteractiveTrainKGE	Abstract/base class for training knowledge graph embedding models interactively.

Module Contents

`dicee.abstracts.`**`logger`**

class `dicee.abstracts.`**`AbstractTrainer`**(*args*, *callbacks*)

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (TorchTrainer, TorchDDPTrainer, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered *`AbstractCallback`* can react.

Parameters

- **`args`** (*`argparse.Namespace or similar`*) – Processed configuration object. Must expose at least `random_seed` (int).
- **`callbacks`** (*`list of AbstractCallback`*) – Ordered list of callback instances to invoke at each lifecycle hook.

`attributes`

`callbacks`

`is_global_zero = True`

`global_rank = 0`

`local_rank = 0`

`strategy = None`

`on_fit_start`(**args*, ***kwargs*)

Dispatch `on_fit_start` to all registered callbacks.

Called once before the first training epoch begins.

`on_fit_end`(**args*, ***kwargs*)

Dispatch `on_fit_end` to all registered callbacks.

Called once after the last training epoch completes.

`on_train_epoch_start`(**args*, ***kwargs*)

Dispatch `on_train_epoch_start` to all registered callbacks.

Called at the beginning of every epoch.

`on_train_epoch_end`(**args*, ***kwargs*)

Dispatch `on_train_epoch_end` to all registered callbacks.

Called at the end of every epoch after the loss has been accumulated.

`on_train_batch_end`(**args*, ***kwargs*)

Dispatch `on_train_batch_end` to all registered callbacks.

Called after each mini-batch gradient update.

`static save_checkpoint`(*full_path: str*, *model*) → None

Persist model weights to disk.

Parameters

- **`full_path`** (*`str`*) – Absolute or relative file path (including filename) where the `state_dict` will be written, e.g. `'Experiments/run1/model.pt'`.

- **model** (*torch.nn.Module*) – The model whose `state_dict` is to be saved.

class dicee.abstracts.**BaseInteractiveKGE**(*path: str = None*, *url: str = None*, *construct_ensemble: bool = False*, *model_name: str = None*, *apply_semantic_constraint: bool = False*)

Base class for interactive, post-training use of KGE models.

Loads a pre-trained model from disk (or a remote URL) together with its entity/relation index mappings and exposes the prediction API used by *KGE*.

Parameters

- **path** (*str, optional*) – Path to the experiment directory produced by `Execute`. Must contain `model.pt`, `configuration.json`, `entity_to_idx.csv` and `relation_to_idx.csv`.
- **url** (*str, optional*) – Remote URL of a pre-trained model to download. Mutually exclusive with *path*.
- **construct_ensemble** (*bool, optional*) – When `True`, load all checkpoint files in *path* and average their weights to form an ensemble model. Defaults to `False`.
- **model_name** (*str, optional*) – Filename (without extension) of the checkpoint to load when multiple `.pt` files exist in *path*.
- **apply_semantic_constraint** (*bool, optional*) – Reserved for future use. Defaults to `False`.

construct_ensemble = False

apply_semantic_constraint = False

configs

get_eval_report() → dict

get_bpe_token_representation(*str_entity_or_relation: List[str] | str*) → List[List[int]] | List[int]

Parameters

str_entity_or_relation (*corresponds to a str or a list of strings to be tokenized via BPE and shaped.*)

Return type

A list integer(s) or a list of lists containing integer(s)

get_padded_bpe_triple_representation(*triples: List[List[str]]*) → Tuple[List, List, List]

Parameters

triples

set_model_train_mode() → None

Switch the underlying model to training mode.

Calls `model.train()` and re-enables gradient computation for all parameters so that subsequent calls to optimisation steps work correctly after a period of inference.

set_model_eval_mode() → None

Switch the underlying model to evaluation mode.

Calls `model.eval()` and freezes all parameters (`requires_grad = False`) so that dropout and batch-norm layers behave deterministically during inference.

property name

sample_entity(*n: int*) → List[str]

Return *n* random entity strings without replacement.

Parameters
`n` (*int*) – Number of entities to sample. Must be non-negative and at most `num_entities`.

Returns
Randomly selected entity string labels.

Return type
List[str]

sample_relation(*n: int*) → List[str]

Return *n* random relation strings without replacement.

Parameters
`n` (*int*) – Number of relations to sample. Must be non-negative and at most `num_relations`.

Returns
Randomly selected relation string labels.

Return type
List[str]

is_seen(*entity: str = None*, *relation: str = None*) → bool

Check whether an entity or relation was present in the training set.

Exactly one of *entity* or *relation* should be provided.

Parameters

- **entity** (*str, optional*) – Entity string label to look up.
- **relation** (*str, optional*) – Relation string label to look up.

Returns
`True` if the given string is in the respective index mapping, `False` otherwise.

Return type
bool

save() → None

Persist the current model weights to the experiment directory.

The checkpoint filename encodes the current timestamp so successive calls do not overwrite each other. Ensemble models are saved with an `_ensemble_` infix in the filename.

get_entity_index(*x: str*) → int

Return the integer index for a given entity string.

Parameters
`x` (*str*) – Entity string label (must have been seen during training).

Returns
Corresponding row index in the entity embedding matrix.

Return type
int

get_relation_index(*x: str*) → int

Return the integer index for a given relation string.

Parameters
x (*str*) – Relation string label (must have been seen during training).

Returns
Corresponding row index in the relation embedding matrix.

Return type
int

index_triple(*head_entity: List[str]*, *relation: List[str]*, *tail_entity: List[str]*)
→ Tuple[torch.LongTensor, torch.LongTensor, torch.LongTensor]

Convert string triple lists to integer index tensors.

Parameters

- **head_entity** (*List[str]*) – Head entity string labels.
- **relation** (*List[str]*) – Relation string labels.
- **tail_entity** (*List[str]*) – Tail entity string labels.

Returns
idx_head_entity, idx_relation, idx_tail_entity – Each has shape `(n, 1)` containing the integer indices for the corresponding strings.

Return type
torch.LongTensor

add_new_entity_embeddings(*entity_name: str = None*, *embeddings: torch.FloatTensor = None*)
→ None

Extend the entity embedding table with a new entity at inference time.

The new entity is appended to both `entity_to_idx` / `idx_to_entity` mappings and the `entity_embeddings` weight tensor so that subsequent calls to prediction methods can reference it by name.

Parameters

- **entity_name** (*str*) – String label for the new entity. If the entity already exists in the index no modification is made.
- **embeddings** (*torch.FloatTensor*) – 1-D float tensor of length `embedding_dim` containing the pre-computed embedding for the new entity.

get_entity_embeddings(*items: List[str]*) → torch.FloatTensor

Return the embedding vectors for the given entity strings.

For standard (non-BPE) models the method looks up each string in `entity_to_idx` and returns the corresponding rows of the entity embedding matrix. For BPE models subword token embeddings are fetched and flattened into a single vector per entity.

Parameters
items (*List[str]*) – Entity string labels to retrieve.

Returns
Shape `(len(items), embedding_dim)`.

Return type
torch.FloatTensor

get_relation_embeddings(*items: List[str]*) → torch.FloatTensor

Return the embedding vectors for the given relation strings.

Parameters

items (*List[str]*) – Relation string labels to retrieve.

Returns

Shape `(len(items), embedding_dim)`.

Return type

torch.FloatTensor

construct_input_and_output(*head_entity: List[str]*, *relation: List[str]*, *tail_entity: List[str]*, *labels*) → Tuple[torch.LongTensor, torch.FloatTensor]

Build an indexed triple tensor and a label tensor from string inputs.

Parameters

- **head_entity** (*List[str]*) – Head entity string labels.
- **relation** (*List[str]*) – Relation string labels.
- **tail_entity** (*List[str]*) – Tail entity string labels.
- **labels** (*list or array-like*) – Binary or soft labels (one per triple) used as training targets.

Returns

- **x** (*torch.LongTensor*) – Shape `(n, 3)` integer-indexed triples.
- **labels** (*torch.FloatTensor*) – Shape `(n,)` float label tensor.

parameters()

class dicee.abstracts.**InteractiveQueryDecomposition**

Mixin that provides fuzzy-logic operators for multi-hop EPFO query answering.

The three families of operators — T-norm, T-conorm, and negation norm — are applied element-wise over entity score tensors to compose complex queries from atomic link-prediction results (e.g. 2p, 3p, 2i, ip, up).

t_norm(*tens_1: torch.Tensor*, *tens_2: torch.Tensor*, *tnorm: str = 'min'*) → torch.Tensor

Apply a T-norm to combine two entity score distributions.

Parameters

- **tens_1** (*torch.Tensor*) – Score tensors of identical shape, values in `[0, 1]`.
- **tens_2** (*torch.Tensor*) – Score tensors of identical shape, values in `[0, 1]`.
- **tnorm** (*str*) – Operator to use. `'min'` applies the Gödel (min) T-norm; `'prod'` applies the product T-norm.

Returns

Element-wise combined scores of the same shape as the inputs.

Return type

torch.Tensor

tensor_t_norm(*subquery_scores: torch.FloatTensor*, *tnorm: str = 'min'*) → torch.FloatTensor

Compute T-norm over [0,1] ^{n imes d} where n denotes the number of hops and d denotes number of entities

t_conorm(*tens_1: torch.Tensor*, *tens_2: torch.Tensor*, *tconorm: str = 'min'*) → torch.Tensor

Apply a T-conorm (S-norm) to combine two score distributions (union).

Parameters

- **tens_1** (*torch.Tensor*) – Score tensors of identical shape, values in `[0, 1]`.
- **tens_2** (*torch.Tensor*) – Score tensors of identical shape, values in `[0, 1]`.
- **tconorm** (*str*) – Operator to use. `'min'` applies the Gödel (max) T-conorm; `'prod'` applies the probabilistic sum T-conorm.

Returns

Element-wise combined scores of the same shape as the inputs.

Return type

torch.Tensor

negnorm(*tens_1: torch.Tensor*, *lambda_: float*, *neg_norm: str = 'standard'*) → torch.Tensor

Apply a negation norm (complement) to an entity score distribution.

Parameters

- **tens_1** (*torch.Tensor*) – Input score tensor, values in `[0, 1]`.
- **lambda** (*float*) – Shape parameter used by the Sugeno and Yager negation norms. Ignored for the standard complement.
- **neg_norm** (*str*) – Which negation to apply: `'standard'` (`1 - x`), `'sugeno'`, or `'yager'`.

Returns

Complemented score tensor of the same shape as *tens_1*.

Return type

torch.Tensor

class dicee.abstracts.**AbstractCallback**

Bases: `abc.ABC`, `lightning.pytorch.callbacks.Callback`

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's *callbacks* list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

on_init_start(**args*, ***kwargs*)

Called when the trainer is about to be constructed.

Override to perform setup that must happen before any trainer state is initialised.

on_init_end(**args*, ***kwargs*)

Called immediately after the trainer has been constructed.

Override to perform setup that requires a fully initialised trainer.

on_fit_start(*trainer*, *model*)

Called once before the first training epoch.

Parameters

- **trainer** (*AbstractTrainer or pl.Trainer*) – The active trainer instance.

- **model** (BaseKGE) – The model about to be trained.

on_train_epoch_end(*trainer*, *model*)

Called at the end of each training epoch.

Parameters

- **trainer** (AbstractTrainer *or* pl.*Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

on_train_batch_end(**args*, ***kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

on_fit_end(**args*, ***kwargs*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

class dicee.abstracts.**AbstractPPECallback**(*num_epochs*, *path*, *epoch_to_start*, *last_percent_to_consider*)

Bases: *AbstractCallback*

Abstract base class for Post-training Parameter Ensembling (PPE) callbacks.

Sub-classes implement weight-averaging strategies (SWA, EMA, SWAG, …) by overriding `on_train_epoch_end()` and *on_fit_end()*. Common book-keeping (epoch counter, sample counter, alpha weights) is managed here.

Parameters

- **num_epochs** (*int*) – Total number of training epochs.
- **path** (*str*) – Experiment directory where averaged checkpoints will be written.
- **epoch_to_start** (*int*) – First epoch at which the averaging procedure should begin.
- **last_percent_to_consider** (*float*) – Fraction of the total training epochs (counted from the end) whose checkpoints are included in the ensemble.

num_epochs

path

sample_counter = 0

epoch_count = 0

alphas = None

on_fit_start(*trainer*, *model*)

Called once before the first training epoch.

Parameters

- **trainer** (AbstractTrainer *or* pl.*Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model about to be trained.

on_fit_end(*trainer*, *model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

store_ensemble(*param_ensemble*) → None

class dicee.abstracts.**BaseInteractiveTrainKGE**

Abstract/base class for training knowledge graph embedding models interactively. This class provides methods for re-training KGE models and also Literal Embedding model.

train_triples(*h: List[str]*, *r: List[str]*, *t: List[str]*, *labels: List[float]*, *iteration=2*, *optimizer=None*)

train_k_vs_all(*h*, *r*, *iteration=1*, *lr=0.001*)

Train k vs all :param head_entity: :param relation: :param iteration: :param lr: :return:

train(*kg*, *lr=0.1*, *epoch=10*, *batch_size=32*, *neg_sample_ratio=10*, *num_workers=1*) → None

Retrained a pretrain model on an input KG via negative sampling.

train_literals(*train_file_path: str = None*, *num_epochs: int = 100*, *lit_lr: float = 0.001*, *lit_normalization_type: str = 'z-norm'*, *batch_size: int = 1024*, *sampling_ratio: float = None*, *random_seed=1*, *loader_backend: str = 'pandas'*, *freeze_entity_embeddings: bool = True*, *gate_residual: bool = True*, *device: str = None*, *suffle_data: bool = True*)

Trains the Literal Embeddings model using literal data.

Parameters

- **train_file_path** (*str*) – Path to the training data file.
- **num_epochs** (*int*) – Number of training epochs.
- **lit_lr** (*float*) – Learning rate for the literal model.
- **norm_type** (*str*) – Normalization type to use ('z-norm', 'min-max', or None).
- **batch_size** (*int*) – Batch size for training.
- **sampling_ratio** (*float*) – Ratio of training triples to use.
- **loader_backend** (*str*) – Backend for loading the dataset ('pandas' or 'rdflib').
- **freeze_entity_embeddings** (*bool*) – If True, freeze the entity embeddings during training.
- **gate_residual** (*bool*) – If True, use gate residual connections in the model.
- **device** (*str*) – Device to use for training ('cuda' or 'cpu'). If None, will use available GPU or CPU.
- **suffle_data** (*bool*) – If True, shuffle the dataset before training.

dicee.analyse_experiments

This script should be moved to dicee/scripts Example: python dicee/analyse_experiments.py –dir Experiments –features "model" "trainMRR" "testMRR"

Attributes

logger

Classes

Experiment

Functions

get_default_arguments()
analyse(args)

Module Contents

dicee.analyse_experiments.**logger**

dicee.analyse_experiments.**get_default_arguments**()

class dicee.analyse_experiments.**Experiment**

model_name = []

callbacks = []

embedding_dim = []

num_params = []

num_epochs = []

batch_size = []

lr = []

byte_pair_encoding = []

aswa = []

path_dataset_folder = []

full_storage_path = []

pq = []

train_mrr = []

train_h1 = []

train_h3 = []

train_h10 = []

val_mrr = []

val_h1 = []

val_h3 = []

val_h10 = []

test_mrr = []

test_h1 = []

test_h3 = []

test_h10 = []

runtime = []

normalization = []

scoring_technique = []

save_experiment(*x*)

to_df()

dicee.analyse_experiments.**analyse**(*args*)

dicee.callbacks

Callbacks for training lifecycle events.

Provides callback classes for various training events including epoch end, model saving, weight averaging, and evaluation.

Attributes

logger

Classes

AccumulateEpochLossCallback	Callback to store epoch losses to a CSV file.
PrintCallback	Callback that prints training start/end times and total runtime.
KGESaveCallback	Callback that periodically saves model checkpoints during training.
PseudoLabellingCallback	Callback that augments the training set with pseudo-labelled triples.
Eval	Abstract base class for KGE training lifecycle callbacks.
KronE	Abstract base class for KGE training lifecycle callbacks.
Perturb	A callback for a three-Level Perturbation
PeriodicEvalCallback	Callback to periodically evaluate the model and optionally save checkpoints during training.
LRScheduler	Callback for managing learning rate scheduling and model snapshots.

Functions

estimate_q(eps)	estimate rate of convergence q from sequence esp
compute_convergence(seq, i)	

Module Contents

`dicee.callbacks.logger`

class `dicee.callbacks.AccumulateEpochLossCallback`(*path: str*)

Bases: *dicee.abstracts.AbstractCallback*

Callback to store epoch losses to a CSV file.

Parameters
path – Directory path where the loss file will be saved.

path

on_fit_end(*trainer*, *model*) → None

Store epoch loss history to CSV file.

Parameters

- **trainer** – The trainer instance.
- **model** – The model being trained.

class `dicee.callbacks.PrintCallback`

Bases: *dicee.abstracts.AbstractCallback*

Callback that prints training start/end times and total runtime.

start_time

on_fit_start(*trainer*, *pl_module*)

Called once before the first training epoch.

Parameters

- **trainer** (AbstractTrainer *or pl.Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model about to be trained.

on_fit_end(*trainer*, *pl_module*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

on_train_batch_end(**args*, ***kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

on_train_epoch_end(**args*, ***kwargs*)

Called at the end of each training epoch.

Parameters

- **trainer** (AbstractTrainer *or pl.Trainer*) – The active trainer instance.

- **model** (BaseKGE) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

class dicee.callbacks.**KGESaveCallback**(*every_x_epoch: int*, *max_epochs: int*, *path: str*)

Bases: *dicee.abstracts.AbstractCallback*

Callback that periodically saves model checkpoints during training.

Parameters

- **every_x_epoch** (*int or None*) – Save a checkpoint every *every_x_epoch* epochs. When `None`, the interval defaults to `max(max_epochs // 2, 1)`.
- **max_epochs** (*int*) – Total number of training epochs (used to compute the default interval when *every_x_epoch* is `None`).
- **path** (*str*) – Directory where checkpoint files will be written.

every_x_epoch

max_epochs

epoch_counter = 0

path

on_train_batch_end(**args*, ***kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

on_fit_start(*trainer*, *pl_module*)

Called once before the first training epoch.

Parameters

- **trainer** (AbstractTrainer *or pl.Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model about to be trained.

on_train_epoch_end(**args*, ***kwargs*)

Called at the end of each training epoch.

Parameters

- **trainer** (AbstractTrainer *or pl.Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

on_fit_end(**args*, ***kwargs*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

on_epoch_end(*model*, *trainer*, ***kwargs*)

class dicee.callbacks.**PseudoLabellingCallback**(*data_module*, *kg*, *batch_size*)

Bases: *dicee.abstracts.AbstractCallback*

Callback that augments the training set with pseudo-labelled triples.

At the end of each epoch the current model scores a batch of unlabelled triples and those with a predicted probability >= 0.90 are appended to the training dataset (semi-supervised self-training).

Parameters

- **data_module** (*object*) – Dataset module exposing a `train_set_idx` attribute and a `train_dataloader()` method.
- **kg** (KG) – The knowledge graph, providing `num_entities`, `num_relations`, and `unlabelled_set`.
- **batch_size** (*int*) – Number of unlabelled triples to sample and score each epoch.

data_module

kg

num_of_epochs = 0

unlabelled_size

batch_size

create_random_data()

on_epoch_end(*trainer*, *model*)

dicee.callbacks.**estimate_q**(*eps*)

estimate rate of convergence q from sequence esp

dicee.callbacks.**compute_convergence**(*seq*, *i*)

class dicee.callbacks.**Eval**(*path*, *epoch_ratio: int = None*)

Bases: *dicee.abstracts.AbstractCallback*

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's *callbacks* list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

path

reports = []

epoch_ratio

epoch_counter = 0

on_fit_start(*trainer*, *model*)

Called once before the first training epoch.

Parameters

- **trainer** (AbstractTrainer *or pl.Trainer*) – The active trainer instance.
- **model** (BaseKGE) – The model about to be trained.

on_fit_end(*trainer*, *model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

on_train_epoch_end(*trainer*, *model*)

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer or pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

on_train_batch_end(**args*, ***kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

class dicee.callbacks.**KronE**

Bases: *dicee.abstracts.AbstractCallback*

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's *callbacks* list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

f = None

static batch_kronecker_product(*a*, *b*)

Kronecker product of matrices a and b with leading batch dimensions. Batch dimensions are broadcast. The number of them mush :type a: torch.Tensor :type b: torch.Tensor :rtype: torch.Tensor

get_kronecker_triple_representation(*indexed_triple: torch.LongTensor*)

Get kronecker embeddings

on_fit_start(*trainer*, *model*)

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer or pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

class dicee.callbacks.**Perturb**(*level: str = 'input'*, *ratio: float = 0.0*, *method: str = None*, *scaler: float = None*, *frequency=None*)

Bases: *dicee.abstracts.AbstractCallback*

A callback for a three-Level Perturbation

Input Perturbation: During training an input x is perturbed by randomly replacing its element. In the context of knowledge graph embedding models, x can denote a triple, a tuple of an entity and a relation, or a tuple of two entities. A perturbation means that a component of x is randomly replaced by an entity or a relation.

Parameter Perturbation:

Output Perturbation:

level = 'input'

ratio = 0.0

method = None

scaler = None

frequency = None

on_train_batch_start(*trainer*, *model*, *batch*, *batch_idx*)

Called when the train batch begins.

class dicee.callbacks.**PeriodicEvalCallback**(*experiment_path: str*, *max_epochs: int*, *eval_every_n_epoch: int = 0*, *eval_at_epochs: list = None*, *save_model_every_n_epoch: bool = True*, *n_epochs_eval_model: str = 'val_test'*)

Bases: *dicee.abstracts.AbstractCallback*

Callback to periodically evaluate the model and optionally save checkpoints during training.

Evaluates at regular intervals (every N epochs) or at explicitly specified epochs. Stores evaluation reports and model checkpoints.

experiment_dir

max_epochs

epoch_counter = 0

save_model_every_n_epoch = True

reports

n_epochs_eval_model = 'val_test'

default_eval_model = None

eval_epochs

on_fit_end(*trainer*, *model*)

Called at the end of training. Saves final evaluation report.

on_train_epoch_end(*trainer*, *model*)

Called at the end of each training epoch. Performs evaluation and checkpointing if scheduled.

class dicee.callbacks.**LRScheduler**(*adaptive_lr_config: dict*, *total_epochs: int*, *experiment_dir: str*, *eta_max: float = 0.1*, *snapshot_dir: str = 'snapshots'*)

Bases: *dicee.abstracts.AbstractCallback*

Callback for managing learning rate scheduling and model snapshots.

Supports cosine annealing ("cca"), MMCCLR ("mmcclr"), and their deferred (warmup) variants: - "deferred_cca" - "deferred_mmcclr"

At the end of each learning rate cycle, the model can optionally be saved as a snapshot.

total_epochs

experiment_dir

snapshot_dir

batches_per_epoch = None

total_steps = None

cycle_length = None

warmup_steps = None

lr_lambda = None

scheduler = None

step_count = 0

snapshot_loss

on_train_start(*trainer*, *model*)

Initialize training parameters and LR scheduler at start of training.

on_train_batch_end(*trainer*, *model*, *outputs*, *batch*, *batch_idx*)

Step the LR scheduler and save model snapshot if needed after each batch.

on_fit_end(*trainer*, *model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

dicee.config

Configuration module for DICE embeddings.

Provides the Namespace class with default configuration values for training knowledge graph embedding models.

Classes

`Namespace`	Extended Namespace with default KGE training configuration.

Module Contents

class `dicee.config.`**Namespace**(***kwargs*)

Bases: `argparse.Namespace`

Extended Namespace with default KGE training configuration.

Provides sensible defaults for all training parameters while allowing easy customization through command-line arguments or direct assignment.

dataset_dir: str | None = None

The path of a folder containing train.txt, and/or valid.txt and/or test.txt

save_embeddings_as_csv: bool = False

Embeddings of entities and relations are stored into CSV files to facilitate easy usage.

storage_path: str = 'Experiments'

A directory named with time of execution under –storage_path that contains related data about embeddings.

path_to_store_single_run: str | None = None

A single directory created that contains related data about embeddings.

path_single_kg = None

Path of a file corresponding to the input knowledge graph

sparql_endpoint = None

An endpoint of a triple store.

model: str = 'Keci'

KGE model

optim: str = 'Adam'

Optimizer

embedding_dim: int = 64

Size of continuous vector representation of an entity/relation

num_epochs: int = 150

Number of pass over the training data

batch_size: int = 1024

Mini-batch size if it is None, an automatic batch finder technique applied

lr: float = 0.1

Learning rate

add_noise_rate: float | None = None

The ratio of added random triples into training dataset

gpus = None

Number GPUs to be used during training

callbacks

10}}

Type

Callbacks, e.g., {“PPE”

Type

{ “last_percent_to_consider”

backend: str = 'pandas'

Backend to read, process, and index input knowledge graph. pandas, polars and rdflib available

separator: str = '\\s+'

separator for extracting head, relation and tail from a triple

trainer: str = 'torchCPUTrainer'

‘torchCPUTrainer’ (CPU/single GPU), ‘PL’ (PyTorch Lightning multi-GPU), ‘torchDDP’ (native DDP), ‘TP’ (Tensor Parallelism - implements ‘Multiple Run Ensemble Learning with Low-Dimensional Knowledge Graph Embeddings’)

Type

Trainer for knowledge graph embedding model. Options

scoring_technique: str = 'KvsAll'

Scoring technique for knowledge graph embedding models

neg_ratio: int = 0

Negative ratio for a true triple in NegSample training_technique

weight_decay: float = 0.0
: Weight decay for all trainable params

normalization: str = 'None'
: LayerNorm, BatchNorm1d, or None

init_param: str | None = None
: xavier_normal or None

gradient_accumulation_steps: int = 0
: Not tested e

num_folds_for_cv: int = 0
: Number of folds for CV

eval_model: str = 'train_val_test'
: ["None", "train", "train_val", "train_val_test", "test"]

  Type
  : Evaluate trained model choices

save_model_at_every_epoch: int | None = None
: Not tested

label_smoothing_rate: float = 0.0

num_core: int = 0
: Number of CPUs to be used in the mini-batch loading process

random_seed: int = 0
: Random Seed

sample_triples_ratio: float | None = None
: Read some triples that are uniformly at random sampled. Ratio being between 0 and 1

read_only_few: int | None = None
: Read only first few triples

pykeen_model_kwargs
: Additional keyword arguments for pykeen models

pl_trainer_kwargs
: Additional keyword arguments for the PyTorch Lightning Trainer

kernel_size: int = 3
: Size of a square kernel in a convolution operation

num_of_output_channels: int = 32
: Number of slices in the generated feature map by convolution.

p: int = 0
: P parameter of Clifford Embeddings

q: int = 1
: Q parameter of Clifford Embeddings

input_dropout_rate: float = 0.0
: Dropout rate on embeddings of input triples

hidden_dropout_rate: float = 0.0
Dropout rate on hidden representations of input triples

feature_map_dropout_rate: float = 0.0
Dropout rate on a feature map generated by a convolution operation

byte_pair_encoding: bool = False
Byte pair encoding

Type
WIP

adaptive_swa: bool = False
Adaptive stochastic weight averaging

swa: bool = False
Stochastic weight averaging

swag: bool = False
Stochastic weight averaging - Gaussian

ema: bool = False
Exponential Moving Average

twa: bool = False
Trainable weight averaging

block_size: int | None = None
block size of LLM

continual_learning: str | None = None
Path of a pretrained model size of LLM

auto_batch_finding: bool = False
A flag for using auto batch finding

eval_every_n_epochs: int = 0
Evaluate model every n epochs. If 0, no evaluation is applied.

save_every_n_epochs: bool = False
Save model every n epochs. If True, save model at every epoch.

eval_at_epochs: list | None = None
List of epoch numbers at which to evaluate the model (e.g., 1 5 10).

n_epochs_eval_model: str = 'val_test'
Evaluating link prediction performance on data splits while performing periodic evaluation.

adaptive_lr
"cca"}'

Type
Adaptive learning rate parameters, e.g., '{"scheduler_name"

swa_start_epoch: int | None = None
Epoch at which to start applying stochastic weight averaging.

swa_c_epochs: int = 1
Number of epochs to average over for SWA, SWAG, EMA, TWA.

__iter__()

dicee.dataset_classes

Dataset classes for knowledge graph embedding training.

This package groups the various `torch.utils.data.Dataset` implementations used throughout DICE Embeddings into thematic sub-modules:

- `_bpe` – Byte-pair-encoding related datasets.
- `_negative_sampling` – Negative sampling based datasets.
- `_label_based` – Multi-label / multi-class scoring datasets.
- `_literal` – Literal (numeric) embedding dataset.
- `_factory` – `construct_dataset` / `reload_dataset` helpers.

All public names are re-exported here so that existing `from dicee.dataset_classes import` … statements continue to work unchanged.

Classes

BPE_NegativeSamplingDataset	Dataset for negative sampling with byte-pair encoded triples.
MultiClassClassificationDataset	Dataset for autoregressive multi-class classification on sub-word units.
MultiLabelDataset	Multi-label dataset for BPE-based KvsAll / AllvsAll training.
AllvsAll	Dataset for AllvsAll training (multi-label, exhaustive).
FSDP1vsSampleDataset	Positive-triple dataset for FSDP 1vsSample training with true-negative sampling.
KvsAll	Dataset for KvsAll training (multi-label).
KvsSampleDataset	Dataset for KvsSample training (dynamic multi-label).
OnevsAllDataset	Dataset for the 1-vs-All training strategy (multi-class).
LiteralDataset	Dataset for loading and processing literal data for Literal Embedding models.
FixedNegSampleDataset	Pre-computed (fixed) negative sampling dataset.
OnevsSample	Dataset for 1-vs-Sample training (dynamic multi-class with negatives).
TriplePredictionDataset	Dataset for triple prediction with on-the-fly negative sampling.

Functions

construct_dataset(→ torch.utils.data.Dataset)	Build the appropriate dataset for the given training configuration.
reload_dataset(path, form_of_labelling, ...)	Reload training data from disk and construct a Pytorch dataset.

Package Contents

class dicee.dataset_classes.**BPE_NegativeSamplingDataset**(*train_set: torch.LongTensor*, *ordered_shaped_bpe_entities: torch.LongTensor*, *neg_ratio: int*)

Bases: `torch.utils.data.Dataset`

Dataset for negative sampling with byte-pair encoded triples.

Each sample is a BPE-encoded triple. The custom `collate_fn` constructs negatives by corrupting head or tail entities with random BPE entities.

Parameters

- **train_set** (*torch.LongTensor*) – Integer-encoded triples of shape `(N, 3)`.
- **ordered_shaped_bpe_entities** (*torch.LongTensor*) – All BPE entity representations, ordered by entity index.
- **neg_ratio** (*int*) – Number of negative samples per positive triple.

train_set

ordered_bpe_entities

num_bpe_entities

neg_ratio

num_datapoints

__len__()

__getitem__(*idx*)

collate_fn(*batch_shaped_bpe_triples: List[Tuple[torch.Tensor, torch.Tensor]]*)

class dicee.dataset_classes.**MultiClassClassificationDataset**(*subword_units: numpy.ndarray*, *block_size: int = 8*)

Bases: `torch.utils.data.Dataset`

Dataset for autoregressive multi-class classification on sub-word units.

Splits a flat sequence of sub-word token ids into overlapping windows of size `block_size` for next-token prediction.

Parameters

- **subword_units** (*numpy.ndarray*) – 1-D array of sub-word token ids.
- **block_size** (*int, optional*) – Context window length (default `8`).

train_data

block_size = 8

num_of_data_points

collate_fn = None

__len__()

`__getitem__`(*idx*)

class dicee.dataset_classes.**MultiLabelDataset**(*train_set: torch.LongTensor, train_indices_target: torch.LongTensor, target_dim: int, torch_ordered_shaped_bpe_entities: torch.LongTensor*)

Bases: `torch.utils.data.Dataset`

Multi-label dataset for BPE-based KvsAll / AllvsAll training.

Each sample is a BPE-encoded `(head, relation)` pair together with a binary multi-label target vector over all entities.

Parameters

- **train_set** (*torch.LongTensor*) – BPE-encoded input pairs of shape `(N, 2, token_length)`.
- **train_indices_target** (*torch.LongTensor*) – Per-sample lists of positive target entity indices.
- **target_dim** (*int*) – Dimensionality of the target vector (number of entities).
- **torch_ordered_shaped_bpe_entities** (*torch.LongTensor*) – Ordered BPE entity representations.

train_set

train_indices_target

target_dim

num_datapoints

torch_ordered_shaped_bpe_entities

collate_fn = None

`__len__`()

`__getitem__`(*idx*)

dicee.dataset_classes.**construct_dataset**(** (Keyword-only parameters separator (PEP 3102)), train_set: numpy.ndarray | list, valid_set=None, test_set=None, ordered_bpe_entities=None, train_target_indices=None, target_dim: int = None, entity_to_idx: dict, relation_to_idx: dict, form_of_labelling: str, scoring_technique: str, neg_ratio: int, label_smoothing_rate: float, byte_pair_encoding=None, block_size: int = None, seed: int = None, sort_train_set: bool = True*) → torch.utils.data.Dataset

Build the appropriate dataset for the given training configuration.

Parameters

- **train_set** (*numpy.ndarray or list*) – Raw integer-indexed triples.
- **entity_to_idx** (*dict*) – Name → index mappings.
- **relation_to_idx** (*dict*) – Name → index mappings.
- **form_of_labelling** (*str*) – `'EntityPrediction'` or `'RelationPrediction'`.
- **scoring_technique** (*str*) – One of `'NegSample'`, `'FixedNegSample'`, `'1vsAll'`, `'1vsSample'`, `'KvsAll'`, `'AllvsAll'`, `'KvsSample'`.
- **neg_ratio** (*int*) – Negative sample ratio.

- **label_smoothing_rate** (*float*) – Label smoothing coefficient.

Return type
torch.utils.data.Dataset

dicee.dataset_classes.**reload_dataset**(*path: str*, *form_of_labelling*, *scoring_technique*, *neg_ratio*, *label_smoothing_rate*)

Reload training data from disk and construct a Pytorch dataset.

class dicee.dataset_classes.**AllvsAll**(*train_set_idx: numpy.ndarray*, *entity_idxs*, *relation_idxs*, *label_smoothing_rate=0.0*)

Bases: `torch.utils.data.Dataset`

Dataset for AllvsAll training (multi-label, exhaustive).

Extends the `KvsAll` idea: every *possible* `(entity, relation)` combination is included — not just those observed in the KG. Pairs without any known tail entities receive an all-zeros label vector.

Parameters

- **train_set_idx** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.
- **entity_idxs** (*dict*) – Entity-name → index mapping.
- **relation_idxs** (*dict*) – Relation-name → index mapping.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default `0.0`).

train_data = None

train_target = None

label_smoothing_rate

collate_fn = None

target_dim

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**FSDP1vsSampleDataset**(*train_set_idx: numpy.ndarray*, *entity_idxs*, *relation_idxs*, *form*, *neg_ratio=None*, *label_smoothing_rate: float = 0.0*)

Bases: `torch.utils.data.Dataset`

Positive-triple dataset for FSDP 1vsSample training with true-negative sampling.

Each dataset item is a single positive triple (h, r, t). The collate_fn builds the full (source, target_idx, labels) batch in a DataLoader worker, sampling true negatives via index remapping so they never coincide with any known positive tail for that (h, r) pair.

Fixed batch width follows KvsSample:
max_num_of_classes = max_positives_per_pair + neg_ratio

Each sample contributes 1 positive and (max_num_of_classes - 1) negatives.

train_data

num_entities

num_relations

neg_ratio = None

label_smoothing_rate = 0.0

max_num_of_classes

num_negatives

collate_fn

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**KvsAll**(*train_set_idx: numpy.ndarray*, *entity_idxs*, *relation_idxs*, *form*, *store=None*, *label_smoothing_rate: float = 0.0*)

Bases: `torch.utils.data.Dataset`

Dataset for KvsAll training (multi-label).

D := {(x, y)_i}_{i=1}^{N} where

- x = (h, r) is a unique (entity, relation) pair observed in the KG,
- y ∈ [0, 1]^{**|E|**} is a multi-label vector with y_j = 1 iff (h, r, e_j) ∈ KG.

Parameters

- **train_set_idx** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.
- **entity_idxs** (*dict*) – Entity-name → index mapping.
- **relation_idxs** (*dict*) – Relation-name → index mapping.
- **form** (*str*) – `'EntityPrediction'` or `'RelationPrediction'`.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default `0.0`).

train_data = None

train_target = None

label_smoothing_rate

collate_fn = None

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**KvsSampleDataset**(*train_set_idx: numpy.ndarray*, *entity_idxs*, *relation_idxs*, *form*, *store=None*, *neg_ratio=None*, *label_smoothing_rate: float = 0.0*)

Bases: `torch.utils.data.Dataset`

Dataset for KvsSample training (dynamic multi-label).

Like `KvsAll` but sub-samples the target vector at each access to keep mini-batch sizes manageable when the entity set is large.

Parameters

- **train_set_idx** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.

- **entity_idxs** (*dict*) – Entity-name → index mapping.
- **relation_idxs** (*dict*) – Relation-name → index mapping.
- **form** (*str*) – 'EntityPrediction'.
- **neg_ratio** (*int*) – Number of negative samples per positive target.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

train_data = None

train_target = None

neg_ratio = None

num_entities

label_smoothing_rate

collate_fn = None

max_num_of_classes

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**OnevsAllDataset**(*train_set_idx: numpy.ndarray*, *entity_idxs*)

Bases: torch.utils.data.Dataset

Dataset for the 1-vs-All training strategy (multi-class).

Each sample is a (head, relation) pair with a one-hot target vector whose single active position corresponds to the true tail entity.

Parameters

- **train_set_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **entity_idxs** (*dict*) – Entity-name → index mapping (used to determine the target dimension).

train_data

target_dim

collate_fn = None

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**LiteralDataset**(*file_path: str*, *ent_idx: dict = None*, *normalization_type: str = 'z-norm'*, *sampling_ratio: float = None*, *loader_backend: str = 'pandas'*)

Bases: torch.utils.data.Dataset

Dataset for loading and processing literal data for Literal Embedding models.

Handles loading, normalization, and preparation of (entity, attribute, value) triples. Supports z-score and min-max normalization as well as optional sub-sampling for ablation studies.

Parameters

- **file_path** (*str*) – Path to the training data file (CSV / TSV / RDF).
- **ent_idx** (*dict*) – Entity-name → index mapping.
- **normalization_type** (*str, optional*) – 'z-norm', 'min-max', or None (default 'z-norm').
- **sampling_ratio** (*float or None, optional*) – Fraction of the training set to keep (default None = use all).
- **loader_backend** (*str, optional*) – 'pandas' or 'rdflib' (default 'pandas').

train_file_path

loader_backend = 'pandas'

normalization_type = 'z-norm'

normalization_params

sampling_ratio = None

entity_to_idx = None

num_entities

__getitem__(*index*)

__len__()

static load_and_validate_literal_data(*file_path: str = None*, *loader_backend: str = 'pandas'*) → pandas.DataFrame

Load and validate a literal data file.

Parameters

- **file_path** (*str*) – Path to the data file.
- **loader_backend** (*str*) – 'pandas' or 'rdflib'.

Returns

Three-column DataFrame with columns head, attribute, value.

Return type

pandas.DataFrame

static denormalize(*preds_norm*, *attributes*, *normalization_params*) → numpy.ndarray

Reverse the normalization applied during training.

Parameters

- **preds_norm** (*numpy.ndarray*) – Normalized predictions.
- **attributes** (*list*) – Attribute names corresponding to each prediction.
- **normalization_params** (*dict*) – Parameters stored during training.

Returns

Denormalized predictions.

Return type

numpy.ndarray

class dicee.dataset_classes.**FixedNegSampleDataset**(*train_set: numpy.ndarray*, *num_entities: int*, *num_relations: int*, *neg_sample_ratio: int = 1*, *label_smoothing_rate: float = 0.0*, *seed: int = None*)

Bases: `torch.utils.data.Dataset`

Pre-computed (fixed) negative sampling dataset.

At construction time every positive triple is paired with one random negative (head- or tail-corrupted) using vectorized operations for efficiency. The pairs are stored so that `__getitem__` is a simple lookup.

This is useful when you want deterministic negatives across epochs (e.g., for reproducibility or debugging).

Parameters

- **train_set** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int, optional*) – Number of negative samples per positive triple (default `1`).
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default `0.0`).

neg_sample_ratio = 1

num_entities

num_relations

label_smoothing_rate = 0.0

collate_fn = None

seed = None

train_triples

length

__len__() → int

__getitem__(*idx: int*) → Tuple[torch.Tensor, torch.Tensor]

class dicee.dataset_classes.**OnevsSample**(*train_set: numpy.ndarray*, *num_entities: int*, *num_relations: int*, *neg_sample_ratio: int = None*, *label_smoothing_rate: float = 0.0*)

Bases: `torch.utils.data.Dataset`

Dataset for 1-vs-Sample training (dynamic multi-class with negatives).

For every positive triple `(h, r, t)` the dataset draws `neg_sample_ratio` random entities as negatives and returns a label vector that marks the true tail and the negatives.

Parameters

- **train_set** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int*) – Number of negative samples per positive.

- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default `0.0`).

train_data

num_entities

num_relations

neg_sample_ratio = None

label_smoothing_rate

collate_fn = None

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**TriplePredictionDataset**(*train_set: numpy.ndarray, num_entities: int, num_relations: int, neg_sample_ratio: int = 1, label_smoothing_rate: float = 0.0, seed: int = None, sort_train_set: bool = True*)

Bases: `torch.utils.data.Dataset`

Dataset for triple prediction with on-the-fly negative sampling.

Each item is a single positive triple; the custom `collate_fn` generates a batch of mixed positive and negative triples.

Parameters

- **train_set** (*numpy.ndarray*) – `(N, 3)` integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int, optional*) – Number of negative samples per positive triple (default `1`).
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default `0.0`).

label_smoothing_rate

neg_sample_ratio

seed = None

length

num_entities

num_relations

__len__()

__getitem__(*idx*)

collate_fn(*batch: List[torch.Tensor]*)

dicee.eval_static_funcs

Static evaluation functions for KGE models.

This module provides backward compatibility by re-exporting from the new dicee.evaluation module.

Deprecated since version Use: `dicee.evaluation` submodules instead. This module will be removed in a future version.

Functions

evaluate_ensemble_link_prediction_performa	Evaluate link prediction performance of an ensemble of KGE models.
evaluate_link_prediction_performance(→ Dict[str, float])	Evaluate link prediction performance with head and tail prediction.
evaluate_link_prediction_performance_with_l	Evaluate link prediction with BPE encoding (head and tail).
evaluate_link_prediction_performance_with_l	Evaluate link prediction with BPE encoding and reciprocals.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with reciprocal relations.
evaluate_lp_bpe_k_vs_all(→ Dict[str, float])	Evaluate BPE link prediction with KvsAll scoring.
evaluate_literal_prediction(→ Optional[pandas.DataFrame])	Evaluate trained literal prediction model on a test file.

Module Contents

dicee.eval_static_funcs.**evaluate_ensemble_link_prediction_performance**(*models: List*, *triples*, *er_vocab: Dict[Tuple, List]*, *weights: List[float] | None = None*, *batch_size: int = 512*, *weighted_averaging: bool = True*, *normalize_scores: bool = True*) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if weighted_averaging is True. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize_scores** – If True, normalize scores to [0, 1] range per sample before averaging.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If weighted_averaging is True but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

dicee.eval_static_funcs.**evaluate_link_prediction_performance**(*model*, *triples*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.eval_static_funcs.**evaluate_link_prediction_performance_with_bpe**(*model*, *within_entities: List[str]*, *triples: List[Tuple[str]]*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.eval_static_funcs.**evaluate_link_prediction_performance_with_bpe_reciprocals**(*model*, *within_entities: List[str]*, *triples: List[List[str]]*, *er_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.eval_static_funcs.`**`evaluate_link_prediction_performance_with_reciprocals`**`(`*model*, *triples*, *er_vocab: Dict[Tuple, List]*`)` → Dict[str, float]

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **`model`** – KGE model wrapper.
- **`triples`** – Test triples as list of (head, relation, tail) strings.
- **`er_vocab`** – Mapping (entity, relation) -> list of valid tail entities.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.eval_static_funcs.`**`evaluate_lp_bpe_k_vs_all`**`(`*model*, *triples: List[List[str]]*, *er_vocab: Dict | None = None*, *batch_size: int | None = None*, *func_triple_to_bpe_representation: Callable | None = None*, *str_to_bpe_entity_to_idx: Dict | None = None*`)` → Dict[str, float]

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **`model`** – The KGE model wrapper.
- **`triples`** – List of string triples.
- **`er_vocab`** – Entity-relation vocabulary for filtering.
- **`batch_size`** – Batch size for processing.
- **`func_triple_to_bpe_representation`** – Function to convert triples to BPE.
- **`str_to_bpe_entity_to_idx`** – Mapping from string entities to BPE indices.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises
`ValueError` – If batch_size is not provided.

`dicee.eval_static_funcs.`**`evaluate_literal_prediction`**`(`*kge_model*, *eval_file_path: str = None*, *store_lit_preds: bool = True*, *eval_literals: bool = True*, *loader_backend: str = 'pandas'*, *return_attr_error_metrics: bool = False*`)` → pandas.DataFrame | None

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **`kge_model`** – Trained KGE model with literal prediction capability.
- **`eval_file_path`** – Path to the evaluation file containing test literals.
- **`store_lit_preds`** – If True, stores predictions to CSV file.
- **`eval_literals`** – If True, evaluates and prints error metrics.
- **`loader_backend`** – Backend for loading dataset ('pandas' or 'rdflib').

- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns

DataFrame with per-attribute MAE and RMSE if return_attr_error_metrics is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

dicee.evaluation

Evaluation module for knowledge graph embedding models.

This module provides comprehensive evaluation capabilities for KGE models, including link prediction, literal prediction, and ensemble evaluation.

Modules:

link_prediction: Functions for evaluating link prediction performance literal_prediction: Functions for evaluating literal/attribute prediction ensemble: Functions for ensemble model evaluation evaluator: Main Evaluator class for integrated evaluation utils: Shared utility functions for evaluation _filtering: Internal helpers for filtered ranking (not exported)

Example

```
>>> from dicee.evaluation import Evaluator
>>> from dicee.evaluation.link_prediction import evaluate_link_prediction_performance
>>> from dicee.evaluation.ensemble import evaluate_ensemble_link_prediction_
↪performance
```

Submodules

dicee.evaluation.ensemble

Ensemble evaluation functions.

This module provides functions for evaluating ensemble models, including weighted averaging and score normalization.

Functions

evaluate_ensemble_link_prediction_performa	Evaluate link prediction performance of an ensemble of KGE models.

Module Contents

dicee.evaluation.ensemble.**evaluate_ensemble_link_prediction_performance**(*models: List*, *triples*, *er_vocab: Dict[Tuple, List]*, *weights: List[float] | None = None*, *batch_size: int = 512*, *weighted_averaging: bool = True*, *normalize_scores: bool = True*) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if weighted_averaging is True. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize_scores** – If True, normalize scores to [0, 1] range per sample before averaging.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If weighted_averaging is True but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

dicee.evaluation.evaluator

Main Evaluator class for KGE model evaluation.

This module provides the Evaluator class which orchestrates evaluation of knowledge graph embedding models across different datasets and scoring techniques.

Attributes

logger	
VALID_SCORING_TECHNIQUES	

Classes

Evaluator	Evaluator class for KGE models in various downstream tasks.

Module Contents

`dicee.evaluation.evaluator.`**`logger`**

`dicee.evaluation.evaluator.`**`VALID_SCORING_TECHNIQUES`**

`class` `dicee.evaluation.evaluator.`**`Evaluator`**(*args*, *is_continual_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

`er_vocab`

Entity-relation to tail vocabulary for filtered ranking.

`re_vocab`

Relation-entity (tail) to head vocabulary.

`ee_vocab`

Entity-entity to relation vocabulary.

`num_entities`

Total number of entities in the knowledge graph.

`num_relations`

Total number of relations in the knowledge graph.

`args`

Configuration arguments.

`report`

Dictionary storing evaluation results.

`during_training`

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation(*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval(*dataset*, *trained_model*, *form_of_labelling: str*, *during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity(**, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*) → None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity(*, *train_set=None*, *valid_set=None*, *test_set=None*, *ordered_bpe_entities*, *trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate BytE model with generation.

eval_with_bpe_vs_all(**, raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all(**, train_set, valid_set=None, test_set=None, trained_model, form_of_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all(*model, triple_idx, info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte(*model, triples: List[List[str]], info: str | None = None*) → Dict[str, float]

Evaluate BytE model with text generation.

Parameters

- **model** – BytE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all(*model, triples: List[List[str]], info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp(*model, triple_idx, info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dummy_eval(*trained_model*, *form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data(*dataset*, *trained_model*, *triple_idx: numpy.ndarray*, *form_of_labelling: str*) → Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

dicee.evaluation.link_prediction

Link prediction evaluation functions.

This module provides various functions for evaluating link prediction performance of knowledge graph embedding models.

Attributes

logger

Functions

evaluate_link_prediction_performance(→ Dict[str, float])	Evaluate link prediction performance with head and tail prediction.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with reciprocal relations.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with BPE encoding and reciprocals.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with BPE encoding (head and tail).
evaluate_lp(→ Dict[str, float])	Evaluate link prediction with batched processing.
evaluate_bpe_lp(→ Dict[str, float])	Evaluate link prediction with BPE-encoded entities.
evaluate_lp_bpe_k_vs_all(→ Dict[str, float])	Evaluate BPE link prediction with KvsAll scoring.

Module Contents

`dicee.evaluation.link_prediction.`**`logger`**

`dicee.evaluation.link_prediction.`**`evaluate_link_prediction_performance`**(*model*, *triples*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **`model`** – KGE model wrapper with entity/relation mappings.
- **`triples`** – Test triples as list of (head, relation, tail) strings.
- **`er_vocab`** – Mapping (entity, relation) -> list of valid tail entities.
- **`re_vocab`** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_link_prediction_performance_with_reciprocals`**(*model*, *triples*, *er_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **`model`** – KGE model wrapper.
- **`triples`** – Test triples as list of (head, relation, tail) strings.
- **`er_vocab`** – Mapping (entity, relation) -> list of valid tail entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_link_prediction_performance_with_bpe_reciprocals`**(*model*, *within_entities: List[str]*, *triples: List[List[str]]*, *er_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **`model`** – KGE model wrapper with BPE support.
- **`within_entities`** – List of entities to evaluate within.
- **`triples`** – Test triples as list of [head, relation, tail] strings.
- **`er_vocab`** – Mapping (entity, relation) -> list of valid tail entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_link_prediction_performance_with_bpe`**(*model*, *within_entities: List[str]*, *triples: List[Tuple[str]]*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_lp`**(*model*, *triple_idx*, *num_entities: int*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*, *batch_size: int = 128*, *chunk_size: int = 1000*) → Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_bpe_lp`**(*model*, *triple_idx: List[Tuple]*, *all_bpe_shaped_entities*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*) → Dict[str, float]

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.link_prediction.`**`evaluate_lp_bpe_k_vs_all`**(*model*, *triples: List[List[str]]*, *er_vocab: Dict | None = None*, *batch_size: int | None = None*, *func_triple_to_bpe_representation: Callable | None = None*, *str_to_bpe_entity_to_idx: Dict | None = None*) → Dict[str, float]

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er_vocab** – Entity-relation vocabulary for filtering.
- **batch_size** – Batch size for processing.
- **func_triple_to_bpe_representation** – Function to convert triples to BPE.
- **str_to_bpe_entity_to_idx** – Mapping from string entities to BPE indices.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

ValueError – If batch_size is not provided.

dicee.evaluation.literal_prediction

Literal prediction evaluation functions.

This module provides functions for evaluating literal/attribute prediction performance of knowledge graph embedding models.

Attributes

logger

Functions

evaluate_literal_prediction(→ Optional[pandas.DataFrame])	Evaluate trained literal prediction model on a test file.

Module Contents

dicee.evaluation.literal_prediction.**logger**

dicee.evaluation.literal_prediction.**evaluate_literal_prediction**(*kge_model*, *eval_file_path: str = None*, *store_lit_preds: bool = True*, *eval_literals: bool = True*, *loader_backend: str = 'pandas'*, *return_attr_error_metrics: bool = False*) → pandas.DataFrame | None

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **kge_model** – Trained KGE model with literal prediction capability.
- **eval_file_path** – Path to the evaluation file containing test literals.

- **store_lit_preds** – If True, stores predictions to CSV file.
- **eval_literals** – If True, evaluates and prints error metrics.
- **loader_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns
DataFrame with per-attribute MAE and RMSE if return_attr_error_metrics is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

dicee.evaluation.utils

Utility functions for evaluation module.

This module contains shared helper functions used across different evaluation components.

Attributes

DEFAULT_HITS_RANGE
ALL_HITS_RANGE

Functions

make_iterable_verbose(→ Iterable)	Wrap an iterable with tqdm progress bar if verbose is True.
compute_metrics_from_ranks(→ Dict[str, float])	Compute standard link prediction metrics from ranks.
compute_metrics_from_ranks_simple(→ Dict[str, float])	Compute link prediction metrics without scaling factor.
update_hits(→ None)	Update hits dictionary based on rank.
create_hits_dict(→ Dict[int, List[float]])	Create an initialized hits dictionary.
efficient_zero_grad(→ None)	Efficiently zero gradients using parameter.grad = None.

Module Contents

dicee.evaluation.utils.**DEFAULT_HITS_RANGE: List[int] = [1, 3, 10]**

dicee.evaluation.utils.**ALL_HITS_RANGE: List[int]**

dicee.evaluation.utils.**make_iterable_verbose**(*iterable_object: Iterable*, *verbose: bool*, *desc: str = 'Default'*, *position: int | None = None*, *leave: bool = True*) → Iterable

Wrap an iterable with tqdm progress bar if verbose is True.

Parameters

- **iterable_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

Returns

The original iterable or a tqdm-wrapped version.

dicee.evaluation.utils.**compute_metrics_from_ranks**(*ranks: List[int]*, *num_triples: int*, *hits_dict: Dict[int, List[float]]*, *scale_factor: int = 1*) → Dict[str, float]

Compute standard link prediction metrics from ranks.

Parameters

- **ranks** – List of ranks for each prediction.
- **num_triples** – Total number of triples evaluated.
- **hits_dict** – Dictionary mapping hit levels to lists of hits.
- **scale_factor** – Factor to scale the denominator (e.g., 2 for head+tail).

Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.utils.**compute_metrics_from_ranks_simple**(*ranks: List[int]*, *num_triples: int*, *hits_dict: Dict[int, List[float]]*) → Dict[str, float]

Compute link prediction metrics without scaling factor.

Parameters

- **ranks** – List of ranks for each prediction.
- **num_triples** – Total number of triples evaluated.
- **hits_dict** – Dictionary mapping hit levels to lists of hits.

Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.utils.**update_hits**(*hits: Dict[int, List[float]]*, *rank: int*, *hits_range: List[int] | None = None*) → None

Update hits dictionary based on rank.

Parameters

- **hits** – Dictionary to update in-place.
- **rank** – The rank to check against hit levels.

- **hits_range** – List of hit levels to check (default: ALL_HITS_RANGE).

dicee.evaluation.utils.**create_hits_dict**(*hits_range: List[int] | None = None*) → Dict[int, List[float]]

Create an initialized hits dictionary.

Parameters
hits_range – List of hit levels to initialize (default: ALL_HITS_RANGE).

Returns
Dictionary with empty lists for each hit level.

dicee.evaluation.utils.**efficient_zero_grad**(*model*) → None

Efficiently zero gradients using parameter.grad = None.

This is more efficient than optimizer.zero_grad() as it avoids memory operations.

See: https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html

Parameters
model – PyTorch model to zero gradients for.

Classes

Evaluator	Evaluator class for KGE models in various downstream tasks.

Functions

evaluate_ensemble_link_prediction_performa	Evaluate link prediction performance of an ensemble of KGE models.
evaluate_bpe_lp(→ Dict[str, float])	Evaluate link prediction with BPE-encoded entities.
evaluate_link_prediction_performance(→ Dict[str, float])	Evaluate link prediction performance with head and tail prediction.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with BPE encoding (head and tail).
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with BPE encoding and reciprocals.
*evaluate_link_prediction_performance_with_*	Evaluate link prediction with reciprocal relations.
evaluate_lp(→ Dict[str, float])	Evaluate link prediction with batched processing.
evaluate_lp_bpe_k_vs_all(→ Dict[str, float])	Evaluate BPE link prediction with KvsAll scoring.
evaluate_literal_prediction(→ Optional[pandas.DataFrame])	Evaluate trained literal prediction model on a test file.

Package Contents

dicee.evaluation.**evaluate_ensemble_link_prediction_performance**(*models: List*, *triples*, *er_vocab: Dict[Tuple, List]*, *weights: List[float] | None = None*, *batch_size: int = 512*, *weighted_averaging: bool = True*, *normalize_scores: bool = True*) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if weighted_averaging is True. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize_scores** – If True, normalize scores to [0, 1] range per sample before averaging.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If weighted_averaging is True but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

class dicee.evaluation.**Evaluator**(*args*, *is_continual_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab

Entity-relation to tail vocabulary for filtered ranking.

re_vocab

Relation-entity (tail) to head vocabulary.

ee_vocab

Entity-entity to relation vocabulary.

num_entities

Total number of entities in the knowledge graph.

num_relations

Total number of relations in the knowledge graph.

args

Configuration arguments.

report

Dictionary storing evaluation results.

during_training

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation(*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval(*dataset*, *trained_model*, *form_of_labelling: str*, *during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity(*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*) → None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity(*, *train_set=None*, *valid_set=None*, *test_set=None*, *ordered_bpe_entities*, *trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate BytE model with generation.

eval_with_bpe_vs_all(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all(*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all(*model*, *triple_idx*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte(*model*, *triples: List[List[str]]*, *info: str | None = None*) → Dict[str, float]

Evaluate BytE model with text generation.

Parameters

- **model** – BytE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all(*model*, *triples: List[List[str]]*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp(*model*, *triple_idx*, *info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

dummy_eval(*trained_model*, *form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data(*dataset*, *trained_model*, *triple_idx: numpy.ndarray*, *form_of_labelling: str*) → Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns
Dictionary with evaluation metrics.

Raises
ValueError – If scoring technique is invalid.

dicee.evaluation.**evaluate_bpe_lp**(*model*, *triple_idx: List[Tuple]*, *all_bpe_shaped_entities*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*) → Dict[str, float]

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_link_prediction_performance**(*model*, *triples*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_link_prediction_performance_with_bpe**(*model*, *within_entities: List[str]*, *triples: List[Tuple[str]]*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_link_prediction_performance_with_bpe_reciprocals**(*model*, *within_entities: List[str]*, *triples: List[List[str]]*, *er_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_link_prediction_performance_with_reciprocals**(*model*, *triples*, *er_vocab: Dict[Tuple, List]*) → Dict[str, float]

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.

- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_lp**(*model*, *triple_idx*, *num_entities: int*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*, *batch_size: int = 128*, *chunk_size: int = 1000*) → Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.evaluation.**evaluate_lp_bpe_k_vs_all**(*model*, *triples: List[List[str]]*, *er_vocab: Dict | None = None*, *batch_size: int | None = None*, *func_triple_to_bpe_representation: Callable | None = None*, *str_to_bpe_entity_to_idx: Dict | None = None*) → Dict[str, float]

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er_vocab** – Entity-relation vocabulary for filtering.
- **batch_size** – Batch size for processing.
- **func_triple_to_bpe_representation** – Function to convert triples to BPE.
- **str_to_bpe_entity_to_idx** – Mapping from string entities to BPE indices.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises
ValueError – If batch_size is not provided.

dicee.evaluation.**evaluate_literal_prediction**(*kge_model*, *eval_file_path: str = None*, *store_lit_preds: bool = True*, *eval_literals: bool = True*, *loader_backend: str = 'pandas'*, *return_attr_error_metrics: bool = False*) → pandas.DataFrame | None

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **kge_model** – Trained KGE model with literal prediction capability.
- **eval_file_path** – Path to the evaluation file containing test literals.
- **store_lit_preds** – If True, stores predictions to CSV file.
- **eval_literals** – If True, evaluates and prints error metrics.
- **loader_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns

DataFrame with per-attribute MAE and RMSE if return_attr_error_metrics is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

dicee.evaluator

Evaluator module for knowledge graph embedding models.

This module provides backward compatibility by re-exporting from the new dicee.evaluation module.

Deprecated since version Use: `dicee.evaluation.Evaluator` instead. This module will be removed in a future version.

Classes

Evaluator	Evaluator class for KGE models in various downstream tasks.

Module Contents

class dicee.evaluator.**Evaluator**(*args*, *is_continual_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab
: Entity-relation to tail vocabulary for filtered ranking.

re_vocab
: Relation-entity (tail) to head vocabulary.

ee_vocab
: Entity-entity to relation vocabulary.

num_entities
: Total number of entities in the knowledge graph.

num_relations
: Total number of relations in the knowledge graph.

args
: Configuration arguments.

report
: Dictionary storing evaluation results.

during_training
: Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation(*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval(*dataset*, *trained_model*, *form_of_labelling: str*, *during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity(**, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*) → None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity(*, *train_set=None*, *valid_set=None*, *test_set=None*, *ordered_bpe_entities*, *trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate BytE model with generation.

eval_with_bpe_vs_all(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all(*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all(*model*, *triple_idx*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte(*model*, *triples: List[List[str]]*, *info: str | None = None*) → Dict[str, float]

Evaluate BytE model with text generation.

Parameters

- **model** – BytE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all(*model*, *triples: List[List[str]]*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp(*model*, *triple_idx*, *info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dummy_eval(*trained_model*, *form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data(*dataset*, *trained_model*, *triple_idx: numpy.ndarray*, *form_of_labelling: str*) → Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns
Dictionary with evaluation metrics.

Raises
`ValueError` – If scoring technique is invalid.

dicee.executer

Executor module for training, retraining and evaluating KGE models.

This module provides the Execute and ContinuousExecute classes for managing the full lifecycle of knowledge graph embedding model training.

Attributes

logger	

Classes

Execute	Executor class for training, retraining and evaluating KGE models.
ContinuousExecute	A subclass of Execute Class for retraining

Module Contents

dicee.executer.**logger**

class dicee.executer.**Execute**(*args*, *continuous_training: bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

args

Processed input arguments.

distributed

Whether distributed training is enabled.

rank

Process rank in distributed training.

world_size

Total number of processes.

local_rank

Local GPU rank.

trainer

Training handler instance.

trained_model

The trained model after training completes.

knowledge_graph
: The loaded knowledge graph.

report
: Dictionary storing training metrics and results.

evaluator
: Model evaluation handler.

distributed

rank

world_size

local_rank

args

is_continual_training = False

trainer: *dicee.trainer.DICE_Trainer* **| None = None**

trained_model = None

knowledge_graph: *dicee.knowledge_graph.KG* **| None = None**

report: Dict

evaluator: *dicee.evaluator.Evaluator* **| None = None**

start_time: float | None = None

is_local_rank_zero() → bool

is_global_rank_zero() → bool

cleanup()

setup_executor() → None

Set up storage directories for the experiment.

Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

create_and_store_kg() → None

Create knowledge graph and store as memory-mapped file.

Only executed on global rank 0 in distributed training, so exactly one process writes the shared memmap/vocab files regardless of how many nodes are involved. Skips if memmap already exists.

load_from_memmap() → None

Load knowledge graph from memory-mapped file.

save_trained_model() → None

Save a knowledge graph embedding model

(1) Send model to eval mode and cpu.

(2) Store the memory footprint of the model.

(3) Save the model into disk.

(4) Update the stats of KG again ?

Parameter

rtype
None

`end(form_of_labelling: str)` → dict

End training

(1) Store trained model.

(2) Report runtimes.

(3) Eval model if required.

Parameter

rtype
A dict containing information about the training and/or evaluation

`write_report()` → None

Report training related information in a report.json file

`start()` → dict

Start training

(1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

Parameter

rtype
A dict containing information about the training and/or evaluation

`class dicee.executer.ContinuousExecute(args)`

Bases: `Execute`

A subclass of Execute Class for retraining

(1) Loading & Preprocessing & Serializing input data.

(2) Training & Validation & Testing

(3) Storing all necessary info

During the continual learning we can only modify *** num_epochs *** parameter. Trained model stored in the same folder as the seed model for the training. Trained model is noted with the current time.

`continual_start()` → dict

Start Continual Training

(1) Initialize training.

(2) Start continual training.

(3) Save trained model.

Parameter

rtype
A dict containing information about the training and/or evaluation

dicee.knowledge_graph

Knowledge Graph module for data loading and preprocessing.

Provides the KG class for handling knowledge graph data including loading, preprocessing, and indexing operations.

Classes

KG	Knowledge Graph container and processor.

Module Contents

class dicee.knowledge_graph.**KG**(*dataset_dir: str | None = None*, *byte_pair_encoding: bool = False*, *padding: bool = False*, *add_noise_rate: float | None = None*, *sparql_endpoint: str | None = None*, *path_single_kg: str | None = None*, *path_for_deserialization: str | None = None*, *add_reciprocal: bool | None = None*, *eval_model: str | None = None*, *read_only_few: int | None = None*, *sample_triples_ratio: float | None = None*, *path_for_serialization: str | None = None*, *entity_to_idx: Dict | None = None*, *relation_to_idx: Dict | None = None*, *backend: str | None = None*, *training_technique: str | None = None*, *separator: str | None = None*)

Knowledge Graph container and processor.

Handles loading, preprocessing, and indexing of knowledge graph data from various sources including files, SPARQL endpoints, and serialized formats.

dataset_dir
Path to directory containing train/valid/test files.

num_entities
Total number of unique entities.

num_relations
Total number of unique relations.

train_set
Indexed training triples as numpy array.

valid_set
Indexed validation triples (optional).

test_set
Indexed test triples (optional).

entity_to_idx
Mapping from entity strings to indices.

relation_to_idx
Mapping from relation strings to indices.

dataset_dir = None

```
sparql_endpoint = None

path_single_kg = None

byte_pair_encoding = False

ordered_shaped_bpe_tokens = None

add_noise_rate = None

num_entities: int | None = None

num_relations: int | None = None

path_for_deserialization = None

add_reciprocal = None

eval_model = None

read_only_few = None

sample_triples_ratio = None

path_for_serialization = None

entity_to_idx = None

relation_to_idx = None

backend = 'pandas'

training_technique = None

separator = None

raw_train_set = None

raw_valid_set = None

raw_test_set = None

train_set = None

valid_set = None

test_set = None

idx_entity_to_bpe_shaped: Dict

enc

num_tokens

num_bpe_entities: int | None = None

padding = False

dummy_id

max_length_subword_tokens: int | None = None
```

train_set_target = None

target_dim: int | None = None

train_target_indices = None

ordered_bpe_entities = None

description_of_input = None

describe() → None

Generate a description string of the dataset statistics.

property entities_str: List[str]

Get list of all entity strings.

property relations_str: List[str]

Get list of all relation strings.

exists(*h: str*, *r: str*, *t: str*) → bool

Check if a triple exists in the training set.

Parameters

- **h** – Head entity string.
- **r** – Relation string.
- **t** – Tail entity string.

Returns

True if the triple exists, False otherwise.

__iter__() → Iterator[Tuple[str, str, str]]

Iterate over training triples as string tuples.

__len__() → int

Return number of triples in the raw training set.

func_triple_to_bpe_representation(*triple: List[str]*)

dicee.knowledge_graph_embeddings

Attributes

logger

Classes

KGE	Knowledge Graph Embedding Class for interactive usage of pre-trained models

Module Contents

`dicee.knowledge_graph_embeddings.logger`

class `dicee.knowledge_graph_embeddings.`**KGE**(*path=None*, *url=None*, *construct_ensemble=False*, *model_name=None*)

Bases: `dicee.abstracts.BaseInteractiveKGE`, `dicee.abstracts.InteractiveQueryDecomposition`, `dicee.abstracts.BaseInteractiveTrainKGE`

Knowledge Graph Embedding Class for interactive usage of pre-trained models

__str__()

to(*device: str*) → None

get_transductive_entity_embeddings(*indices: torch.LongTensor | List[str]*, *as_pytorch=False*, *as_numpy=False*, *as_list=True*) → torch.FloatTensor | numpy.ndarray | List[float]

create_vector_database(*collection_name: str*, *distance: str*, *location: str = 'localhost'*, *port: int = 6333*)

generate(*h=''*, *r=''*)

eval_lp_performance(*dataset=List[Tuple[str, str, str]]*, *filtered=True*)

predict_missing_head_entity(*relation: List[str] | str*, *tail_entity: List[str] | str*, *within=None*, *batch_size=2*, *topk=1*, *return_indices=False*) → Tuple

Given a relation and a tail entity, return top k ranked head entity.

argmax_{e in E } f(e,r,t), where r in R, t in E.

Parameter

relation: Union[List[str], str]

String representation of selected relations.

tail_entity: Union[List[str], str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_relations(*head_entity: List[str] | str*, *tail_entity: List[str] | str*, *within=None*, *batch_size=2*, *topk=1*, *return_indices=False*) → Tuple

Given a head entity and a tail entity, return top k ranked relations.

argmax_{r in R } f(h,r,t), where h, t in E.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_tail_entity(*head_entity: List[str] | str*, *relation: List[str] | str*, *within: List[str] = None*, *batch_size=2*, *topk=1*, *return_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

argmax_{e in E } f(h,r,e), where h in E and r in R.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

Returns: Tuple

scores

predict(**, *h: List[str] | str | None = None*, *r: List[str] | str | None = None*, *t: List[str] | str | None = None*, *within: List[str] | None = None*, *logits: bool = True*) → torch.FloatTensor

Predict scores for triples or missing triple elements.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **within** – Optional list of entities to restrict predictions to.
- **logits** – If True, return raw scores. If False, return sigmoid scores (0-1).

Returns

- Single triple (h, r, t): scalar score
- Missing element: vector of all possible scores

Return type

torch.FloatTensor of scores. Shape depends on the query type

Raises

- **TypeError** – If inputs are not strings or lists of strings.
- **ValueError** – If a required argument for the query type is missing.

Examples

```
>>> # Score a specific triple
>>> model.predict(h="Mongolia", r="isLocatedIn", t="Asia", logits=False)
tensor(0.9523)
```

```
>>> # Get scores for all possible tail entities
>>> model.predict(h="Mongolia", r="isLocatedIn", t=None)
tensor([0.21, 0.95, 0.03, ...])  # One score per entity
```

predict_topk(**, h: str | List[str] | None = None, r: str | List[str] | None = None, t: str | List[str] | None = None, topk: int = 10, within: List[str] | None = None, batch_size: int = 1024*) → List[Tuple[str, float]] | List[List[Tuple[str, float]]]

Predict top-k missing items in a given triple pattern.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **topk** – Number of top predictions to return.
- **within** – Optional list of entities to restrict predictions to.
- **batch_size** – Batch size for processing multiple queries.

Returns

List[(item, score), …] of length topk. For batch query: List of such lists, one per query.

Return type

For single query

Raises

- **TypeError** – If h, r, or t is not a str or list of str.
- **ValueError** – If the required arguments for a query type are None.

Examples

```
>>> model.predict_topk(h=["Mongolia"], r=["isLocatedIn"], topk=3)
[('Asia', 0.99), ('Europe', 0.02), ...]
```

```
>>> model.predict_topk(r=["isLocatedIn"], t=["Asia"], topk=5)
[('Mongolia', 0.85), ('China', 0.82), ...]
```

triple_score(*h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False*) → torch.FloatTensor

Predict triple score

Parameter

head_entity: List[str]

String representation of selected entities.

relation: List[str]

String representation of selected relations.

tail_entity: List[str]

String representation of selected entities.

logits: bool

If logits is True, unnormalized score returned

Returns: Tuple

pytorch tensor of triple score

return_multi_hop_query_results(*aggregated_query_for_all_entities*, *k: int*, *only_scores*)

single_hop_query_answering(*query: tuple*, *only_scores: bool = True*, *k: int = None*, *use_logits: bool = True*)

answer_multi_hop_query(*query_type: str | None = None*, *query: Tuple[str | Tuple[str, str], Ellipsis] | None = None*, *queries: List[Tuple[str | Tuple[str, str], Ellipsis]] | None = None*, *tnorm: str = 'prod'*, *neg_norm: str = 'standard'*, *lambda_: float = 0.0*, *k: int = 10*, *only_scores: bool = False*, *use_logits: bool = True*)
→ List[Tuple[str, torch.Tensor]] | List[List[Tuple[str, torch.Tensor]]]

Answer multi-hop EPFO (Existential Positive First-Order) queries.

Supports 9 query types: 1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up. See docs/guides/multi_hop_queries.md for detailed query patterns.

Parameters

- **query_type** – Query pattern name. One of: - 1p: (e, (r,)) # One-hop - 2p: (e, (r1, r2)) # Two-hop - 3p: (e, (r1, r2, r3)) # Three-hop - 2i: ((e1, (r1,)), (e2, (r2,))) # Two-way intersection - 3i: ((e1, (r1,)), (e2, (r2,)), (e3, (r3,))) # Three-way intersection - ip: (((e1, (r1,)), (e2, (r2,))), (r3,)) # Intersection + projection - pi: ((e, (r1, r2)), (r3,)) # Projection + intersection (2i meets 2p) - 2u: ((e1, (r1,)), (e2, (r2,))) # Two-way union - up: ((e, (r1, r2)), (e, (r3,))) # Union + projection
- **query** – Single query tuple matching the query_type pattern.
- **queries** – Batch of queries. If provided, query must be None.
- **tnorm** – T-norm for intersection/union. Options: "prod", "min".
- **neg_norm** – Negation norm. Options: "standard", "sugeno", "yager".
- **lambda** – Parameter for sugeno and yager negation (0.0-1.0).
- **k** – Number of top answer entities to return.
- **only_scores** – If True, return only scores tensor. If False, return (entity, score) tuples.
- **use_logits** – If True, use raw model logits. If False, use sigmoid probabilities.

Returns

List[(entity, score), …] of top-k answers. For batch queries: List of such lists, one per query.

Return type

For single query

Raises

- **ValueError** – If query_type is not in {1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up}.

- **AssertionError** – If query structure doesn't match query_type pattern.

Examples

```
>>> # 1p: Find entities located in Asia
>>> model.answer_multi_hop_query(
...     query_type="1p",
...     query=("Asia", ("isLocatedIn",)),
...     k=5
... )
[("Mongolia", 0.92), ("China", 0.89), ...]
```

```
>>> # 2p: Two-hop query (e.g., "capital of countries in Europe")
>>> model.answer_multi_hop_query(
...     query_type="2p",
...     query=("Europe", ("isLocatedIn", "hasCapital")),
...     k=3
... )
[("Paris", 0.85), ("Berlin", 0.82), ...]
```

```
>>> # 2i: Intersection query
>>> model.answer_multi_hop_query(
...     query_type="2i",
...     query=(("Asia", ("isLocatedIn",)), ("Mountains", ("hasGeography",))),
...     k=5
... )
[("Nepal", 0.78), ("Tibet", 0.65), ...]
```

See also

- docs/guides/multi_hop_queries.md: Complete guide with all query patterns
- tests/test_answer_multi_hop_query.py: Usage examples

find_missing_triples(*confidence: float*, *entities: List[str] = None*, *relations: List[str] = None*, *topk: int = 10*, *at_most: int = sys.maxsize*) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

orall e in E and orall r in R f(e,r,x)

Return (e,r,x)

otin G and f(e,r,x) > confidence

confidence: float

A threshold for an output of a sigmoid function given a triple.

topk: int

Highest ranked k item to select triples with f(e,r,x) > confidence .

at_most: int

Stop after finding at_most missing triples

{(e,r,x) | f(e,r,x) > confidence land (e,r,x)

otin G

predict_literals(*entity: List[str] | str = None*, *attribute: List[str] | str = None*, *denormalize_preds: bool = True*) → numpy.ndarray

Predicts literal values for given entities and attributes.

Parameters

- **entity** (*Union[List[str], str]*) – Entity or list of entities to predict literals for.
- **attribute** (*Union[List[str], str]*) – Attribute or list of attributes to predict literals for.
- **denormalize_preds** (*bool*) – If True, denormalizes the predictions.

Returns

Predictions for the given entities and attributes.

Return type

numpy ndarray

dicee.models

Submodules

dicee.models.adopt

ADOPT Optimizer Implementation.

This module implements the ADOPT (Adaptive Optimization with Precise Tracking) algorithm, an advanced optimization method for training neural networks.

ADOPT Overview:

ADOPT is an adaptive learning rate optimization algorithm that combines the benefits of momentum-based methods with per-parameter learning rate adaptation. Unlike Adam, which applies momentum to raw gradients, ADOPT normalizes gradients first and then applies momentum, leading to more stable training dynamics.

Key Features: - Gradient normalization before momentum application - Adaptive per-parameter learning rates - Optional gradient clipping that grows with training steps - Support for decoupled weight decay (AdamW-style) - Multiple execution modes: single-tensor, multi-tensor (foreach), and fused (planned)

Algorithm Comparison:

Adam: $m = \beta_1 * m + (1-\beta_1) * g$, $\theta = \theta - \alpha * m/\sqrt{v}$ ADOPT: $m = \beta_1 * m + (1-\beta_1) * g/\sqrt{v}$, $\theta = \theta - \alpha * m$

The key difference is that ADOPT normalizes gradients before momentum, which provides better stability and can lead to improved convergence.

Classes:

- ADOPT: Main optimizer class (extends torch.optim.Optimizer)

Functions:

- adopt: Functional API for ADOPT algorithm computation
- _single_tensor_adopt: Single-tensor implementation (TorchScript compatible)
- _multi_tensor_adopt: Multi-tensor implementation using foreach operations

Performance:

- Single-tensor: Default, compatible with torch.jit.script
- Multi-tensor (foreach): 2-3x faster on GPU through vectorization
- Fused (planned): Would provide maximum performance via specialized kernels

Example:

```
>>> import torch
>>> from dicee.models.adopt import ADOPT
>>>
>>> model = torch.nn.Linear(10, 1)
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01, decouple=True)
>>>
>>> # Training loop
>>> for epoch in range(num_epochs):
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     optimizer.step()
```

References:

Original implementation: https://github.com/iShohei220/adopt

Notes:

This implementation is based on the original ADOPT implementation and adapted to work with the PyTorch optimizer interface and the dicee framework.

Classes

ADOPT	ADOPT Optimizer.

Functions

adopt(params, grads, exp_avgs, exp_avg_sqs, state_steps)	Functional API that performs ADOPT algorithm computation.

Module Contents

class dicee.models.adopt.**ADOPT**(*params: torch.optim.optimizer.ParamsT*, *lr: float | torch.Tensor = 0.001*, *betas: Tuple[float, float] = (0.9, 0.9999)*, *eps: float = 1e-06*, *clip_lambda: Callable[[int], float] | None = lambda step: ...*, *weight_decay: float = 0.0*, *decouple: bool = False*, *, *foreach: bool | None = None*, *maximize: bool = False*, *capturable: bool = False*, *differentiable: bool = False*, *fused: bool | None = None*)

Bases: `torch.optim.optimizer.Optimizer`

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

Mathematical formulation:

m_t = β_1 * m_{t-1} + (1 - β_1) * clip(g_t / √(v_t)) v_t = β_2 * v_{t-1} + (1 - β_2) * g_t^2 θ_t = θ_{t-1} - α * m_t

where:

- θ_t: parameter at step t
- g_t: gradient at step t
- m_t: first moment estimate (momentum)
- v_t: second moment estimate (variance)
- α: learning rate
- β_1, β_2: exponential decay rates
- clip(): optional gradient clipping function

Reference:

Original implementation: https://github.com/iShohei220/adopt

Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.
- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients (β_1, β_2) for computing running averages of gradient and its square. β_1 controls momentum, β_2 controls variance. Default: (0.9, 0.9999)
- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: step**0.25 (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: step**0.25
- **weight_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0

- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

Raises

- **ValueError** – If learning rate, epsilon, betas, or weight_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↪decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↪step**0.5))
```

Note

- For most use cases, the default hyperparameters work well
- Consider using decouple=True for better generalization (similar to AdamW)
- The clip_lambda function helps stabilize training in early steps

clip_lambda

__setstate__(*state*)

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

Parameters
`state` (*dict*) – Optimizer state dictionary (typically from torch.load()).

> **Note**
>
> - This enables loading checkpoints saved with older ADOPT versions
> - Step counters are converted to appropriate device/dtype for compatibility
> - Capturable and fused modes require step tensors on parameter devices

`step`(*closure=None*)

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)
2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional adopt() API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (adopt()) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access grad_scale and found_inf attributes for gradient unscaling and inf/nan detection when used with GradScaler.

Parameters
`closure` (*Callable, optional*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (torch.enable_grad()) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: lambda: (loss := model(x), loss.backward(), loss)[-1] Default: None

Returns

The loss value returned by the closure, or None if no
closure was provided.

Return type
Optional[Tensor]

Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

Note

- Call zero_grad() before computing gradients for the next step
- CUDA graph capture is checked for safety when capturable=True
- The method is thread-safe for different parameter groups

`dicee.models.adopt.`**`adopt`**(*params: List[torch.Tensor], grads: List[torch.Tensor], exp_avgs: List[torch.Tensor], exp_avg_sqs: List[torch.Tensor], state_steps: List[torch.Tensor], foreach: bool | None = None, capturable: bool = False, differentiable: bool = False, fused: bool | None = None, grad_scale: torch.Tensor | None = None, found_inf: torch.Tensor | None = None, has_complex: bool = False, *, beta1: float, beta2: float, lr: float | torch.Tensor, clip_lambda: Callable[[int], float] | None, weight_decay: float, decouple: bool, eps: float, maximize: bool*)

Functional API that performs ADOPT algorithm computation.

This is the main functional interface for the ADOPT optimization algorithm. It dispatches to one of three implementations based on the execution mode:

1. **Single-tensor mode** (default): Updates parameters one at a time - Compatible with torch.jit.script - More flexible but slower - Used when foreach=False or automatically for small models
2. **Multi-tensor (foreach) mode**: Batches operations across tensors - 2-3x faster on GPU through vectorization - Groups tensors by device/dtype automatically - Used when foreach=True
3. **Fused mode**: Uses specialized fused kernels (not yet implemented) - Would provide maximum performance - Currently raises RuntimeError if enabled

Algorithm overview (ADOPT):

ADOPT adapts learning rates per-parameter while using momentum on normalized gradients. The key innovation is normalizing gradients before momentum, which provides more stable training than standard Adam.

Mathematical formulation:

Normalize gradient by its historical variance normed_g_t = g_t / √(v_t + ε)

Optional gradient clipping for stability normed_g_t = clip(normed_g_t, threshold(t))

Momentum on normalized gradients (key difference from Adam) m_t = β_1 * m_{t-1} + (1 - β_1) * normed_g_t

Parameter update θ_t = θ_{t-1} - α * m_t

Update variance estimate v_t = β_2 * v_{t-1} + (1 - β_2) * g_t^2

where:

- θ: parameters

- g: gradients
- m: first moment (momentum of normalized gradients)
- v: second moment (variance of raw gradients)
- α: learning rate
- β_1, β_2: exponential decay rates
- ε: numerical stability constant
- clip(): gradient clipping function based on step

Automatic mode selection:

When foreach and fused are both None (default), the function automatically selects the best implementation based on: - Parameter types and devices - Whether differentiable mode is enabled - Learning rate type (float vs Tensor) - Capturable mode requirements

param params
Parameters to optimize.

type params
List[Tensor]

param grads
Gradients for each parameter.

type grads
List[Tensor]

param exp_avgs
First moment estimates (momentum).

type exp_avgs
List[Tensor]

param exp_avg_sqs
Second moment estimates (variance).

type exp_avg_sqs
List[Tensor]

param state_steps
Step counters (must be singleton tensors).

type state_steps
List[Tensor]

param foreach
Whether to use multi-tensor implementation. None: auto-select based on configuration (default).

type foreach
Optional[bool]

param capturable
If True, ensure CUDA graph capture safety.

type capturable
bool

param differentiable
If True, allow gradients through optimization step.

type differentiable
bool

param fused
If True, use fused kernels (not implemented).

type fused
Optional[bool]

param grad_scale
Gradient scaler for AMP training.

type grad_scale
Optional[Tensor]

param found_inf
Flag for inf/nan detection in AMP.

type found_inf
Optional[Tensor]

param has_complex
Whether any parameters are complex-valued.

type has_complex
bool

param beta1
Exponential decay rate for first moment (momentum). Typical range: 0.9-0.95.

type beta1
float

param beta2
Exponential decay rate for second moment (variance). Typical range: 0.999-0.9999 (higher than Adam).

type beta2
float

param lr
Learning rate. Can be a scalar Tensor for dynamic learning rate with capturable=True.

type lr
Union[float, Tensor]

param clip_lambda
Function that maps step number to gradient clipping threshold. None disables clipping.

type clip_lambda
Optional[Callable[[int], float]]

param weight_decay
Weight decay coefficient (L2 penalty).

type weight_decay
float

param decouple
If True, use decoupled weight decay (AdamW-style). Recommended for better generalization.

type decouple
bool

param eps
Small constant for numerical stability in normalization.

type eps
float

param maximize
If True, maximize objective instead of minimize.

type maximize
bool

raises RuntimeError
If torch.jit.script is used with foreach or fused.

raises RuntimeError
If state_steps contains non-tensor elements.

raises RuntimeError
If fused=True (not yet implemented).

raises RuntimeError
If lr is Tensor with foreach=True and capturable=False.

Example

```
>>> # Typically called by ADOPT optimizer, not directly
>>> adopt(
...     params=[p1, p2],
...     grads=[g1, g2],
...     exp_avgs=[m1, m2],
...     exp_avg_sqs=[v1, v2],
...     state_steps=[step1, step2],
...     beta1=0.9,
...     beta2=0.9999,
...     lr=0.001,
...     clip_lambda=lambda s: s**0.25,
...     weight_decay=0.01,
...     decouple=True,
...     eps=1e-6,
...     maximize=False,
... )
```

Note

- For distributed training, this API is compatible with torch/distributed/optim
- The foreach mode is generally preferred for GPU training
- Complex parameters are handled transparently by viewing as real
- First optimization step only initializes variance, doesn't update parameters

> **See also**
>
> - ADOPT class: High-level optimizer interface
> - _single_tensor_adopt: Single-tensor implementation details
> - _multi_tensor_adopt: Multi-tensor implementation details

dicee.models.base_model

Attributes

logger

Classes

BaseKGELightning	Thin PyTorch Lightning wrapper shared by all KGE models.
BaseKGE	Base class for all Knowledge Graph Embedding models.
IdentityClass	No-op normalisation / dropout placeholder.

Module Contents

`dicee.models.base_model.`**`logger`**

`class` `dicee.models.base_model.`**`BaseKGELightning`**(**args*, ***kwargs*)

Bases: `lightning.LightningModule`

Thin PyTorch Lightning wrapper shared by all KGE models.

Provides the standard Lightning training loop hooks (`training_step`, `on_train_epoch_end`, `configure_optimizers`) as well as a helper for reporting model size. All concrete KGE models should extend *BaseKGE* rather than this class directly.

`training_step_outputs = []`

`mem_of_model`() → Dict

Size of model in MB and number of params

`training_step`(*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **`batch`** (*tuple*) – `(x, y)` for standard scoring, or `(x, y_select, y)` for sample-based labelling.
- **`batch_idx`** (*int, optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns

Scalar loss value for this batch.

Return type
torch.FloatTensor

loss_function(*yhat_batch: torch.FloatTensor*, *y_batch: torch.FloatTensor*) → torch.FloatTensor

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification).

Parameters

- **yhat_batch** (*torch.FloatTensor*) – Model output scores, shape `(batch_size, *)`.
- **y_batch** (*torch.FloatTensor*) – Ground-truth labels of the same shape as *yhat_batch*.

Returns
Scalar loss value.

Return type
torch.FloatTensor

on_train_epoch_end(**args*, ***kwargs*)

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

test_epoch_end(*outputs: List[Any]*)

test_dataloader() → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

> **Warning**
>
> do not assign state in prepare_data

- `test()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

> **Note**
>
> If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

`val_dataloader``()` → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:paramref:`~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

> **Note**
>
> If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

`predict_dataloader``()` → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Returns
A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

`train_dataloader``()` → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:paramref:`~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`** to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

> **Warning**
>
> do not assign state in prepare_data

- `fit()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

`configure_optimizers``(`*parameters=None*`)`

Instantiate and return the optimiser for training.

The optimiser type is taken from `self.optimizer_name` which is set in *`BaseKGE.init_params_with_sanity_checking()`* from the `--optim` argument. Supported values: `'SGD'`, `'Adam'`, `'Adopt'`, `'AdamW'`, `'NAdam'`, `'Adagrad'`, `'ASGD'`, `'Muon'`.

Parameters
parameters (*iterable, optional*) – Model parameters to optimise. Defaults to `self.parameters()` when `None`.

Returns
The configured optimiser instance.

Return type
torch.optim.Optimizer

class `dicee.models.base_model.`**`BaseKGE`**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where T is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type
torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]
Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

class dicee.models.base_model.**IdentityClass**(*args=None*)
Bases: `torch.nn.Module`

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(*x*)

static forward(*x*)

dicee.models.clifford

Attributes

logger

Classes

Keci	Keci: Knowledge Graph Embedding via Clifford Algebra.
KeciTransformer	Keci with Transformer architecture.
TransformerBlock	A single transformer block with self-attention and MLP.
TransformerSelfAttention	Multi-head self-attention for the Keci Transformer.
TransformerMLP	MLP for the Keci Transformer.
CKeci	Without learning dimension scaling
DeCaL	Base class for all Knowledge Graph Embedding models.

Module Contents

dicee.models.clifford.**logger**

class dicee.models.clifford.**Keci**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Keci: Knowledge Graph Embedding via Clifford Algebra.

Embeds entities and relations as multi-vectors in the Clifford algebra Cl_{p,q}(R^d) and scores triples via the Clifford product. The algebra is parameterised by two non-negative integers *p* and *q*:

- `p = 0, q = 0` — reduces to a standard bilinear (DistMult-like) model.
- `p = 0, q = 1` — equivalent to ComplEx.
- Larger `p` and `q` capture higher-order geometric interactions.

The embedding dimension must satisfy `embedding_dim % (p + q + 1) == 0`; the resulting quotient is stored as `self.r`.

Parameters

args (*dict*) – Configuration dictionary. Recognised keys (beyond those in `BaseKGE`): `p` (int, default 0) and `q` (int, default 0).

References

Demir et al., *Clifford Embeddings — A Generalized Approach for Embedding in Normed Algebras*, ECML 2023.

name = 'Keci'

p

q

r

requires_grad_for_interactions = True

compute_sigma_pp(*hp*, *rp*)

Compute sigma_{pp} = sum_{i=1}^{p-1} sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k

sigma_{pp} captures the interactions between along p bases For instance, let p e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):
results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_qq(*hq*, *rq*)

Compute sigma_{qq} = sum_{j=1}^{p+q-1} sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k sigma_{q} captures the interactions between along q bases For instance, let q e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops results = [] for j in range(q - 1):

for k in range(j + 1, q):
results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

`compute_sigma_pq`(*, *hp*, *hq*, *rp*, *rq*)

sum_{i=1}^{p} sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

`apply_coefficients`(*hp*, *hq*, *rp*, *rq*)

Multiplying a base vector with its scalar coefficient

`clifford_multiplication`(*h0*, *hp*, *hq*, *r0*, *rp*, *rq*)

Compute our CL multiplication

h = h_0 + sum_{i=1}^p h_i e_i + sum_{j=p+1}^{p+q} h_j e_j r = r_0 + sum_{i=1}^p r_i e_i + sum_{j=p+1}^{p+q} r_j e_j

ei ^2 = +1 for i =< i =< p ej ^2 = -1 for p < j =< p+q ei ej = -eje1 for i

eq j

h r = sigma_0 + sigma_p + sigma_q + sigma_{pp} + sigma_{q}+ sigma_{pq} where

(1) sigma_0 = h_0 r_0 + sum_{i=1}^p (h_0 r_i) e_i - sum_{j=p+1}^{p+q} (h_j r_j) e_j

(2) sigma_p = sum_{i=1}^p (h_0 r_i + h_i r_0) e_i

(3) sigma_q = sum_{j=p+1}^{p+q} (h_0 r_j + h_j r_0) e_j

(4) sigma_{pp} = sum_{i=1}^{p-1} sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k

(5) sigma_{qq} = sum_{j=1}^{p+q-1} sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k

(6) sigma_{pq} = sum_{i=1}^{p} sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j

`construct_cl_multivector`(*x: torch.FloatTensor*, *r: int*, *p: int*, *q: int*)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a flat embedding vector into the three Clifford components.

Given an embedding `x` of dimension `d = r + r*p + r*q`, returns the scalar part `a0`, the *p*-blade part `ap`, and the *q*-blade part `aq`.

Parameters

- **x** (*torch.FloatTensor*) – Shape `(batch_size, d)`.
- **r** (*int*) – Scalar block size (`embedding_dim // (p + q + 1)`).
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape `(batch_size, r)` — scalar (grade-0) part.
- **ap** (*torch.FloatTensor*) – Shape `(batch_size, r, p)` — positive-blade part.
- **aq** (*torch.FloatTensor*) – Shape `(batch_size, r, q)` — negative-blade part.

forward_k_vs_with_explicit(*x: torch.Tensor*) → torch.FloatTensor

KvsAll scoring using an explicit loop over sigma_pp/qq/pq terms.

Functionally equivalent to *forward_k_vs_all()* but computes the higher-order interaction terms (sigma_pp, sigma_qq, sigma_pq) with explicit nested loops rather than einsum contractions. Kept for reference and correctness verification.

Parameters

x (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

k_vs_all_score(*bpe_head_ent_emb: torch.FloatTensor*, *bpe_rel_ent_emb: torch.FloatTensor*, *E: torch.FloatTensor*) → torch.FloatTensor

Compute Clifford-product scores for a head/relation batch vs. all entities.

Decomposes the head-entity and relation embeddings into Clifford multi-vectors, performs the Cl_{p,q} product, and inner-products the result against the entity embedding matrix *E*.

Parameters

- **bpe_head_ent_emb** (*torch.FloatTensor*) – Head-entity embeddings, shape `(batch_size, embedding_dim)`.
- **bpe_rel_ent_emb** (*torch.FloatTensor*) – Relation embeddings, shape `(batch_size, embedding_dim)`.
- **E** (*torch.FloatTensor*) – All entity embeddings, shape `(num_entities, embedding_dim)`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

(1) Retrieve real-valued embedding vectors for heads and relations mathbb{R}^d .

(2) Construct head entity and relation embeddings according to Cl_{p,q}(mathbb{R}^d) .

(3) Perform Cl multiplication

(4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this funcitons are identical Parameter ——— x: torch.LongTensor with (n,2) shape :rtype: torch.FloatTensor with (n, |E|) shape

construct_batch_selected_cl_multivector(*x: torch.FloatTensor*, *r: int*, *p: int*, *q: int*)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a batched, *k*-selected embedding tensor into Clifford components.

A variant of `construct_cl_multivector()` for tensors that have an extra *k* dimension (e.g. when scoring against *k* sampled targets).

Parameters

- **x** (*torch.FloatTensor*) – Shape `(batch_size, k, d)`.
- **r** (*int*) – Scalar block size.
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape `(batch_size, k, r)`.
- **ap** (*torch.FloatTensor*) – Shape `(batch_size, k, r, p)`.
- **aq** (*torch.FloatTensor*) – Shape `(batch_size, k, r, q)`.

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,2) shape

target_entity_idx: torch.LongTensor with (n, k) shape k denotes the selected number of examples.

rtype
torch.FloatTensor with (n, k) shape

score(*h*, *r*, *t*)

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,3) shape

rtype
torch.FloatTensor with (n) shape

class dicee.models.clifford.**KeciTransformer**(*args*)

Bases: `Keci`

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

name = 'KeciTransformer'

use_clifford_mul

seq_len = 1

transformer

lm_head

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

Parameter

x: torch.LongTensor with (n,2) shape

rtype

torch.FloatTensor with (n, |E|) shape

class dicee.models.clifford.**TransformerBlock**(*n_embd*, *n_head*, *dropout=0.0*, *bias=False*)

Bases: torch.nn.Module

A single transformer block with self-attention and MLP.

ln_1

attn

ln_2

mlp

forward(*x*)

class dicee.models.clifford.**TransformerSelfAttention**(*n_embd*, *n_head*, *dropout=0.0*, *bias=False*)

Bases: torch.nn.Module

Multi-head self-attention for the Keci Transformer.

c_attn

c_proj

attn_dropout

resid_dropout

n_head

n_embd

dropout = 0.0

forward(*x*)

class dicee.models.clifford.**TransformerMLP**(*n_embd*, *dropout=0.0*, *bias=False*)

Bases: torch.nn.Module

MLP for the Keci Transformer.

c_fc

gelu

c_proj

dropout

forward(*x*)

class dicee.models.clifford.**CKeci**(*args*)

Bases: *Keci*

Without learning dimension scaling

name = 'CKeci'

requires_grad_for_interactions = False

class dicee.models.clifford.**DeCaL**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'DeCaL'

entity_embeddings

relation_embeddings

p

q

r

re

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,) shape

rtype

torch.FloatTensor with (n) shape

cl_pqr(*a: torch.tensor*) → torch.tensor

Input: tensor(batch_size, emb_dim) —> output: tensor with 1+p+q+r components with size (batch_size, emb_dim/(1+p+q+r)) each.

1) takes a tensor of size (batch_size, emb_dim), split it into 1 + p + q +r components, hence 1+p+q+r must be a divisor of the emb_dim. 2) Return a list of the 1+p+q+r components vectors, each are tensors of size (batch_size, emb_dim/(1+p+q+r))

compute_sigmas_single (*list_h_emb*, *list_r_emb*, *list_t_emb*)

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s0 = h_0 r_0 t_0 s1 = \sum_{i=1}^{p} h_i r_i t_0 s2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 s3 = \sum_{i=1}^{q} (h_0 r_i t_i + h_i r_0 t_i) s4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) s5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t$$

and return:

$$sigma_0 t = \sigma_0 \cdot t_0 = s0 + s1 - s2 s3, s4 and s5$$

compute_sigmas_multivect (*list_h_emb*, *list_r_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_p p = \sum_{i=1}^{p-1} \sum_{i'=i+1}^{p} (h_i r_{i'} - h_{i'} r_i)(models the interactions between e_i and e'_i for 1 <= i, i' <= p) \sigma_q q = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'}$$

For different base vector interactions, we have

$$\sigma_p q = \sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i)(interactionsn between e_i and e_j for 1 <= i <= p and p+1 <= j <= p+q) \sigma_p r = \sum_{i=}$$

forward_k_vs_all (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

(1) Retrieve real-valued embedding vectors for heads and relations

(2) Construct head entity and relation embeddings according to Cl_{p,q, r}(mathbb{R}^d) .

(3) Perform Cl multiplication

(4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this funcitons are identical Parameter ——— x: torch.LongTensor with (n,) shape :rtype: torch.FloatTensor with (n, |E|) shape

apply_coefficients (*h0*, *hp*, *hq*, *hk*, *r0*, *rp*, *rq*, *rk*)

Multiplying a base vector with its scalar coefficient

construct_cl_multivector (*x: torch.FloatTensor*, *re: int*, *p: int*, *q: int*, *r: int*) → tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors Cl_{p,q,r}(mathbb{R}^d)

Parameter

x: torch.FloatTensor with (n,d) shape

returns

- **a0** (*torch.FloatTensor*)
- **ap** (*torch.FloatTensor*)
- **aq** (*torch.FloatTensor*)
- **ar** (*torch.FloatTensor*)

compute_sigma_pp(*hp*, *rp*)

Compute .. math:

```
\sigma_{p,p}^* = \sum_{i=1}^{p-1}\sum_{i'=i+1}^{p}(x_iy_{i'}-x_{i'}y_i)
```

sigma_{pp} captures the interactions between along p bases For instance, let p e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):
results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_qq(*hq*, *rq*)

Compute

$$\sigma_{q,q}^* = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) Eq.16$$

sigma_{q} captures the interactions between along q bases For instance, let q e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):
results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_rr(*hk*, *rk*)

$$\sigma_{r,r}^* = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^{p} (x_k y_{k'} - x_{k'} y_k)$$

compute_sigma_pq(*, *hp*, *hq*, *rp*, *rq*)

Compute

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

compute_sigma_pr(*, *hp*, *hk*, *rp*, *rk*)

Compute

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

compute_sigma_qr(*, *hq*, *hk*, *rq*, *rk*)

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

dicee.models.complex

Classes

`ConEx`	Convolutional ComplEx Knowledge Graph Embeddings
`AConEx`	Additive Convolutional ComplEx Knowledge Graph Embeddings
`ComplEx`	Base class for all Knowledge Graph Embedding models.
`RotatE`	RotatE: knowledge graph embedding by relational rotation in complex space.

Module Contents

class dicee.models.complex.**ConEx**(*args*)

Bases: `dicee.models.base_model.BaseKGE`

Convolutional ComplEx Knowledge Graph Embeddings

name = 'ConEx'

conv2d

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution(*C_1: Tuple[torch.Tensor, torch.Tensor]*, *C_2: Tuple[torch.Tensor, torch.Tensor]*) → torch.FloatTensor

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

class dicee.models.complex.**AConEx**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional ComplEx Knowledge Graph Embeddings

name = 'AConEx'

conv2d

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution(*C_1: Tuple[torch.Tensor, torch.Tensor]*, *C_2: Tuple[torch.Tensor, torch.Tensor]*) → torch.FloatTensor

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

class dicee.models.complex.**ComplEx**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'ComplEx'

static score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*)

static k_vs_all_score(*emb_h: torch.FloatTensor*, *emb_r: torch.FloatTensor*, *emb_E: torch.FloatTensor*)

Parameters

- **emb_h**
- **emb_r**
- **emb_E**

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

class dicee.models.complex.**RotatE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

RotatE: knowledge graph embedding by relational rotation in complex space.

Represents each entity as a complex vector in ℂ^(d/2), obtained by splitting its `d`-dimensional embedding into a real half and an imaginary half. Each relation is represented by `d/2` phase angles θ that define a unit-modulus rotation `r_i = e^{iθ_i}`; since a phase angle needs only one real number, the relation embedding table is reinitialised at half of the entity embedding size. A true triple `(h, r, t)` should satisfy `h ∘ r ≈ t` under element-wise complex multiplication, giving the score:

```
f(h, r, t) = margin - ||h ∘ r - t||_2
```

Unlike TransE, RotatE can model symmetric, antisymmetric, inverse, and composition relation patterns.

References

Sun et al., *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*, ICLR 2019. https://arxiv.org/abs/1902.10197

name = 'RotatE'

margin = 6.0

half_dim

relation_embeddings

score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*) → torch.FloatTensor

Score a batch of triples using the RotatE margin-distance formula.

Parameters

- **head_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **tail_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **rel_ent_emb** (*torch.FloatTensor*) – Shape `(batch_size, d/2)` relation phase angles θ.

Returns

Shape `(batch_size,)` scores equal to `margin - ||h ∘ r - t||_2`.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes `margin - ||h ∘ r - e||_2` for every entity embedding *e*.

Parameters

x (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Computes `margin - ||h ∘ r - e||_2` for each of the *k* sampled entities *e*.

Parameters

- **x** (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.Tensor*) – Shape `(batch_size, k)` indices of the *k* target entities per sample.

Returns

Shape `(batch_size, k)` score matrix.

Return type
torch.FloatTensor

dicee.models.dualE

Classes

`DualE`	Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)

Module Contents

class dicee.models.dualE.**DualE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)

name = 'DualE'

entity_embeddings

relation_embeddings

num_ent = None

kvsall_score(*e_1_h, e_2_h, e_3_h, e_4_h, e_5_h, e_6_h, e_7_h, e_8_h, e_1_t, e_2_t, e_3_t, e_4_t, e_5_t, e_6_t, e_7_t, e_8_t, r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8*) → torch.tensor

KvsAll scoring function

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_triples(*idx_triple: torch.tensor*) → torch.tensor

Negative Sampling forward pass:

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_k_vs_all(*x*)

KvsAll forward pass

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

T(*x: torch.tensor*) → torch.tensor

Transpose function

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)

dicee.models.ensemble

Classes

EnsembleKGE

Module Contents

class dicee.models.ensemble.**EnsembleKGE**(*models: list = None*, *seed_model=None*, *pretrained_models: List = None*)

name

train_mode = True

args

named_children()

property example_input_array

parameters()

modules()

__iter__()

__len__()

eval()

to(*device*)

state_dict()

Return the state dict of the ensemble.

load_state_dict(*state_dict*, *strict=True*)

Load the state dict into the ensemble.

mem_of_model()

__call__(*x_batch*)

step()

get_embeddings()

__str__()

dicee.models.fsdp_models

Row-wise sharded entity embeddings for multi-GPU training.

Entity embeddings are partitioned row-wise across ranks:

- dist.all_to_all_single routes index requests to the owning rank
- A custom autograd Function propagates gradients back through the all_to_all
- _LocalSparseAdam updates only the rows that received a non-zero gradient

Public API:

model.setup_fsdp_training(device, lr) — called by trainer before loop model.gather_entity_embeddings_on_rank_zero() — called by trainer after loop model.save_local_shard_checkpoint(path) — per-rank, called periodically during loop model.load_local_shard_checkpoint(path) — per-rank, called by trainer on resume create_fsdp_sharded_model_class(ModelClass) — factory used by static_funcs.py

Classes

RowWiseShardedEmbedding	Entity embedding table sharded row-wise across ranks.
FSDPShardedEntityModel	Mixin that defers entity embedding allocation and wires up row-wise sharding.

Functions

create_fsdp_sharded_model_class(...)	Return a row-wise sharded variant of *model_class*.

Module Contents

class dicee.models.fsdp_models.**RowWiseShardedEmbedding**(*num_embeddings: int*, *embedding_dim: int*, *rank: int*, *world_size: int*, *device: torch.device*, *weight_dtype: torch.dtype = torch.bfloat16*)

Bases: `torch.nn.Module`

Entity embedding table sharded row-wise across ranks.

Each rank owns rows [start_row, end_row). forward() looks like nn.Embedding so every existing scoring function works unchanged.

num_embeddings

embedding_dim

rank

world_size

device

shard_size

start_row

end_row

local_rows

weight

forward(*entity_ids: torch.LongTensor*) → torch.FloatTensor

class dicee.models.fsdp_models.**FSDPShardedEntityModel**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Mixin that defers entity embedding allocation and wires up row-wise sharding.

Lifecycle

1. __init__() — entity_embeddings is None
2. setup_fsdp_training(dev, lr) — creates RowWiseShardedEmbedding
3. gather_entity_embeddings_on_rank_zero() — called by trainer after last epoch

setup_fsdp_training(*device: torch.device*, *lr: float*, *optimizer_cls=None*, *optimizer_kwargs: dict = None*, *adam_device: torch.device = None*) → None

Create the per-rank embedding shard and its local sparse Adam.

adam_device — where exp_avg / exp_avg_sq are stored:
GPU (default) : pure GPU kernels, no PCIe, ~42.8 GiB extra GPU RAM per rank. CPU : lower GPU footprint, PCIe transfer each step.

Falls back to the embedding device (GPU) when not specified.

gather_entity_embeddings_on_rank_zero() → torch.nn.Embedding | None

Gather the full entity table on rank 0; return None on other ranks.

save_local_shard_checkpoint(*path: str*) → None

Save this rank's entity embedding shard + its local Adam state to *path*.

Cheap and O(local_rows) — unlike gather_entity_embeddings_on_rank_zero(), no collective communication is involved, so this is safe to call periodically even when the full table is too large to gather.

load_local_shard_checkpoint(*path: str*) → None

Load this rank's entity embedding shard + local Adam state from *path*.

The checkpoint must have been saved by a run with the same world_size: shard boundaries (start_row/local_rows) are derived from world_size and num_entities, so a mismatch means this rank does not own the same rows as when the checkpoint was written.

get_embeddings()

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

forward_k_vs_all(**args*, ***kwargs*)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

dicee.models.fsdp_models.**create_fsdp_sharded_model_class**(*model_class: Type[dicee.models.base_model.BaseKGE]*) → Type[*dicee.models.base_model.BaseKGE*]

Return a row-wise sharded variant of *model_class*.

The returned class inherits all scoring functions from *model_class* unchanged. Only entity_embeddings is replaced at training time.

dicee.models.function_space

Attributes

logger

Classes

FMult	Learning Knowledge Neural Graphs
GFMult	Learning Knowledge Neural Graphs
FMult2	Learning Knowledge Neural Graphs
LFMult1	Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:
LFMult	Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:

Module Contents

dicee.models.function_space.**logger**

class dicee.models.function_space.**FMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'FMult'

entity_embeddings

relation_embeddings

k

num_sample = 50

gamma

roots

weights

compute_func(*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func(*weights*, *x: torch.FloatTensor*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.function_space.**GFMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'GFMult'

entity_embeddings

relation_embeddings

k

num_sample = 250

roots

weights

compute_func(*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func(*weights*, *x: torch.FloatTensor*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.function_space.**FMult2**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'FMult2'

n_layers = 3

k

n = 50

score_func = 'compositional'

discrete_points

entity_embeddings

relation_embeddings

build_func(*Vec*)

build_chain_funcs(*list_Vec*)

compute_func(W, b, x) → torch.FloatTensor

function(*list_W*, *list_b*)

trapezoid(*list_W*, *list_b*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.function_space.**LFMult1**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Embedding with trigonometric functions. We represent all entities and relations in the complex number space as: f(x) = sum_{k=0}^{k=d-1}wk e^{kix}. and use the three differents scoring function as in the paper to evaluate the score

name = 'LFMult1'

entity_embeddings

relation_embeddings

forward_triples(*idx_triple*)

Score a batch of (head, relation, tail) index triples.

Parameters
x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns
Shape (batch_size,) triple scores.

Return type
torch.FloatTensor

tri_score(*h*, *r*, *t*)

vtp_score(*h*, *r*, *t*)

class dicee.models.function_space.**LFMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Embedding with polynomial functions. We represent all entities and relations in the polynomial space as: f(x) = sum_{i=0}^{d-1} a_k x^{i%d} and use the three differents scoring function as in the paper to evaluate the score. We also consider combining with Neural Networks.

name = 'LFMult'

entity_embeddings

relation_embeddings

degree

m

x_values

forward_triples(*idx_triple*)

Score a batch of (head, relation, tail) index triples.

Parameters
x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns
Shape (batch_size,) triple scores.

Return type
torch.FloatTensor

construct_multi_coeff(*x*)

poly_NN(*x*, *coefh*, *coefr*, *coeft*)

Constructing a 2 layers NN to represent the embeddings. h = sigma(wh^T x + bh), r = sigma(wr^T x + br), t = sigma(wt^T x + bt)

linear(*x*, *w*, *b*)

scalar_batch_NN(*a*, *b*, *c*)

element wise multiplication between a,b and c: Inputs : a, b, c ====> torch.tensor of size batch_size x m x d Output : a tensor of size batch_size x d

tri_score(*coeff_h*, *coeff_r*, *coeff_t*)

this part implement the trilinear scoring techniques:

score(h,r,t) = int_{0}{1} h(x)r(x)t(x) dx = sum_{i,j,k = 0}^{d-1} dfrac{a_i*b_j*c_k}{1+(i+j+k)%d}

1. generate the range for i,j and k from [0 d-1]
2. perform dfrac{a_i*b_j*c_k}{1+(i+j+k)%d} in parallel for every batch
3. take the sum over each batch

vtp_score(*h*, *r*, *t*)

this part implement the vector triple product scoring techniques:

score(h,r,t) = int_{0}{1} h(x)r(x)t(x) dx = sum_{i,j,k = 0}^{d-1} dfrac{a_i*c_j*b_k - b_i*c_j*a_k}{(1+(i+j)%d)(1+k)}

1. generate the range for i,j and k from [0 d-1]
2. Compute the first and second terms of the sum
3. Multiply with then denominator and take the sum
4. take the sum over each batch

comp_func(*h*, *r*, *t*)

this part implement the function composition scoring techniques: i.e. score = <hor, t>

polynomial(*coeff*, *x*, *degree*)

This function takes a matrix tensor of coefficients (coeff), a tensor vector of points x and range of integer [0,1,…d] and return a vector tensor (coeff[0][0] + coeff[0][1]x +…+ coeff[0][d]x^d,

> **coeff[1][0] + coeff[1][1]x +…+ coeff[1][d]x^d)**

pop(*coeff*, *x*, *degree*)

This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (coeff), a matrix tensor of points x and range of integer [0,1,…d]

> **and return a tensor (coeff[0][0] + coeff[0][1]x +…+ coeff[0][d]x^d,**
>
> > **coeff[1][0] + coeff[1][1]x +…+ coeff[1][d]x^d)**

dicee.models.literal

Classes

LiteralEmbeddings	A model for learning and predicting numerical literals using pre-trained KGE.

Module Contents

class dicee.models.literal.**LiteralEmbeddings**(*num_of_data_properties: int*, *embedding_dims: int*, *entity_embeddings: torch.tensor*, *dropout: float = 0.3*, *gate_residual=True*, *freeze_entity_embeddings=True*)

Bases: `torch.nn.Module`

A model for learning and predicting numerical literals using pre-trained KGE.

`num_of_data_properties`
: Number of data properties (attributes).

  Type
  : int

`embedding_dims`
: Dimension of the embeddings.

  Type
  : int

`entity_embeddings`
: Pre-trained entity embeddings.

  Type
  : torch.tensor

`dropout`
: Dropout rate for regularization.

  Type
  : float

`gate_residual`
: Whether to use gated residual connections.

  Type
  : bool

`freeze_entity_embeddings`
: Whether to freeze the entity embeddings during training.

  Type
  : bool

`embedding_dim`

`num_of_data_properties`

`hidden_dim`

`gate_residual = True`

`freeze_entity_embeddings = True`

`entity_embeddings`

`data_property_embeddings`

`fc`

`fc_out`

`dropout`

`gated_residual_proj`

`layer_norm`

forward(*entity_idx*, *attr_idx*)

Parameters

- **entity_idx** (*Tensor*) – Entity indices (batch).
- **attr_idx** (*Tensor*) – Attribute (Data property) indices (batch).

Returns
scalar predictions.

Return type
Tensor

property device

dicee.models.octonion

Classes

OMult	Base class for all Knowledge Graph Embedding models.
ConvO	Base class for all Knowledge Graph Embedding models.
AConvO	Additive Convolutional Octonion Knowledge Graph Embeddings

Functions

octonion_mul(*, O_1, O_2)	
octonion_mul_norm(*, O_1, O_2)	

Module Contents

dicee.models.octonion.**octonion_mul**(*, *O_1*, *O_2*)

dicee.models.octonion.**octonion_mul_norm**(*, *O_1*, *O_2*)

class dicee.models.octonion.**OMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'OMult'

static octonion_normalizer(*emb_rel_e0*, *emb_rel_e1*, *emb_rel_e2*, *emb_rel_e3*, *emb_rel_e4*, *emb_rel_e5*, *emb_rel_e6*, *emb_rel_e7*)

score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*)

k_vs_all_score(*bpe_head_ent_emb*, *bpe_rel_ent_emb*, *E*)

forward_k_vs_all(*x*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.octonion.**ConvO**(*args: dict*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'ConvO'

conv2d

fc_num_input

fc1

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(*emb_rel_e0*, *emb_rel_e1*, *emb_rel_e2*, *emb_rel_e3*, *emb_rel_e4*, *emb_rel_e5*, *emb_rel_e6*, *emb_rel_e7*)

residual_convolution(*O_1*, *O_2*)

forward_triples(*x: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.octonion.**AConvO**(*args: dict*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional Octonion Knowledge Graph Embeddings

name = 'AConvO'

conv2d

fc_num_input

fc1

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

residual_convolution(*O_1, O_2*)

forward_triples(*x: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

dicee.models.pykeen_models

Attributes

logger	

Classes

PykeenKGE	A class for using knowledge graph embedding models implemented in Pykeen

Module Contents

`dicee.models.pykeen_models.`**`logger`**

`class` `dicee.models.pykeen_models.`**`PykeenKGE`**`(`*args: dict*`)`

Bases: *dicee.models.base_model.BaseKGE*

A class for using knowledge graph embedding models implemented in Pykeen

Notes: Pykeen_DistMult: C Pykeen_ComplEx: Pykeen_QuatE: Pykeen_MuRE: Pykeen_CP: Pykeen_HolE: Pykeen_HolE: Pykeen_HolE: Pykeen_TransD: Pykeen_TransE: Pykeen_TransF: Pykeen_TransH: Pykeen_TransR:

`model_kwargs`

`name`

`model`

`loss_history = []`

`args`

`entity_embeddings = None`

`relation_embeddings = None`

`forward_k_vs_all``(`*x: torch.LongTensor*`)`

=> Explicit version by this we can apply bn and dropout

(1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r = self.get_head_relation_representation(x) # (2) Reshape (1). if self.last_dim > 0:

h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim, self.last_dim)

(3) Reshape all entities. if self.last_dim > 0:

t = self.entity_embeddings.weight.reshape(self.num_entities, self.embedding_dim, self.last_dim)

else:

t = self.entity_embeddings.weight

(4) Call the score_t from interactions to generate triple scores. return self.interaction.score_t(h=h, r=r, all_entities=t, slice_size=1)

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

=> Explicit version by this we can apply bn and dropout

(1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t = self.get_triple_representation(x) # (2) Reshape (1). if self.last_dim > 0:

h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim, self.last_dim) t = t.reshape(len(x), self.embedding_dim, self.last_dim)

(3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice_size=None, slice_dim=0)

abstractmethod forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

dicee.models.quaternion

Classes

QMult	Base class for all Knowledge Graph Embedding models.
ConvQ	Convolutional Quaternion Knowledge Graph Embeddings
AConvQ	Additive Convolutional Quaternion Knowledge Graph Embeddings

Functions

quaternion_mul_with_unit_norm(*, Q_1, Q_2)	

Module Contents

dicee.models.quaternion.**quaternion_mul_with_unit_norm**(*, *Q_1*, *Q_2*)

class dicee.models.quaternion.**QMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'QMult'

explicit = True

quaternion_multiplication_followed_by_inner_product (*h*, *r*, *t*)

Parameters

- **h** – shape: (**batch_dims*, dim) The head representations.
- **r** – shape: (**batch_dims*, dim) The head representations.
- **t** – shape: (**batch_dims*, dim) The tail representations.

Returns
Triple scores.

static quaternion_normalizer (*x: torch.FloatTensor*) → torch.FloatTensor

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^{d} |x_i|^2 = \sum_{i=1}^{d} (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

Parameters
x – The vector.

Returns
The normalized vector.

score (*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*)

k_vs_all_score (*bpe_head_ent_emb*, *bpe_rel_ent_emb*, *E*)

Parameters

- **bpe_head_ent_emb**
- **bpe_rel_ent_emb**
- **E**

forward_k_vs_all (*x*)

Parameters
x

forward_k_vs_sample (*x*, *target_entity_idx*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.quaternion.**ConvQ**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional Quaternion Knowledge Graph Embeddings

name = 'ConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(*Q_1*, *Q_2*)

forward_triples(*indexed_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.quaternion.**AConvQ**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional Quaternion Knowledge Graph Embeddings

name = 'AConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(*Q_1*, *Q_2*)

forward_triples(*indexed_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

dicee.models.real

Classes

DistMult	DistMult: bilinear diagonal knowledge graph embedding.
TransE	TransE: translation-based knowledge graph embedding.
TransH	TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.
MuRE	MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.
Shallom	Shallom: shallow neural model for relation prediction.
Pyke	Pyke: Physical Embedding Model for Knowledge Graphs.
CoKEConfig	Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.
CoKE	Contextualized Knowledge Graph Embedding (CoKE) model.

Module Contents

class dicee.models.real.**DistMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

DistMult: bilinear diagonal knowledge graph embedding.

Scores a triple `(h, r, t)` as the element-wise product of the head, relation, and tail embeddings summed over the embedding dimension:

```
f(h, r, t) = \sum_i  h_i  \cdot  r_i  \cdot  t_i
```

Simple yet effective baseline; incapable of modelling asymmetric relations.

References

Yang et al., *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*, ICLR 2015. https://arxiv.org/abs/1412.6575

name = 'DistMult'

k_vs_all_score(*emb_h: torch.FloatTensor*, *emb_r: torch.FloatTensor*, *emb_E: torch.FloatTensor*) → torch.FloatTensor

Score a head/relation batch against all entity embeddings.

Computes `(h * r) @ E^T` after applying hidden dropout and normalisation to the element-wise product.

Parameters

- **emb_h** (*torch.FloatTensor*) – Head entity embeddings, shape `(batch_size, embedding_dim)`.
- **emb_r** (*torch.FloatTensor*) – Relation embeddings, shape `(batch_size, embedding_dim)`.
- **emb_E** (*torch.FloatTensor*) – All entity embeddings, shape `(num_entities, embedding_dim)`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the k target entities per sample.

Returns

Shape `(batch_size, k)` score matrix.

Return type

torch.FloatTensor

score(*h: torch.FloatTensor*, *r: torch.FloatTensor*, *t: torch.FloatTensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` embedding triples.

Parameters

- **h** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **r** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **t** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).

Returns
Shape (batch_size,) triple scores.

Return type
torch.FloatTensor

class dicee.models.real.**TransE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

TransE: translation-based knowledge graph embedding.

Models a relation *r* as a translation in embedding space such that h + r ≈ t for a true triple (h, r, t). The score function is defined as:

```
f(h, r, t) = margin - ||h + r - t||_2
```

TransE is effective for 1-to-1 relations but struggles with reflexive, one-to-many, and many-to-one patterns.

References

Bordes et al., *Translating Embeddings for Modeling Multi-relational Data*, NeurIPS 2013. https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf

name = 'TransE'

margin = 4

score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*) → torch.FloatTensor

Score a batch of triples using the TransE margin-distance formula.

Parameters

- **head_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **rel_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **tail_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).

Returns
Shape (batch_size,) scores equal to margin - ||h + r - t||_2.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes margin - ||h + r - e||_2 for every entity embedding *e*.

Parameters
x (*torch.Tensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

class `dicee.models.real.`**`TransH`**`(`*args*`)`

Bases: *dicee.models.base_model.BaseKGE*

TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.

Addresses TransE's inability to model one-to-many, many-to-one, and many-to-many relations by letting each relation *r* define its own hyperplane. An entity is first projected onto that hyperplane before the translation is applied, so the same entity can be positioned differently depending on the relation. Concretely, for a triple `(h, r, t)` with unit-norm hyperplane normal `r_w` and in-hyperplane translation vector `r_d`:

```
h_r = h - (r_w^T h) * r_w
t_r = t - (r_w^T t) * r_w
f(h, r, t) = -||h_r + r_d - t_r||_2^2
```

`r_w` reuses the inherited `relation_embeddings` table and is normalised to unit norm before every use, since only a unit-norm normal yields a true orthogonal projection onto the hyperplane. `r_d` is a second, relation-specific embedding table analogous to TransE's relation vector.

References

Wang et al., *Knowledge Graph Embedding by Translating on Hyperplanes*, AAAI 2014. https://aaai.org/papers/8870-knowledge-graph-embedding-by-translating-on-hyperplanes/

`name = 'TransH'`

`relation_normal_translations`

`forward_triples``(`*x: torch.LongTensor*`)` → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

`forward_k_vs_all``(`*x: torch.LongTensor*`)` → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Every candidate tail entity must be projected onto the batch row's relation-specific hyperplane, and the hyperplane normal differs per batch row, so unlike TransE this materialises a full `(batch_size, num_entities, embedding_dim)` tensor of projected tail candidates – O(B * E * D) memory, versus TransE's O(B * E).

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Same hyperplane-projection logic as *forward_k_vs_all()*, but restricted to the k sampled tail candidates instead of every entity, so it costs `O(B * k * D)` memory instead of `O(B * E * D)`.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the k target entities per sample.

Returns
Shape `(batch_size, k)` score matrix.

Return type
torch.FloatTensor

class dicee.models.real.**MuRE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.

Scores a triple `(h, r, t)` as:

```
f(h, r, t) = -||R_r \odot h + t_r - t||_2 + b_h + b_t
```

`R_r` is a relation-specific diagonal matrix applied to the head via an element-wise (Hadamard) product; it reuses the inherited `relation_embeddings` table directly, shape `(num_relations, embedding_dim)`. `t_r` is a second, relation-specific translation vector (own embedding table, same shape convention: `(num_relations, embedding_dim)`) analogous to TransE's relation vector. `b_h` and `b_t` are learnable scalar biases indexed per entity (head and tail respectively), not per relation.

References

Balažević et al., *Multi-relational Poincaré Graph Embeddings*, NeurIPS 2019. https://arxiv.org/abs/1905.09791

name = 'MuRE'

relation_translations

b_h

b_t

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes `-||R_r \odot h + t_r - e||_2 + b_h + b_e` for every entity embedding *e*.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the *k* target entities per sample.

Returns

Shape `(batch_size, k)` score matrix.

Return type

torch.FloatTensor

class dicee.models.real.**Shallom**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Shallom: shallow neural model for relation prediction.

Represents each triple as the concatenation of head and tail entity embeddings and feeds it through a two-layer MLP to predict the relation. Designed for the `RelationPrediction` labelling form.

References

Demir et al., *A Shallow Neural Model for Relation Prediction*, ISWC 2021. https://arxiv.org/abs/2101.09090

name = 'Shallom'

shallom

get_embeddings() → Tuple[numpy.ndarray, None]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

forward_k_vs_all(*x*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_triples (*x*) → torch.FloatTensor

Score a batch of triples by looking up relation scores from `forward_k_vs_all`.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

class dicee.models.real.**Pyke** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Pyke: Physical Embedding Model for Knowledge Graphs.

Scores a triple `(h, r, t)` based on the average pairwise distance between head-to-relation and relation-to-tail in embedding space:

```
f(h, r, t) = margin - (||h - r||_2 + ||r - t||_2) / 2
```

The model encodes geometric proximity between entities and the relations that connect them.

name = 'Pyke'

dist_func

margin = 1.0

forward_triples (*x: torch.LongTensor*) → torch.FloatTensor

Score a batch of triples using the Pyke distance formula.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

class dicee.models.real.**CoKEConfig**

Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

block_size

Sequence length for transformer (3 for triples: head, relation, tail)

vocab_size

Total vocabulary size (num_entities + num_relations)

n_layer

Number of transformer layers

n_head

Number of attention heads per layer

n_embd

Embedding dimension (set to match model embedding_dim)

dropout

Dropout rate applied throughout the model

bias

Whether to use bias in linear layers

causal

Whether to use causal masking (False for bidirectional attention)

block_size: int = 3

vocab_size: int = None

n_layer: int = 6

n_head: int = 8

n_embd: int = None

dropout: float = 0.3

bias: bool = True

causal: bool = False

class dicee.models.real.**CoKE**(*args*, *config: CoKEConfig = CoKEConfig()*)

Bases: *dicee.models.base_model.BaseKGE*

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: https://arxiv.org/pdf/1911.02168.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head_emb, relation_emb, mask_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

name = 'CoKE'

config

pos_emb

mask_emb

blocks

ln_f

coke_dropout

forward_k_vs_all(*x: torch.Tensor*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

score(*emb_h*, *emb_r*, *emb_t*)

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

dicee.models.static_funcs

Functions

quaternion_mul(→ Tuple[torch.Tensor, torch.Tensor, ...)	Perform quaternion multiplication

Module Contents

`dicee.models.static_funcs.`**quaternion_mul**(*, *Q_1*, *Q_2*)
→ Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

Perform quaternion multiplication :param Q_1: :param Q_2: :return:

dicee.models.transformers

Full definition of a GPT Language Model, all of it in this single file. References: 1) the official GPT-2 TensorFlow implementation released by OpenAI: https://github.com/openai/gpt-2/blob/master/src/model.py 2) huggingface/transformers PyTorch implementation: https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py

Attributes

logger

Classes

ByteE	Base class for all Knowledge Graph Embedding models.
LayerNorm	LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False
SelfAttention	Base class for all neural network modules.
MLP	Base class for all neural network modules.
Block	Base class for all neural network modules.
GPTConfig	
GPT	Base class for all neural network modules.

Module Contents

dicee.models.transformers.**logger**

class dicee.models.transformers.**BytE**(**args*, ***kwargs*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- `forward_k_vs_all()` — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'BytE'

config

temperature = 0.5

topk = 2

transformer

lm_head

loss_function(*yhat_batch*, *y_batch*)

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification).

Parameters

- **yhat_batch** (*torch.FloatTensor*) – Model output scores, shape `(batch_size, *)`.
- **y_batch** (*torch.FloatTensor*) – Ground-truth labels of the same shape as *yhat_batch*.

Returns
Scalar loss value.

Return type
torch.FloatTensor

forward(*x: torch.LongTensor*)

Parameters
x (*B by T tensor*)

generate(*idx*, *max_new_tokens*, *temperature=1.0*, *top_k=None*)

Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete the sequence max_new_tokens times, feeding the predictions back into the model each time. Most likely you'll want to make sure to be in model.eval() mode of operation for this.

training_step(*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **batch** (*tuple*) – `(x, y)` for standard scoring, or `(x, y_select, y)` for sample-based labelling.
- **batch_idx** (*int, optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns
Scalar loss value for this batch.

Return type
torch.FloatTensor

class dicee.models.transformers.**LayerNorm**(*ndim*, *bias*)

Bases: `torch.nn.Module`

LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False

weight

bias

forward(*input*)

class dicee.models.transformers.**SelfAttention**(*config*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F
```

(continues on next page)

(continued from previous page)

```
class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

c_attn

c_proj

attn_dropout

resid_dropout

n_head

n_embd

dropout

causal

flash = True

forward(*x*)

class dicee.models.transformers.**MLP**(*config*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F
```

(continues on next page)

(continued from previous page)

```
class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

> **Note**
>
> As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables
: **training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

c_fc

gelu

c_proj

dropout

forward(*x*)

class dicee.models.transformers.**Block**(*config*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F


class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

> **Note**
>
> As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

ln_1

attn

ln_2

mlp

forward(*x*)

class dicee.models.transformers.**GPTConfig**

block_size: int = 1024

vocab_size: int = 50304

n_layer: int = 12

n_head: int = 12

n_embd: int = 768

dropout: float = 0.0

bias: bool = False

causal: bool = True

class dicee.models.transformers.**GPT**(*config*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F


class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
```

(continues on next page)

(continued from previous page)

```
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

config

transformer

lm_head

get_num_params(*non_embedding=True*)

Return the number of parameters in the model. For non-embedding count (default), the position embeddings get subtracted. The token embeddings would too, except due to the parameter sharing these params are actually used as weights in the final layer, so we include them.

forward(*idx*, *targets=None*)

crop_block_size(*block_size*)

classmethod from_pretrained(*model_type*, *override_args=None*)

configure_optimizers(*weight_decay*, *learning_rate*, *betas*, *device_type*)

estimate_mfu(*fwdbwd_per_iter*, *dt*)

estimate model flops utilization (MFU) in units of A100 bfloat16 peak FLOPS

Attributes

logger
logger
logger

Classes

ADOPT	ADOPT Optimizer.
BaseKGELightning	Thin PyTorch Lightning wrapper shared by all KGE models.

continues on next page

Table 55 – continued from previous page

BaseKGE	Base class for all Knowledge Graph Embedding models.
IdentityClass	No-op normalisation / dropout placeholder.
Block	Base class for all neural network modules.
BaseKGE	Base class for all Knowledge Graph Embedding models.
DistMult	DistMult: bilinear diagonal knowledge graph embedding.
TransE	TransE: translation-based knowledge graph embedding.
TransH	TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.
MuRE	MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.
Shallom	Shallom: shallow neural model for relation prediction.
Pyke	Pyke: Physical Embedding Model for Knowledge Graphs.
CoKEConfig	Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.
CoKE	Contextualized Knowledge Graph Embedding (CoKE) model.
BaseKGE	Base class for all Knowledge Graph Embedding models.
ConEx	Convolutional ComplEx Knowledge Graph Embeddings
AConEx	Additive Convolutional ComplEx Knowledge Graph Embeddings
ComplEx	Base class for all Knowledge Graph Embedding models.
RotatE	RotatE: knowledge graph embedding by relational rotation in complex space.
BaseKGE	Base class for all Knowledge Graph Embedding models.
IdentityClass	No-op normalisation / dropout placeholder.
QMult	Base class for all Knowledge Graph Embedding models.
ConvQ	Convolutional Quaternion Knowledge Graph Embeddings
AConvQ	Additive Convolutional Quaternion Knowledge Graph Embeddings
BaseKGE	Base class for all Knowledge Graph Embedding models.
IdentityClass	No-op normalisation / dropout placeholder.
OMult	Base class for all Knowledge Graph Embedding models.
ConvO	Base class for all Knowledge Graph Embedding models.
AConvO	Additive Convolutional Octonion Knowledge Graph Embeddings
Keci	Keci: Knowledge Graph Embedding via Clifford Algebra.
CKeci	Without learning dimension scaling
DeCaL	Base class for all Knowledge Graph Embedding models.
KeciTransformer	Keci with Transformer architecture.
BaseKGE	Base class for all Knowledge Graph Embedding models.
PykeenKGE	A class for using knowledge graph embedding models implemented in Pykeen
BaseKGE	Base class for all Knowledge Graph Embedding models.
FMult	Learning Knowledge Neural Graphs
GFMult	Learning Knowledge Neural Graphs
FMult2	Learning Knowledge Neural Graphs
LFMult1	Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:
LFMult	Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:

continues on next page

Table 55 – continued from previous page

DualE	Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)

Functions

quaternion_mul(→ Tuple[torch.Tensor, torch.Tensor, ...)	Perform quaternion multiplication
quaternion_mul_with_unit_norm(*, Q_1, Q_2)	
octonion_mul(*, O_1, O_2)	
octonion_mul_norm(*, O_1, O_2)	

Package Contents

class dicee.models.**ADOPT**(*params: torch.optim.optimizer.ParamsT*, *lr: float | torch.Tensor = 0.001*, *betas: Tuple[float, float] = (0.9, 0.9999)*, *eps: float = 1e-06*, *clip_lambda: Callable[[int], float] | None = lambda step: ...*, *weight_decay: float = 0.0*, *decouple: bool = False*, ***, *foreach: bool | None = None*, *maximize: bool = False*, *capturable: bool = False*, *differentiable: bool = False*, *fused: bool | None = None*)

Bases: `torch.optim.optimizer.Optimizer`

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

Mathematical formulation:

m_t = β_1 * m_{t-1} + (1 - β_1) * clip(g_t / √(v_t)) v_t = β_2 * v_{t-1} + (1 - β_2) * g_t^2 θ_t = θ_{t-1} - α * m_t

where:

- θ_t: parameter at step t
- g_t: gradient at step t
- m_t: first moment estimate (momentum)
- v_t: second moment estimate (variance)
- α: learning rate
- β_1, β_2: exponential decay rates
- clip(): optional gradient clipping function

Reference:

Original implementation: https://github.com/iShohei220/adopt

Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.

- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients (β_1, β_2) for computing running averages of gradient and its square. β_1 controls momentum, β_2 controls variance. Default: (0.9, 0.9999)
- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: step**0.25 (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: step**0.25
- **weight_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0
- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

Raises

- **ValueError** – If learning rate, epsilon, betas, or weight_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↪decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↪step**0.5))
```

Note

- For most use cases, the default hyperparameters work well
- Consider using decouple=True for better generalization (similar to AdamW)
- The clip_lambda function helps stabilize training in early steps

`clip_lambda`

`__setstate__`(*state*)

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

Parameters

`state` (*`dict`*) – Optimizer state dictionary (typically from torch.load()).

Note

- This enables loading checkpoints saved with older ADOPT versions
- Step counters are converted to appropriate device/dtype for compatibility
- Capturable and fused modes require step tensors on parameter devices

`step`(*closure=None*)

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)
2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional adopt() API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (adopt()) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access grad_scale and found_inf attributes for gradient unscaling and inf/nan detection when used with GradScaler.

Parameters

`closure` (*`Callable, optional`*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (torch.enable_grad()) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: lambda: (loss := model(x), loss.backward(), loss)[-1] Default: None

Returns

The loss value returned by the closure, or None if no
closure was provided.

Return type
Optional[Tensor]

Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

Note

- Call zero_grad() before computing gradients for the next step
- CUDA graph capture is checked for safety when capturable=True
- The method is thread-safe for different parameter groups

`dicee.models.logger`

class `dicee.models.BaseKGELightning(*args, **kwargs)`

Bases: `lightning.LightningModule`

Thin PyTorch Lightning wrapper shared by all KGE models.

Provides the standard Lightning training loop hooks (`training_step`, `on_train_epoch_end`, `configure_optimizers`) as well as a helper for reporting model size. All concrete KGE models should extend *BaseKGE* rather than this class directly.

training_step_outputs = []

mem_of_model() → Dict

Size of model in MB and number of params

training_step(*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **batch** (*tuple*) – (x, y) for standard scoring, or (x, y_select, y) for sample-based labelling.
- **batch_idx** (*int, optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns
Scalar loss value for this batch.

Return type
torch.FloatTensor

loss_function(*yhat_batch: torch.FloatTensor*, *y_batch: torch.FloatTensor*) → torch.FloatTensor

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification).

Parameters

- **yhat_batch** (*torch.FloatTensor*) – Model output scores, shape (batch_size, *).
- **y_batch** (*torch.FloatTensor*) – Ground-truth labels of the same shape as *yhat_batch*.

Returns
Scalar loss value.

Return type
torch.FloatTensor

on_train_epoch_end(**args*, ***kwargs*)

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

test_epoch_end(*outputs: List[Any]*)

test_dataloader() → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

> **Warning**
>
> do not assign state in prepare_data

- `test()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

> **Note**
>
> If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

`val_dataloader``()` → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:paramref:`~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

> **Note**
>
> If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

predict_dataloader() → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

> **Note**
>
> Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Returns
A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

train_dataloader() → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:paramref:`~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`** to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

> **Warning**
>
> do not assign state in prepare_data

- `fit()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

configure_optimizers(*parameters=None*)

Instantiate and return the optimiser for training.

The optimiser type is taken from `self.optimizer_name` which is set in *BaseKGE.init_params_with_sanity_checking()* from the `--optim` argument. Supported values: `'SGD'`, `'Adam'`, `'Adopt'`, `'AdamW'`, `'NAdam'`, `'Adagrad'`, `'ASGD'`, `'Muon'`.

Parameters

parameters (*iterable, optional*) – Model parameters to optimise. Defaults to `self.parameters()` when `None`.

Returns

The configured optimiser instance.

Return type

torch.optim.Optimizer

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where T is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type
torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

class dicee.models.**IdentityClass**(*args=None*)

Bases: `torch.nn.Module`

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(*x*)

static forward(*x*)

class dicee.models.**Block**(*config*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```python
import torch.nn as nn
import torch.nn.functional as F


class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

ln_1

attn

ln_2

mlp

forward(*x*)

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns
head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns
head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type
torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

class dicee.models.**DistMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

DistMult: bilinear diagonal knowledge graph embedding.

Scores a triple (h, r, t) as the element-wise product of the head, relation, and tail embeddings summed over the embedding dimension:

```
f(h, r, t) = \sum_i  h_i  \cdot  r_i  \cdot  t_i
```

Simple yet effective baseline; incapable of modelling asymmetric relations.

References

Yang et al., *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*, ICLR 2015. https://arxiv.org/abs/1412.6575

name = 'DistMult'

k_vs_all_score(*emb_h: torch.FloatTensor*, *emb_r: torch.FloatTensor*, *emb_E: torch.FloatTensor*) → torch.FloatTensor

Score a head/relation batch against all entity embeddings.

Computes `(h * r) @ E^T` after applying hidden dropout and normalisation to the element-wise product.

Parameters

- **emb_h** (*torch.FloatTensor*) – Head entity embeddings, shape `(batch_size, embedding_dim)`.
- **emb_r** (*torch.FloatTensor*) – Relation embeddings, shape `(batch_size, embedding_dim)`.
- **emb_E** (*torch.FloatTensor*) – All entity embeddings, shape `(num_entities, embedding_dim)`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the k target entities per sample.

Returns

Shape `(batch_size, k)` score matrix.

Return type
torch.FloatTensor

score (*h: torch.FloatTensor*, *r: torch.FloatTensor*, *t: torch.FloatTensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` embedding triples.

Parameters

- **h** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **r** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **t** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

class dicee.models.**TransE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

TransE: translation-based knowledge graph embedding.

Models a relation *r* as a translation in embedding space such that `h + r ≈ t` for a true triple `(h, r, t)`. The score function is defined as:

```
f(h, r, t) = margin - ||h + r - t||_2
```

TransE is effective for 1-to-1 relations but struggles with reflexive, one-to-many, and many-to-one patterns.

References

Bordes et al., *Translating Embeddings for Modeling Multi-relational Data*, NeurIPS 2013. https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf

name = 'TransE'

margin = 4

score (*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*) → torch.FloatTensor

Score a batch of triples using the TransE margin-distance formula.

Parameters

- **head_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **rel_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **tail_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.

Returns
Shape `(batch_size,)` scores equal to `margin - ||h + r - t||_2`.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes `margin - ||h + r - e||_2` for every entity embedding *e*.

Parameters

x (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

class dicee.models.**TransH**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.

Addresses TransE's inability to model one-to-many, many-to-one, and many-to-many relations by letting each relation *r* define its own hyperplane. An entity is first projected onto that hyperplane before the translation is applied, so the same entity can be positioned differently depending on the relation. Concretely, for a triple `(h, r, t)` with unit-norm hyperplane normal `r_w` and in-hyperplane translation vector `r_d`:

```
h_r = h - (r_w^T h) * r_w
t_r = t - (r_w^T t) * r_w
f(h, r, t) = -||h_r + r_d - t_r||_2^2
```

`r_w` reuses the inherited `relation_embeddings` table and is normalised to unit norm before every use, since only a unit-norm normal yields a true orthogonal projection onto the hyperplane. `r_d` is a second, relation-specific embedding table analogous to TransE's relation vector.

References

Wang et al., *Knowledge Graph Embedding by Translating on Hyperplanes*, AAAI 2014. https://aaai.org/papers/8870-knowledge-graph-embedding-by-translating-on-hyperplanes/

name = 'TransH'

relation_normal_translations

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Every candidate tail entity must be projected onto the batch row's relation-specific hyperplane, and the hyperplane normal differs per batch row, so unlike TransE this materialises a full `(batch_size,` `num_entities, embedding_dim)` tensor of projected tail candidates – O(B * E * D) memory, versus TransE's O(B * E).

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Same hyperplane-projection logic as *forward_k_vs_all()*, but restricted to the *k* sampled tail candidates instead of every entity, so it costs `O(B * k * D)` memory instead of `O(B * E * D)`.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the *k* target entities per sample.

Returns
Shape `(batch_size, k)` score matrix.

Return type
torch.FloatTensor

class dicee.models.**MuRE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.

Scores a triple `(h, r, t)` as:

```
f(h, r, t) = -||R_r \odot h + t_r - t||_2 + b_h + b_t
```

`R_r` is a relation-specific diagonal matrix applied to the head via an element-wise (Hadamard) product; it reuses the inherited `relation_embeddings` table directly, shape `(num_relations, embedding_dim)`. `t_r` is a second, relation-specific translation vector (own embedding table, same shape convention: `(num_relations, embedding_dim)`) analogous to TransE's relation vector. `b_h` and `b_t` are learnable scalar biases indexed per entity (head and tail respectively), not per relation.

References

Balažević et al., *Multi-relational Poincaré Graph Embeddings*, NeurIPS 2019. https://arxiv.org/abs/1905.09791

name = 'MuRE'

relation_translations

b_h

b_t

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes `-||R_r \odot h + t_r - e||_2 + b_h + b_e` for every entity embedding *e*.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.LongTensor*) – Shape `(batch_size, k)` indices of the *k* target entities per sample.

Returns

Shape `(batch_size, k)` score matrix.

Return type

torch.FloatTensor

class dicee.models.**Shallom**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Shallom: shallow neural model for relation prediction.

Represents each triple as the concatenation of head and tail entity embeddings and feeds it through a two-layer MLP to predict the relation. Designed for the `RelationPrediction` labelling form.

References

Demir et al., *A Shallow Neural Model for Relation Prediction*, ISWC 2021. https://arxiv.org/abs/2101.09090

name = 'Shallom'

shallom

get_embeddings() → Tuple[numpy.ndarray, None]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

forward_k_vs_all(*x*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_triples(*x*) → torch.FloatTensor

Score a batch of triples by looking up relation scores from `forward_k_vs_all`.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

class dicee.models.**Pyke**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Pyke: Physical Embedding Model for Knowledge Graphs.

Scores a triple `(h, r, t)` based on the average pairwise distance between head-to-relation and relation-to-tail in embedding space:

```
f(h, r, t) = margin - (||h - r||_2 + ||r - t||_2) / 2
```

The model encodes geometric proximity between entities and the relations that connect them.

name = 'Pyke'

dist_func

margin = 1.0

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

Score a batch of triples using the Pyke distance formula.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.**CoKEConfig**

Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

block_size

Sequence length for transformer (3 for triples: head, relation, tail)

vocab_size

Total vocabulary size (num_entities + num_relations)

n_layer

Number of transformer layers

n_head

Number of attention heads per layer

n_embd

Embedding dimension (set to match model embedding_dim)

dropout

Dropout rate applied throughout the model

bias

Whether to use bias in linear layers

causal

Whether to use causal masking (False for bidirectional attention)

block_size: int = 3

vocab_size: int = None

n_layer: int = 6

n_head: int = 8

n_embd: int = None

dropout: float = 0.3

bias: bool = True

causal: bool = False

class dicee.models.**CoKE**(*args*, *config: CoKEConfig = CoKEConfig()*)

Bases: *dicee.models.base_model.BaseKGE*

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: https://arxiv.org/pdf/1911.02168.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head_emb, relation_emb, mask_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

name = 'CoKE'

config

pos_emb

mask_emb

blocks

ln_f

coke_dropout

forward_k_vs_all(*x: torch.Tensor*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

score(*emb_h*, *emb_r*, *emb_t*)

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
: **indexed_triple** (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
: **head_ent_emb, rel_ent_emb** – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
: torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
: **x** (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where T is `max_length_subword_tokens`.

Returns
: **head_ent_emb, rel_emb, tail_emb** – Each has shape `(batch_size, T, embedding_dim)`.

Return type
: torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
: **x** (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns
: **head_ent_emb, rel_emb** – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type
: torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

class dicee.models.**ConEx**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional ComplEx Knowledge Graph Embeddings

name = 'ConEx'

conv2d

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution(*C_1: Tuple[torch.Tensor, torch.Tensor]*, *C_2: Tuple[torch.Tensor, torch.Tensor]*) → torch.FloatTensor

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class dicee.models.**AConEx**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional ComplEx Knowledge Graph Embeddings

name = 'AConEx'

conv2d

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution(*C_1: Tuple[torch.Tensor, torch.Tensor]*, *C_2: Tuple[torch.Tensor, torch.Tensor]*) → torch.FloatTensor

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class dicee.models.**ComplEx**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *`forward_k_vs_all()`* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'ComplEx'

static score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*)

static k_vs_all_score(*emb_h: torch.FloatTensor*, *emb_r: torch.FloatTensor*, *emb_E: torch.FloatTensor*)

Parameters

- **emb_h**
- **emb_r**
- **emb_E**

forward_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape `(batch_size, k)` score matrix where k is the number of sampled target entities.

Return type

torch.FloatTensor

class dicee.models.**RotatE**(*args*)

Bases: *`dicee.models.base_model.BaseKGE`*

RotatE: knowledge graph embedding by relational rotation in complex space.

Represents each entity as a complex vector in ℂ^(d/2), obtained by splitting its `d`-dimensional embedding into a real half and an imaginary half. Each relation is represented by `d/2` phase angles θ that define a unit-modulus rotation `r_i = e^{iθ_i}`; since a phase angle needs only one real number, the relation embedding table is reinitialised at half of the entity embedding size. A true triple `(h, r, t)` should satisfy `h ∘ r ≈ t` under element-wise complex multiplication, giving the score:

```
f(h, r, t) = margin - ||h ∘ r - t||_2
```

Unlike TransE, RotatE can model symmetric, antisymmetric, inverse, and composition relation patterns.

References

Sun et al., *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*, ICLR 2019. https://arxiv.org/abs/1902.10197

name = 'RotatE'

margin = 6.0

half_dim

relation_embeddings

score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*) → torch.FloatTensor

Score a batch of triples using the RotatE margin-distance formula.

Parameters

- **head_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **tail_ent_emb** (*torch.FloatTensor*) – Each has shape `(batch_size, embedding_dim)`.
- **rel_ent_emb** (*torch.FloatTensor*) – Shape `(batch_size, d/2)` relation phase angles θ.

Returns

Shape `(batch_size,)` scores equal to `margin - ||h ∘ r - t||_2`.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes `margin - ||h ∘ r - e||_2` for every entity embedding *e*.

Parameters

x (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*x: torch.Tensor*, *target_entity_idx: torch.Tensor*) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Computes `margin - ||h ∘ r - e||_2` for each of the *k* sampled entities *e*.

Parameters

- **x** (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.
- **target_entity_idx** (*torch.Tensor*) – Shape `(batch_size, k)` indices of the *k* target entities per sample.

Returns
Shape `(batch_size, k)` score matrix.

Return type
torch.FloatTensor

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *`forward_k_vs_sample()`*
- `(batch, 3)` tensor → *`forward_triples()`*
- `(batch, 2)` tensor → *`forward_k_vs_all()`*
- BPE triple tensor → *`forward_byte_pair_encoded_triple()`*
- BPE pair tensor → *`forward_byte_pair_encoded_k_vs_all()`*

Parameters

- **x** (*`torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]`*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*`torch.LongTensor, optional`*) – Target entity indices used by *`forward_k_vs_sample()`*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*`torch.LongTensor`*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where k is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where T is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns

head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type

torch.FloatTensor

get_embeddings`()` → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

class dicee.models.**IdentityClass**(*args=None*)

Bases: `torch.nn.Module`

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(*x*)

static forward(*x*)

dicee.models.**quaternion_mul**(*, *Q_1*, *Q_2*) → Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

Perform quaternion multiplication :param Q_1: :param Q_2: :return:

dicee.models.**quaternion_mul_with_unit_norm**(*, *Q_1*, *Q_2*)

class dicee.models.**QMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'QMult'

explicit = True

quaternion_multiplication_followed_by_inner_product(*h*, *r*, *t*)

Parameters

- **h** – shape: (**batch_dims*, dim) The head representations.
- **r** – shape: (**batch_dims*, dim) The head representations.
- **t** – shape: (**batch_dims*, dim) The tail representations.

Returns

Triple scores.

static quaternion_normalizer(*x: torch.FloatTensor*) → torch.FloatTensor

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^{d} |x_i|^2 = \sum_{i=1}^{d} (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

Parameters

x – The vector.

Returns

The normalized vector.

score(*head_ent_emb: torch.FloatTensor*, *rel_ent_emb: torch.FloatTensor*, *tail_ent_emb: torch.FloatTensor*)

k_vs_all_score(*bpe_head_ent_emb*, *bpe_rel_ent_emb*, *E*)

Parameters

- **bpe_head_ent_emb**
- **bpe_rel_ent_emb**
- **E**

forward_k_vs_all(*x*)

Parameters

x

forward_k_vs_sample(*x*, *target_entity_idx*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**ConvQ**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional Quaternion Knowledge Graph Embeddings

name = 'ConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(*Q_1*, *Q_2*)

forward_triples(*indexed_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**AConvQ**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional Quaternion Knowledge Graph Embeddings

name = 'AConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(*Q_1*, *Q_2*)

forward_triples(*indexed_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor* | *Tuple[torch.LongTensor, torch.LongTensor]*, *y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where *T* is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of *T* subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type

torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

class dicee.models.**IdentityClass**(*args=None*)

Bases: `torch.nn.Module`

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(*x*)

static forward(*x*)

dicee.models.**octonion_mul**(*, *O_1*, *O_2*)

dicee.models.**octonion_mul_norm**(*, *O_1*, *O_2*)

class dicee.models.**OMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'OMult'

static octonion_normalizer(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

score(*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*)

k_vs_all_score(*bpe_head_ent_emb, bpe_rel_ent_emb, E*)

forward_k_vs_all(*x*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**ConvO**(*args: dict*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'ConvO'

conv2d

fc_num_input

fc1

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

residual_convolution(*O_1, O_2*)

forward_triples(*x: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**AConvO**(*args: dict*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional Octonion Knowledge Graph Embeddings

name = 'AConvO'

conv2d

fc_num_input

fc1

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

residual_convolution(*O_1, O_2*)

forward_triples(*x: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,…,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.**Keci**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Keci: Knowledge Graph Embedding via Clifford Algebra.

Embeds entities and relations as multi-vectors in the Clifford algebra Cl_{p,q}(R^d) and scores triples via the Clifford product. The algebra is parameterised by two non-negative integers *p* and *q*:

- `p = 0, q = 0` — reduces to a standard bilinear (DistMult-like) model.

- `p = 0, q = 1` — equivalent to ComplEx.
- Larger `p` and `q` capture higher-order geometric interactions.

The embedding dimension must satisfy `embedding_dim % (p + q + 1) == 0`; the resulting quotient is stored as `self.r`.

Parameters
args (*dict*) – Configuration dictionary. Recognised keys (beyond those in *BaseKGE*): `p` (int, default 0) and `q` (int, default 0).

References

Demir et al., *Clifford Embeddings — A Generalized Approach for Embedding in Normed Algebras*, ECML 2023.

name = 'Keci'

p

q

r

requires_grad_for_interactions = True

compute_sigma_pp(*hp*, *rp*)

Compute sigma_{pp} = sum_{i=1}^{p-1} sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k

sigma_{pp} captures the interactions between along p bases For instance, let p e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):
results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_qq(*hq*, *rq*)

Compute sigma_{qq} = sum_{j=1}^{p+q-1} sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k sigma_{q} captures the interactions between along q bases For instance, let q e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):
results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

`compute_sigma_pq`(***, *hp*, *hq*, *rp*, *rq*)

sum_{i=1}^{p} sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

`apply_coefficients`(*hp*, *hq*, *rp*, *rq*)

Multiplying a base vector with its scalar coefficient

`clifford_multiplication`(*h0*, *hp*, *hq*, *r0*, *rp*, *rq*)

Compute our CL multiplication

h = h_0 + sum_{i=1}^p h_i e_i + sum_{j=p+1}^{p+q} h_j e_j r = r_0 + sum_{i=1}^p r_i e_i + sum_{j=p+1}^{p+q} r_j e_j

ei ^2 = +1 for i =< i =< p ej ^2 = -1 for p < j =< p+q ei ej = -eje1 for i

eq j

h r = sigma_0 + sigma_p + sigma_q + sigma_{pp} + sigma_{q}+ sigma_{pq} where

(1) sigma_0 = h_0 r_0 + sum_{i=1}^p (h_0 r_i) e_i - sum_{j=p+1}^{p+q} (h_j r_j) e_j

(2) sigma_p = sum_{i=1}^p (h_0 r_i + h_i r_0) e_i

(3) sigma_q = sum_{j=p+1}^{p+q} (h_0 r_j + h_j r_0) e_j

(4) sigma_{pp} = sum_{i=1}^{p-1} sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k

(5) sigma_{qq} = sum_{j=1}^{p+q-1} sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k

(6) sigma_{pq} = sum_{i=1}^{p} sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j

`construct_cl_multivector`(*x: torch.FloatTensor*, *r: int*, *p: int*, *q: int*)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a flat embedding vector into the three Clifford components.

Given an embedding `x` of dimension `d = r + r*p + r*q`, returns the scalar part `a0`, the *p*-blade part `ap`, and the *q*-blade part `aq`.

Parameters

- **x** (*torch.FloatTensor*) – Shape `(batch_size, d)`.
- **r** (*int*) – Scalar block size (`embedding_dim // (p + q + 1)`).
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape `(batch_size, r)` — scalar (grade-0) part.
- **ap** (*torch.FloatTensor*) – Shape `(batch_size, r, p)` — positive-blade part.
- **aq** (*torch.FloatTensor*) – Shape `(batch_size, r, q)` — negative-blade part.

forward_k_vs_with_explicit(*x: torch.Tensor*) → torch.FloatTensor

KvsAll scoring using an explicit loop over sigma_pp/qq/pq terms.

Functionally equivalent to *forward_k_vs_all()* but computes the higher-order interaction terms (sigma_pp, sigma_qq, sigma_pq) with explicit nested loops rather than einsum contractions. Kept for reference and correctness verification.

Parameters

x (*torch.Tensor*) – Shape `(batch_size, 2)` integer tensor `[head_idx, relation_idx]`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

k_vs_all_score(*bpe_head_ent_emb: torch.FloatTensor*, *bpe_rel_ent_emb: torch.FloatTensor*, *E: torch.FloatTensor*) → torch.FloatTensor

Compute Clifford-product scores for a head/relation batch vs. all entities.

Decomposes the head-entity and relation embeddings into Clifford multi-vectors, performs the Cl_{p,q} product, and inner-products the result against the entity embedding matrix *E*.

Parameters

- **bpe_head_ent_emb** (*torch.FloatTensor*) – Head-entity embeddings, shape `(batch_size, embedding_dim)`.
- **bpe_rel_ent_emb** (*torch.FloatTensor*) – Relation embeddings, shape `(batch_size, embedding_dim)`.
- **E** (*torch.FloatTensor*) – All entity embeddings, shape `(num_entities, embedding_dim)`.

Returns

Shape `(batch_size, num_entities)` score matrix.

Return type

torch.FloatTensor

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

(1) Retrieve real-valued embedding vectors for heads and relations mathbb{R}^d .

(2) Construct head entity and relation embeddings according to Cl_{p,q}(mathbb{R}^d) .

(3) Perform Cl multiplication

(4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this funcitons are identical Parameter ——— x: torch.LongTensor with (n,2) shape :rtype: torch.FloatTensor with (n, **|E|**) shape

construct_batch_selected_cl_multivector(*x: torch.FloatTensor*, *r: int*, *p: int*, *q: int*) → tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a batched, *k*-selected embedding tensor into Clifford components.

A variant of *construct_cl_multivector()* for tensors that have an extra *k* dimension (e.g. when scoring against *k* sampled targets).

Parameters

- **x** (*torch.FloatTensor*) – Shape `(batch_size, k, d)`.
- **r** (*int*) – Scalar block size.
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape `(batch_size, k, r)`.
- **ap** (*torch.FloatTensor*) – Shape `(batch_size, k, r, p)`.
- **aq** (*torch.FloatTensor*) – Shape `(batch_size, k, r, q)`.

forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx: torch.LongTensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,2) shape

target_entity_idx: torch.LongTensor with (n, k) shape k denotes the selected number of examples.

rtype
torch.FloatTensor with (n, k) shape

score(*h*, *r*, *t*)

forward_triples(*x: torch.Tensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,3) shape

rtype
torch.FloatTensor with (n) shape

class dicee.models.**CKeci**(*args*)

Bases: *Keci*

Without learning dimension scaling

name = 'CKeci'

requires_grad_for_interactions = False

class dicee.models.**DeCaL**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'DeCaL'

entity_embeddings

relation_embeddings

p

q

r

re

forward_triples (*x: torch.Tensor*) → torch.FloatTensor

Parameter

x: torch.LongTensor with (n,) shape

rtype
torch.FloatTensor with (n) shape

cl_pqr (*a: torch.tensor*) → torch.tensor

Input: tensor(batch_size, emb_dim) —> output: tensor with 1+p+q+r components with size (batch_size, emb_dim/(1+p+q+r)) each.

1) takes a tensor of size (batch_size, emb_dim), split it into 1 + p + q +r components, hence 1+p+q+r must be a divisor of the emb_dim. 2) Return a list of the 1+p+q+r components vectors, each are tensors of size (batch_size, emb_dim/(1+p+q+r))

compute_sigmas_single (*list_h_emb*, *list_r_emb*, *list_t_emb*)

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s0 = h_0 r_0 t_0 s1 = \sum_{i=1}^{p} h_i r_i t_0 s2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 s3 = \sum_{i=1}^{q} (h_0 r_i t_i + h_i r_0 t_i) s4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) s5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t$$

and return:

$$sigma_0 t = \sigma_0 \cdot t_0 = s0 + s1 - s2 s3, s4 and s5$$

compute_sigmas_multivect (*list_h_emb*, *list_r_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_p p = \sum_{i=1}^{p-1} \sum_{i'=i+1}^{p} (h_i r_{i'} - h_{i'} r_i)(models the interactions between e_i and e'_i for 1 <= i, i' <= p) \sigma_q q = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'}$$

For different base vector interactions, we have

$$\sigma_p q = \sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i)(interactionsnbetweene_i ande_j for 1 <= i <= pandp + 1 <= j <= p + q)\sigma_p r = \sum_{i=}^{p}$$

forward_k_vs_all (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

(1) Retrieve real-valued embedding vectors for heads and relations

(2) Construct head entity and relation embeddings according to Cl_{p,q, r}(mathbb{R}^d) .

(3) Perform Cl multiplication

(4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this funcitons are identical Parameter ——— x: torch.LongTensor with (n,) shape :rtype: torch.FloatTensor with (n, **|E|**) shape

apply_coefficients (*h0*, *hp*, *hq*, *hk*, *r0*, *rp*, *rq*, *rk*)

Multiplying a base vector with its scalar coefficient

construct_cl_multivector (*x: torch.FloatTensor*, *re: int*, *p: int*, *q: int*, *r: int*) → tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors Cl_{p,q,r}(mathbb{R}^d)

Parameter

x: torch.FloatTensor with (n,d) shape

returns

- **a0** (*torch.FloatTensor*)
- **ap** (*torch.FloatTensor*)
- **aq** (*torch.FloatTensor*)
- **ar** (*torch.FloatTensor*)

compute_sigma_pp (*hp*, *rp*)

Compute .. math:

```
\sigma_{p,p}^* = \sum_{i=1}^{p-1}\sum_{i'=i+1}^{p}(x_iy_{i'}-x_{i'}y_i)
```

sigma_{pp} captures the interactions between along p bases For instance, let p e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):
results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_qq(*hq*, *rq*)

Compute

$$\sigma^*_{q,q} = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) Eq.16$$

sigma_{q} captures the interactions between along q bases For instance, let q e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3 , and e_2 e_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):
results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_rr(*hk*, *rk*)

$$\sigma^*_{r,r} = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^{p} (x_k y_{k'} - x_{k'} y_k)$$

compute_sigma_pq(*, *hp*, *hq*, *rp*, *rq*)

Compute

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

compute_sigma_pr(*, *hp*, *hk*, *rp*, *rk*)

Compute

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

compute_sigma_qr(*, *hq*, *hk*, *rq*, *rk*)

$$\sum_{i=1}^{p} \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):
sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

class dicee.models.**KeciTransformer**(*args*)

Bases: *Keci*

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

name = 'KeciTransformer'

use_clifford_mul

seq_len = 1

transformer

lm_head

forward_k_vs_all(*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

Parameter

x: torch.LongTensor with (n,2) shape

rtype
torch.FloatTensor with (n, |E|) shape

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters
args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type
torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor], y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where T is `max_length_subword_tokens`.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns

head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type

torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

dicee.models.**logger**

class dicee.models.**PykeenKGE**(*args: dict*)

Bases: *dicee.models.base_model.BaseKGE*

A class for using knowledge graph embedding models implemented in Pykeen

Notes: Pykeen_DistMult: C Pykeen_ComplEx: Pykeen_QuatE: Pykeen_MuRE: Pykeen_CP: Pykeen_HolE: Pykeen_HolE: Pykeen_HolE: Pykeen_TransD: Pykeen_TransE: Pykeen_TransF: Pykeen_TransH: Pykeen_TransR:

model_kwargs

name

model

loss_history = []

args

entity_embeddings = None

relation_embeddings = None

forward_k_vs_all(*x: torch.LongTensor*)

=> Explicit version by this we can apply bn and dropout

(1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r = self.get_head_relation_representation(x) # (2) Reshape (1). if self.last_dim > 0:

h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim, self.last_dim)

(3) Reshape all entities. if self.last_dim > 0:

t = self.entity_embeddings.weight.reshape(self.num_entities, self.embedding_dim, self.last_dim)

else:

t = self.entity_embeddings.weight

(4) Call the score_t from interactions to generate triple scores. return self.interaction.score_t(h=h, r=r, all_entities=t, slice_size=1)

forward_triples(*x: torch.LongTensor*) → torch.FloatTensor

=> Explicit version by this we can apply bn and dropout

(1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t = self.get_triple_representation(x) # (2) Reshape (1). if self.last_dim > 0:

h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim, self.last_dim) t = t.reshape(len(x), self.embedding_dim, self.last_dim)

(3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice_size=None, slice_dim=0)

abstractmethod forward_k_vs_sample(*x: torch.LongTensor*, *target_entity_idx*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class dicee.models.**BaseKGE**(*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of `(h, r, t)` triples.
- *forward_k_vs_all()* — score a `(h, r)` batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

forward_byte_pair_encoded_k_vs_all(*x: torch.LongTensor*) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and T is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple(*x: Tuple[torch.LongTensor, torch.LongTensor]*) → torch.FloatTensor

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` BPE token indices.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward(*x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor], y_idx: torch.LongTensor = None*) → torch.FloatTensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple `(x, y_idx)` → *forward_k_vs_sample()*
- `(batch, 3)` tensor → *forward_triples()*
- `(batch, 2)` tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a `(triple_idx, target_idx)` tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor, optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns
Score tensor whose shape depends on the selected scorer.

Return type
torch.FloatTensor

forward_triples(*x: torch.LongTensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

forward_k_vs_all(**args*, ***kwargs*)

Score a `(head, relation)` batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns
Shape `(batch_size, num_entities)` score matrix.

Return type
torch.FloatTensor

forward_k_vs_sample(**args*, ***kwargs*)

Score a `(head, relation)` batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns
Shape `(batch_size, k)` score matrix where *k* is the number of sampled target entities.

Return type
torch.FloatTensor

get_triple_representation(*idx_hrt*)
→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters
idx_hrt (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor with columns `[head_idx, relation_idx, tail_idx]`.

Returns
head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_head_relation_representation(*indexed_triple*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters
indexed_triple (*torch.LongTensor*) – Shape `(batch_size, 2)` integer tensor with columns `[head_idx, relation_idx]`.

Returns
head_ent_emb, rel_ent_emb – Each has shape `(batch_size, embedding_dim)` after applying the configured dropout and normalisation.

Return type
torch.FloatTensor

get_sentence_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3, T)` where *T* is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb, tail_emb – Each has shape `(batch_size, T, embedding_dim)`.

Return type
torch.FloatTensor

get_bpe_head_and_relation_representation(*x: torch.LongTensor*) → Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of *T* subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 2, T)` where dim 1 indexes `[head, relation]` and *T* is `max_length_subword_tokens`.

Returns
head_ent_emb, rel_emb – Each has shape `(batch_size, T, embedding_dim)`, L2-normalised over the `(T, D)` dimensions.

Return type
torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape `(num_entities, embedding_dim)`.
- **relation_embeddings** (*numpy.ndarray*) – Shape `(num_relations, embedding_dim)`.

dicee.models.**logger**

class dicee.models.**FMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'FMult'

entity_embeddings

relation_embeddings

k

num_sample = 50

gamma

roots

weights

compute_func(*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func(*weights*, *x: torch.FloatTensor*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.**GFMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'GFMult'

entity_embeddings

relation_embeddings

k

num_sample = 250

roots

weights

compute_func(*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func(*weights*, *x: torch.FloatTensor*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.**FMult2**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'FMult2'

n_layers = 3

k

n = 50

score_func = 'compositional'

discrete_points

entity_embeddings

relation_embeddings

build_func(*Vec*)

build_chain_funcs(*list_Vec*)

compute_func(*W, b, x*) → torch.FloatTensor

function(*list_W, list_b*)

trapezoid(*list_W, list_b*)

forward_triples(*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of `(head, relation, tail)` index triples.

Parameters

x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns

Shape `(batch_size,)` triple scores.

Return type

torch.FloatTensor

class dicee.models.**LFMult1**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Embedding with trigonometric functions. We represent all entities and relations in the complex number space as: f(x) = sum_{k=0}^{k=d-1}wk e^{kix}. and use the three differents scoring function as in the paper to evaluate the score

name = 'LFMult1'

entity_embeddings

relation_embeddings

forward_triples(*idx_triple*)

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

tri_score(*h, r, t*)

vtp_score(*h, r, t*)

class dicee.models.**LFMult**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Embedding with polynomial functions. We represent all entities and relations in the polynomial space as: f(x) = sum_{i=0}^{d-1} a_k x^{i%d} and use the three differents scoring function as in the paper to evaluate the score. We also consider combining with Neural Networks.

name = 'LFMult'

entity_embeddings

relation_embeddings

degree

m

x_values

forward_triples(*idx_triple*)

Score a batch of `(head, relation, tail)` index triples.

Parameters
x (*torch.LongTensor*) – Shape `(batch_size, 3)` integer tensor where each row is `[head_idx, relation_idx, tail_idx]`.

Returns
Shape `(batch_size,)` triple scores.

Return type
torch.FloatTensor

construct_multi_coeff(*x*)

poly_NN(*x, coefh, coefr, coeft*)

Constructing a 2 layers NN to represent the embeddings. h = sigma(wh^T x + bh), r = sigma(wr^T x + br), t = sigma(wt^T x + bt)

linear(*x*, *w*, *b*)

scalar_batch_NN(*a*, *b*, *c*)

element wise multiplication between a,b and c: Inputs : a, b, c ====> torch.tensor of size batch_size x m x d Output : a tensor of size batch_size x d

tri_score(*coeff_h*, *coeff_r*, *coeff_t*)

this part implement the trilinear scoring techniques:

score(h,r,t) = int_{0}{1} h(x)r(x)t(x) dx = sum_{i,j,k = 0}^{d-1} dfrac{a_i*b_j*c_k}{1+(i+j+k)%d}

1. generate the range for i,j and k from [0 d-1]

2. perform dfrac{a_i*b_j*c_k}{1+(i+j+k)%d} in parallel for every batch

3. take the sum over each batch

vtp_score(*h*, *r*, *t*)

this part implement the vector triple product scoring techniques:

score(h,r,t) = int_{0}{1} h(x)r(x)t(x) dx = sum_{i,j,k = 0}^{d-1} dfrac{a_i*c_j*b_k - b_i*c_j*a_k}{(1+(i+j)%d)(1+k)}

1. generate the range for i,j and k from [0 d-1]
2. Compute the first and second terms of the sum
3. Multiply with then denominator and take the sum
4. take the sum over each batch

comp_func(*h*, *r*, *t*)

this part implement the function composition scoring techniques: i.e. score = <hor, t>

polynomial(*coeff*, *x*, *degree*)

This function takes a matrix tensor of coefficients (coeff), a tensor vector of points x and range of integer [0,1,…d] and return a vector tensor (coeff[0][0] + coeff[0][1]x +…+ coeff[0][d]x^d,

coeff[1][0] + coeff[1][1]x +…+ coeff[1][d]x^d)

pop(*coeff*, *x*, *degree*)

This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (coeff), a matrix tensor of points x and range of integer [0,1,…d]

and return a tensor (coeff[0][0] + coeff[0][1]x +…+ coeff[0][d]x^d,

coeff[1][0] + coeff[1][1]x +…+ coeff[1][d]x^d)

class dicee.models.**DualE**(*args*)

Bases: *dicee.models.base_model.BaseKGE*

Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)

name = 'DualE'

entity_embeddings

relation_embeddings

num_ent = None

kvsall_score(*e_1_h, e_2_h, e_3_h, e_4_h, e_5_h, e_6_h, e_7_h, e_8_h, e_1_t, e_2_t, e_3_t, e_4_t, e_5_t, e_6_t, e_7_t, e_8_t, r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8*) → torch.tensor

KvsAll scoring function

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_triples(*idx_triple: torch.tensor*) → torch.tensor

Negative Sampling forward pass:

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_k_vs_all(*x*)

KvsAll forward pass

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

T(*x: torch.tensor*) → torch.tensor

Transpose function

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)

dicee.query_generator

Attributes

logger

Classes

QueryGenerator

Module Contents

`dicee.query_generator.`**`logger`**

class `dicee.query_generator.`**`QueryGenerator`**(*train_path*, *val_path: str*, *test_path: str*, *ent2id: Dict = None*, *rel2id: Dict = None*, *seed: int = 1*, *gen_valid: bool = False*, *gen_test: bool = True*)

`train_path`

`val_path`

`test_path`

`gen_valid = False`

`gen_test = True`

`seed = 1`

`max_ans_num = 1000000.0`

`mode`

`ent2id = None`

`rel2id: Dict = None`

`ent_in: Dict`

`ent_out: Dict`

`query_name_to_struct`

`list2tuple`(*list_data*)

`tuple2list`(*x: List | Tuple*) → List | Tuple

Convert a nested tuple to a nested list.

`set_global_seed`(*seed: int*)

Set seed

`construct_graph`(*paths: List[str]*) → Tuple[Dict, Dict]

Construct graph from triples Returns dicts with incoming and outgoing edges

`fill_query`(*query_structure: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*, *answer: int*) → bool

Private method for fill_query logic.

`achieve_answer`(*query: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*) → set

Private method for achieve_answer logic. @TODO: Document the code

`write_links`(*ent_out*, *small_ent_out*)

`ground_queries`(*query_structure: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*, *small_ent_in: Dict*, *small_ent_out: Dict*, *gen_num: int*, *query_name: str*)

Generating queries and achieving answers

`unmap`(*query_type*, *queries*, *tp_answers*, *fp_answers*, *fn_answers*)

`unmap_query`(*query_structure*, *query*, *id2ent*, *id2rel*)

generate_queries(*query_struct: List*, *gen_num: int*, *query_type: str*)

Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting queries and answers in return @ TODO: create a class for each single query struct

save_queries(*query_type: str*, *gen_num: int*, *save_path: str*)

abstractmethod load_queries(*path*)

get_queries(*query_type: str*, *gen_num: int*)

static save_queries_and_answers(*path: str*, *data: List[Tuple[str, Tuple[collections.defaultdict]]]*) → None

Save Queries into Disk

static load_queries_and_answers(*path: str*) → List[Tuple[str, Tuple[collections.defaultdict]]]

Load Queries from Disk to Memory

dicee.read_preprocess_save_load_kg

Submodules

dicee.read_preprocess_save_load_kg.preprocess

Attributes

logger	

Classes

PreprocessKG	Preprocess the data in memory

Module Contents

dicee.read_preprocess_save_load_kg.preprocess.**logger**

class dicee.read_preprocess_save_load_kg.preprocess.**PreprocessKG**(*kg*)

Preprocess the data in memory

kg

start() → None

Preprocess train, valid and test datasets stored in knowledge graph instance

Parameter

rtype

None

preprocess_with_byte_pair_encoding()

preprocess_with_byte_pair_encoding_with_padding() → None

Preprocess with byte pair encoding and add padding

preprocess_with_pandas() → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

preprocess_with_polars() → None

Preprocess with polars: add reciprocal triples and create indexed datasets

sequential_vocabulary_construction() → None

(1) Read input data into memory

(2) Remove triples with a condition

(3) **Serialize vocabularies in a pandas dataframe where**
=> the index is integer and => a single column is string (e.g. URI)

dicee.read_preprocess_save_load_kg.read_from_disk

Attributes

logger	

Classes

ReadFromDisk	Read the data from disk into memory

Module Contents

dicee.read_preprocess_save_load_kg.read_from_disk.**logger**

class dicee.read_preprocess_save_load_kg.read_from_disk.**ReadFromDisk**(*kg*)

Read the data from disk into memory

kg

start() → None

Read a knowledge graph from disk into memory

Data will be available at the train_set, test_set, valid_set attributes.

Parameter

None

rtype
None

add_noisy_triples_into_training()

dicee.read_preprocess_save_load_kg.save_load_disk

Attributes

logger

Classes

LoadSaveToDisk

Module Contents

`dicee.read_preprocess_save_load_kg.save_load_disk.`**`logger`**

`class` `dicee.read_preprocess_save_load_kg.save_load_disk.`**`LoadSaveToDisk`**`(`*kg*`)`

`kg`

`save``()`

`load``()`

dicee.read_preprocess_save_load_kg.util

Attributes

logger

Functions

polars_dataframe_indexer(→ polars.DataFrame)	Replaces 'subject', 'relation', and 'object' columns in the input Polars DataFrame with their corresponding index values
pandas_dataframe_indexer(→ pandas.DataFrame)	Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values
apply_reciprocal_or_noise(add_reciprocal, eval_model)	Add reciprocal triples if conditions are met
timeit(func)	
read_with_polars(→ polars.DataFrame)	Load and Preprocess via Polars
read_with_pandas(data_path[, read_only_few, ...])	Load and Preprocess via Pandas
read_from_disk(→ Tuple[polars.DataFrame, pandas.DataFrame])	
count_triples(→ int)	Returns the total number of triples in the triple store.
fetch_worker(endpoint, offsets, chunk_size, ...)	Worker process: fetch assigned chunks and save to disk with per-worker tqdm.
read_from_triple_store_with_polars(endpoint[, ...])	Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.
read_from_triple_store_with_pandas([endpoint]	Read triples from triple store into pandas dataframe
get_er_vocab(data[, file_path])	
get_re_vocab(data[, file_path])	
get_ee_vocab(data[, file_path])	

continues on next page

Table 66 – continued from previous page

`create_constraints`(triples[, file_path])	
`load_with_pandas`(→ None)	Deserialize data
`save_numpy_ndarray`(*, data, file_path)	
`load_numpy_ndarray`(*, file_path)	
`save_pickle`(*, data[, file_path])	
`load_pickle`(*[, file_path])	
`create_recipriocal_triples`(x)	Add inverse triples into dask dataframe
`dataset_sanity_checking`(→ None)	

Module Contents

`dicee.read_preprocess_save_load_kg.util.`**`logger`**

`dicee.read_preprocess_save_load_kg.util.`**`polars_dataframe_indexer`**`(`
df_polars: polars.DataFrame, *idx_entity: polars.DataFrame*, *idx_relation: polars.DataFrame*`)`
→ polars.DataFrame

Replaces 'subject', 'relation', and 'object' columns in the input Polars DataFrame with their corresponding index values from the entity and relation index DataFrames.

This function processes the DataFrame in three main steps: 1. Replace the 'relation' values with the corresponding index from *idx_relation*. 2. Replace the 'subject' values with the corresponding index from *idx_entity*. 3. Replace the 'object' values with the corresponding index from *idx_entity*.

Parameters:

df_polars
[polars.DataFrame] The input Polars DataFrame containing columns: 'subject', 'relation', and 'object'.

idx_entity
[polars.DataFrame] A Polars DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: 'entity' and 'index'.

idx_relation
[polars.DataFrame] A Polars DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: 'relation' and 'index'.

Returns:

polars.DataFrame
A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

Example Usage:

```
>>> df_polars = pl.DataFrame({
        "subject": ["Alice", "Bob", "Charlie"],
        "relation": ["knows", "works_with", "lives_in"],
        "object": ["Dave", "Eve", "Frank"]
    })
>>> idx_entity = pl.DataFrame({
        "entity": ["Alice", "Bob", "Charlie", "Dave", "Eve", "Frank"],
        "index": [0, 1, 2, 3, 4, 5]
    })
>>> idx_relation = pl.DataFrame({
```

(continues on next page)

(continued from previous page)

```
        "relation": ["knows", "works_with", "lives_in"],
        "index": [0, 1, 2]
    })
>>> polars_dataframe_indexer(df_polars, idx_entity, idx_relation)
```

Steps:

1. Join the input DataFrame *df_polars* on the 'relation' column with *idx_relation* to replace the relations with their indices.
2. Join on 'subject' to replace it with the corresponding entity index using a left join on *idx_entity*.
3. Join on 'object' to replace it with the corresponding entity index using a left join on *idx_entity*.
4. Select only the 'subject', 'relation', and 'object' columns to return the final result.

dicee.read_preprocess_save_load_kg.util.**pandas_dataframe_indexer**(*df_pandas: pandas.DataFrame*, *idx_entity: pandas.DataFrame*, *idx_relation: pandas.DataFrame*) → pandas.DataFrame

Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values from the entity and relation index DataFrames.

Parameters:

df_pandas
[pd.DataFrame] The input Pandas DataFrame containing columns: 'subject', 'relation', and 'object'.

idx_entity
[pd.DataFrame] A Pandas DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: 'entity' and 'index'.

idx_relation
[pd.DataFrame] A Pandas DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: 'relation' and 'index'.

Returns:

pd.DataFrame
A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

dicee.read_preprocess_save_load_kg.util.**apply_reciprocal_or_noise**(*add_reciprocal: bool*, *eval_model: str*, *df: object = None*, *info: str = None*)

Add reciprocal triples if conditions are met

dicee.read_preprocess_save_load_kg.util.**timeit**(*func*)

dicee.read_preprocess_save_load_kg.util.**read_with_polars**(*data_path*, *read_only_few: int = None*, *sample_triples_ratio: float = None*, *separator: str = None*) → polars.DataFrame

Load and Preprocess via Polars

dicee.read_preprocess_save_load_kg.util.**read_with_pandas**(*data_path*, *read_only_few: int = None*, *sample_triples_ratio: float = None*, *separator: str = None*)

Load and Preprocess via Pandas

dicee.read_preprocess_save_load_kg.util.**read_from_disk**(*data_path: str*, *read_only_few: int = None*, *sample_triples_ratio: float = None*, *backend: str = None*, *separator: str = None*) → Tuple[polars.DataFrame, pandas.DataFrame]

dicee.read_preprocess_save_load_kg.util.**count_triples**(*endpoint: str*) → int

Returns the total number of triples in the triple store.

dicee.read_preprocess_save_load_kg.util.**fetch_worker**(*endpoint: str*, *offsets: list[int]*, *chunk_size: int*, *output_dir: str*, *worker_id: int*)

Worker process: fetch assigned chunks and save to disk with per-worker tqdm.

dicee.read_preprocess_save_load_kg.util.**read_from_triple_store_with_polars**(*endpoint: str*, *chunk_size: int = 500000*, *output_dir: str = 'triples_parquet'*)

Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.

dicee.read_preprocess_save_load_kg.util.**read_from_triple_store_with_pandas**(*endpoint: str = None*)

Read triples from triple store into pandas dataframe

dicee.read_preprocess_save_load_kg.util.**get_er_vocab**(*data*, *file_path: str = None*)

dicee.read_preprocess_save_load_kg.util.**get_re_vocab**(*data*, *file_path: str = None*)

dicee.read_preprocess_save_load_kg.util.**get_ee_vocab**(*data*, *file_path: str = None*)

dicee.read_preprocess_save_load_kg.util.**create_constraints**(*triples*, *file_path: str = None*)

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Crete constrainted entities based on the range of relations :param triples: :return: Tuple[dict, dict]

dicee.read_preprocess_save_load_kg.util.**load_with_pandas**(*self*) → None

Deserialize data

dicee.read_preprocess_save_load_kg.util.**save_numpy_ndarray**(*, *data: numpy.ndarray*, *file_path: str*)

dicee.read_preprocess_save_load_kg.util.**load_numpy_ndarray**(*, *file_path: str*)

dicee.read_preprocess_save_load_kg.util.**save_pickle**(*, *data: object*, *file_path=str*)

dicee.read_preprocess_save_load_kg.util.**load_pickle**(*, *file_path=str*)

dicee.read_preprocess_save_load_kg.util.**create_recipriocal_triples**(*x*)

Add inverse triples into dask dataframe :param x: :return:

dicee.read_preprocess_save_load_kg.util.**dataset_sanity_checking**(*train_set: numpy.ndarray*, *num_entities: int*, *num_relations: int*) → None

Parameters

- **train_set**
- **num_entities**
- **num_relations**

Returns

Classes

PreprocessKG	Preprocess the data in memory
LoadSaveToDisk	
ReadFromDisk	Read the data from disk into memory

Package Contents

class dicee.read_preprocess_save_load_kg.**PreprocessKG**(*kg*)

Preprocess the data in memory

kg

start() → None

Preprocess train, valid and test datasets stored in knowledge graph instance

Parameter

rtype

None

preprocess_with_byte_pair_encoding()

preprocess_with_byte_pair_encoding_with_padding() → None

Preprocess with byte pair encoding and add padding

preprocess_with_pandas() → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

preprocess_with_polars() → None

Preprocess with polars: add reciprocal triples and create indexed datasets

sequential_vocabulary_construction() → None

(1) Read input data into memory

(2) Remove triples with a condition

(3) **Serialize vocabularies in a pandas dataframe where**

=> the index is integer and => a single column is string (e.g. URI)

class dicee.read_preprocess_save_load_kg.**LoadSaveToDisk**(*kg*)

kg

save()

load()

class dicee.read_preprocess_save_load_kg.**ReadFromDisk**(*kg*)

Read the data from disk into memory

kg

start() → None

Read a knowledge graph from disk into memory

Data will be available at the train_set, test_set, valid_set attributes.

Parameter

None

rtype
None

add_noisy_triples_into_training()

dicee.sanity_checkers

Attributes

logger

Functions

is_sparql_endpoint_alive([sparql_endpoint])	
validate_knowledge_graph(args)	Validating the source of knowledge graph
sanity_checking_with_arguments(args)	
sanity_check_callback_args(args)	Perform sanity checks on callback-related arguments.

Module Contents

dicee.sanity_checkers.**logger**

dicee.sanity_checkers.**is_sparql_endpoint_alive**(*sparql_endpoint: str = None*)

dicee.sanity_checkers.**validate_knowledge_graph**(*args*)

Validating the source of knowledge graph

dicee.sanity_checkers.**sanity_checking_with_arguments**(*args*)

dicee.sanity_checkers.**sanity_check_callback_args**(*args*)

Perform sanity checks on callback-related arguments.

dicee.scripts

Submodules

dicee.scripts.index_serve

$ docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334 -v $(pwd)/qdrant_storage:/qdrant/storage:z qdrant/qdrant $ dicee_vector_db –index –serve –path CountryEmbeddings –collection "countries_vdb"

Attributes

logger
app
neural_searcher

Classes

NeuralSearcher	
StringListRequest	

Functions

get_default_arguments()	
index(args)	
root()	
search_embeddings(q)	
retrieve_embeddings(q)	
search_embeddings_batch(request)	
serve(args)	
main()	

Module Contents

dicee.scripts.index_serve.**logger**

dicee.scripts.index_serve.**get_default_arguments**()

dicee.scripts.index_serve.**index**(*args*)

dicee.scripts.index_serve.**app**

dicee.scripts.index_serve.**neural_searcher = None**

class dicee.scripts.index_serve.**NeuralSearcher**(*args*)

collection_name

entity_to_idx = None

qdrant_client

topk = 5

retrieve_embedding(*entity: str = None*, *entities: List[str] = None*) → List

search(*entity: str*)

async dicee.scripts.index_serve.**root**()

async dicee.scripts.index_serve.**search_embeddings**(*q: str*)

async dicee.scripts.index_serve.**retrieve_embeddings**(*q: str*)

class dicee.scripts.index_serve.**StringListRequest**

Bases: pydantic.BaseModel

queries: List[str]

reducer: str | None = None

async dicee.scripts.index_serve.**search_embeddings_batch**(*request: StringListRequest*)

dicee.scripts.index_serve.**serve**(*args*)

dicee.scripts.index_serve.**main**()

dicee.scripts.run

Functions

get_default_arguments([description])	Extends lightning Trainer's arguments with ours
main()	

Module Contents

dicee.scripts.run.**get_default_arguments**(*description=None*)

Extends lightning Trainer's arguments with ours

dicee.scripts.run.**main**()

dicee.static_funcs

Static utility functions for DICE embeddings.

This module provides utility functions for model initialization, data loading, serialization, and various helper operations.

Attributes

logger
MODEL_REGISTRY

Functions

create_recipriocal_triples(→ pandas.DataFrame)	Add inverse triples to a DataFrame.
get_er_vocab(→ Dict[Tuple[int, int], List[int]])	Build entity-relation to tail vocabulary.
get_re_vocab(→ Dict[Tuple[int, int], List[int]])	Build relation-entity (tail) to head vocabulary.
get_ee_vocab(→ Dict[Tuple[int, int], List[int]])	Build entity-entity to relation vocabulary.
timeit(→ Callable)	Decorator to measure and print execution time and memory usage.
setup_distributed_training(→ Dict[str, Union[bool, int]])	Resolve distributed-training state and initialize custom DDP when needed.
save_pickle(→ None)	Save data to a pickle file.
load_pickle(→ object)	Load data from a pickle file.
load_term_mapping(→ Union[dict, polars.DataFrame])	Load term-to-index mapping from pickle or CSV file.
select_model(args[, is_continual_training, storage_path])	
load_model(→ Tuple[object, Tuple[dict, dict]])	Load weights and initialize pytorch module from namespace arguments

continues on next page

Table 75 – continued from previous page

`load_model_ensemble(...)`	Construct Ensemble Of weights and initialize pytorch module from namespace arguments
`save_numpy_ndarray(*, data, file_path)`	
`numpy_data_type_changer(→ numpy.ndarray)`	Detect most efficient data type for a given triples
`save_checkpoint_model(→ None)`	Store Pytorch model into disk
`store(→ None)`	
`add_noisy_triples(→ pandas.DataFrame)`	Add randomly constructed triples
`read_or_load_kg(args, cls)`	
`intialize_model(...)`	Initialize a knowledge graph embedding model.
`load_json(→ Dict)`	Load JSON file into a dictionary.
`save_embeddings(→ None)`	Save embeddings to a CSV file.
`vocab_to_parquet(vocab_to_idx, name, ...)`	
`create_experiment_folder(→ str)`	Create a timestamped experiment folder.
`continual_training_setup_executor(→ None)`	
`exponential_function(→ torch.FloatTensor)`	
`load_numpy(→ numpy.ndarray)`	
`evaluate(entity_to_idx, scores, easy_answers, hard_answers)`	# @TODO: CD: Renamed this function
`download_file(url[, destination_folder])`	
`download_files_from_url(→ None)`	
`download_pretrained_model(→ str)`	
`write_csv_from_model_parallel(path)`	Create
`from_pretrained_model_write_embeddings_int(` None)	

Module Contents

dicee.static_funcs.**logger**

dicee.static_funcs.**MODEL_REGISTRY: Dict[str, Tuple[Type, str]]**

dicee.static_funcs.**create_recipriocal_triples**(*df: pandas.DataFrame*) → pandas.DataFrame

Add inverse triples to a DataFrame.

For each triple (s, p, o), creates an inverse triple (o, p_inverse, s).

Parameters
df – DataFrame with 'subject', 'relation', and 'object' columns.

Returns
DataFrame with original and inverse triples concatenated.

dicee.static_funcs.**get_er_vocab**(*data: numpy.ndarray*, *file_path: str | None = None*) → Dict[Tuple[int, int], List[int]]

Build entity-relation to tail vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns
Dictionary mapping (head, relation) pairs to list of tail entities.

dicee.static_funcs.**get_re_vocab**(*data: numpy.ndarray*, *file_path: str* | *None = None*)
→ Dict[Tuple[int, int], List[int]]

Build relation-entity (tail) to head vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns

Dictionary mapping (relation, tail) pairs to list of head entities.

dicee.static_funcs.**get_ee_vocab**(*data: numpy.ndarray*, *file_path: str* | *None = None*)
→ Dict[Tuple[int, int], List[int]]

Build entity-entity to relation vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns

Dictionary mapping (head, tail) pairs to list of relations.

dicee.static_funcs.**timeit**(*func: Callable*) → Callable

Decorator to measure and print execution time and memory usage.

Parameters

func – Function to be timed.

Returns

Wrapped function that prints timing information.

dicee.static_funcs.**setup_distributed_training**(*args*) → Dict[str, bool | int]

Resolve distributed-training state and initialize custom DDP when needed.

dicee.static_funcs.**save_pickle**(*, *data: object* | *None = None*, *file_path: str*) → None

Save data to a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

Parameters

- **data** – Object to serialize. If None, nothing is saved.
- **file_path** – Path where the pickle file will be saved.

dicee.static_funcs.**load_pickle**(*file_path: str*) → object

Load data from a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

Parameters

file_path – Path to the pickle file.

Returns

Deserialized object from the pickle file.

dicee.static_funcs.**load_term_mapping**(*file_path: str*) → dict | polars.DataFrame

Load term-to-index mapping from pickle or CSV file.

Attempts to load from pickle first, falls back to CSV if not found.

Parameters
: **file_path** – Base path without extension.

Returns
: Dictionary or Polars DataFrame containing the mapping.

dicee.static_funcs.**select_model**(*args: dict*, *is_continual_training: bool = None*, *storage_path: str = None*)

dicee.static_funcs.**load_model**(*path_of_experiment_folder: str*, *model_name='model.pt'*, *verbose=0*) → Tuple[object, Tuple[dict, dict]]

Load weights and initialize pytorch module from namespace arguments

dicee.static_funcs.**load_model_ensemble**(*path_of_experiment_folder: str*) → Tuple[*dicee.models.base_model.BaseKGE*, Tuple[pandas.DataFrame, pandas.DataFrame]]

Construct Ensemble Of weights and initialize pytorch module from namespace arguments

(1) Detect models under given path

(2) Accumulate parameters of detected models

(3) Normalize parameters

(4) Insert (3) into model.

dicee.static_funcs.**save_numpy_ndarray**(*, *data: numpy.ndarray*, *file_path: str*)

dicee.static_funcs.**numpy_data_type_changer**(*train_set: numpy.ndarray*, *num: int*) → numpy.ndarray

Detect most efficient data type for a given triples :param train_set: :param num: :return:

dicee.static_funcs.**save_checkpoint_model**(*model*, *path: str*) → None

Store Pytorch model into disk

dicee.static_funcs.**store**(*trained_model*, *model_name: str = 'model'*, *full_storage_path: str = None*, *save_embeddings_as_csv=False*) → None

dicee.static_funcs.**add_noisy_triples**(*train_set: pandas.DataFrame*, *add_noise_rate: float*) → pandas.DataFrame

Add randomly constructed triples :param train_set: :param add_noise_rate: :return:

dicee.static_funcs.**read_or_load_kg**(*args*, *cls*)

dicee.static_funcs.**intialize_model**(*args: Dict*, *verbose: int = 0*, *for_inference: bool = False*) → Tuple[*dicee.models.base_model.BaseKGE*, str]

Initialize a knowledge graph embedding model.

Parameters

- **args** – Dictionary containing model configuration including 'model' key.
- **verbose** – Verbosity level. If > 0, prints initialization message.
- **for_inference** – If True, never build the FSDP row-wise-sharded shell, even if args["trainer"] == "torchFSDP". A completed run's model.pt (written by TorchFSDPTrainer._materialize_model()) already has a plain, full entity_embeddings.weight — loading it into the sharded shell (which defers entity_embeddings until setup_fsdp_training()) fails with "Unexpected key(s): entity_embeddings.weight". Callers loading a finished model for inference (load_model, load_model_ensemble) must set this; callers building a fresh model to train/resume with (select_model) must not.

Returns
Tuple of (initialized model, form of labelling string).

Raises
ValueError – If the model name is not recognized.

dicee.static_funcs.**load_json**(*path: str*) → Dict

Load JSON file into a dictionary.

Parameters
path – Path to the JSON file.

Returns
Dictionary containing the JSON data.

Raises

- **FileNotFoundError** – If the file does not exist.
- **json.JSONDecodeError** – If the file contains invalid JSON.

dicee.static_funcs.**save_embeddings**(*embeddings: numpy.ndarray*, *indexes: List*, *path: str*) → None

Save embeddings to a CSV file.

Parameters

- **embeddings** – NumPy array of embeddings with shape (n_items, embedding_dim).
- **indexes** – List of index labels (entity/relation names).
- **path** – Output file path.

dicee.static_funcs.**vocab_to_parquet**(*vocab_to_idx*, *name*, *path_for_serialization*, *print_into*)

dicee.static_funcs.**create_experiment_folder**(*folder_name: str = 'Experiments'*) → str

Create a timestamped experiment folder.

Parameters
folder_name – Base directory name for experiments.

Returns
Full path to the created experiment folder.

dicee.static_funcs.**continual_training_setup_executor**(*executor*) → None

dicee.static_funcs.**exponential_function**(*x: numpy.ndarray*, *lam: float*, *ascending_order=True*)
→ torch.FloatTensor

dicee.static_funcs.**load_numpy**(*path*) → numpy.ndarray

dicee.static_funcs.**evaluate**(*entity_to_idx*, *scores*, *easy_answers*, *hard_answers*)

@TODO: CD: Renamed this function Evaluate multi hop query answering on different query types

dicee.static_funcs.**download_file**(*url*, *destination_folder='.'*)

dicee.static_funcs.**download_files_from_url**(*base_url: str*, *destination_folder='.'*) → None

Parameters

- **base_url** (e.g. "https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-dim128-epoch256-KvsAll")
- **destination_folder** (*e.g. "KINSHIP-Keci-dim128-epoch256-KvsAll"*)

dicee.static_funcs.**download_pretrained_model**(*url: str*) → str

dicee.static_funcs.**write_csv_from_model_parallel**(*path: str*)

Create

dicee.static_funcs.**from_pretrained_model_write_embeddings_into_csv**(*path: str*) → None

dicee.static_funcs_training

Training-related static functions.

This module provides backward compatibility by re-exporting evaluation functions from the new dicee.evaluation module, along with training utilities.

Deprecated since version Evaluation: functions have moved to `dicee.evaluation`. Use that module for new code. This module will continue to export training utilities.

Functions

evaluate_bpe_lp(→ Dict[str, float])	Evaluate link prediction with BPE-encoded entities.
evaluate_lp(→ Dict[str, float])	Evaluate link prediction with batched processing.
efficient_zero_grad(→ None)	Efficiently zero gradients using parameter.grad = None.
make_iterable_verbose(→ Iterable)	Wrap an iterable with tqdm progress bar if verbose is True.

Module Contents

dicee.static_funcs_training.**evaluate_bpe_lp**(*model*, *triple_idx: List[Tuple]*, *all_bpe_shaped_entities*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*) → Dict[str, float]

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dicee.static_funcs_training.**evaluate_lp**(*model*, *triple_idx*, *num_entities: int*, *er_vocab: Dict[Tuple, List]*, *re_vocab: Dict[Tuple, List]*, *info: str = 'Eval Starts'*, *batch_size: int = 128*, *chunk_size: int = 1000*) → Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.

- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.

Returns
Dictionary with H@1, H@3, H@10, and MRR metrics.

`dicee.static_funcs_training.`**`efficient_zero_grad`**`(`*model*`)` → None

Efficiently zero gradients using parameter.grad = None.

This is more efficient than optimizer.zero_grad() as it avoids memory operations.

See: https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html

Parameters
model – PyTorch model to zero gradients for.

`dicee.static_funcs_training.`**`make_iterable_verbose`**`(`*iterable_object: Iterable*, *verbose: bool*, *desc: str = 'Default'*, *position: int | None = None*, *leave: bool = True*`)` → Iterable

Wrap an iterable with tqdm progress bar if verbose is True.

Parameters

- **iterable_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

Returns
The original iterable or a tqdm-wrapped version.

dicee.static_preprocess_funcs

Attributes

logger
enable_log

Functions

timeit(func)	
preprocesses_input_args(args)	Sanity Checking in input arguments
create_constraints(→ Tuple[dict, dict, dict, dict])	
get_er_vocab(data)	

continues on next page

Table 78 – continued from previous page

get_re_vocab(data)	
get_ee_vocab(data)	
mapping_from_first_two_cols_to_third(train_se	

Module Contents

dicee.static_preprocess_funcs.**logger**

dicee.static_preprocess_funcs.**enable_log = False**

dicee.static_preprocess_funcs.**timeit**(*func*)

dicee.static_preprocess_funcs.**preprocesses_input_args**(*args*)

Sanity Checking in input arguments

dicee.static_preprocess_funcs.**create_constraints**(*triples: numpy.ndarray*)
→ Tuple[dict, dict, dict, dict]

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Create constraints entities based on the range of relations :param triples: :return:

dicee.static_preprocess_funcs.**get_er_vocab**(*data*)

dicee.static_preprocess_funcs.**get_re_vocab**(*data*)

dicee.static_preprocess_funcs.**get_ee_vocab**(*data*)

dicee.static_preprocess_funcs.**mapping_from_first_two_cols_to_third**(*train_set_idx*)

dicee.trainer

Submodules

dicee.trainer.auto_batch_finder

Attributes

logger

Functions

find_good_batch_size(→ Tuple[int, Optional[float]])	Find a batch size that uses GPU memory efficiently.

Module Contents

dicee.trainer.auto_batch_finder.**logger** dicee.trainer.auto_batch_finder.**find_good_batch_size**(*train_loader: torch.utils.data.DataLoader*, *training_step_fn: Callable*, *device*) → Tuple[int, float | None]

Find a batch size that uses GPU memory efficiently.

Progressively increases batch size (with tunable delta) until GPU memory exceeds 90% or a CUDA OOM error is raised, then backs off to the last safe batch size. Only supported on CUDA devices; returns the initial batch size unchanged on CPU.

Parameters

- **train_loader** – DataLoader wrapping the training dataset.
- **training_step_fn** – Callable[[batch], float] — runs one forward+backward pass and returns the scalar loss. Each trainer passes its own implementation so this function stays trainer-agnostic.
- **device** – torch.device (or anything accepted by torch.device()) used to monitor GPU memory.

Returns

(batch_size, runtime) where runtime is the wall-clock seconds of the last successful batch, or None when batch finding was skipped.

dicee.trainer.dice_trainer

DICE Trainer module for knowledge graph embedding training.

Provides the DICE_Trainer class which supports multiple training backends including PyTorch Lightning, DDP, and custom CPU/GPU trainers.

Attributes

logger

Classes

DICE_Trainer	DICE_Trainer implement

Functions

load_term_mapping(→ polars.DataFrame)	Load term-to-index mapping from CSV file.
initialize_trainer(...)	Initialize the appropriate trainer based on configuration.
get_callbacks(→ List)	Create list of callbacks based on configuration.

Module Contents

dicee.trainer.dice_trainer.**logger**

dicee.trainer.dice_trainer.**load_term_mapping**(*file_path: str*) → polars.DataFrame

Load term-to-index mapping from CSV file.

Parameters

file_path – Base path without extension.

Returns
Polars DataFrame containing the mapping.

`dicee.trainer.dice_trainer.initialize_trainer(args, callbacks: List)`
→ *dicee.trainer.torch_trainer.TorchTrainer* | *dicee.trainer.model_parallelism.TensorParallel* | *dicee.trainer.torch_trainer_ddp.*

Initialize the appropriate trainer based on configuration.

Parameters

- **args** – Configuration arguments containing trainer type.
- **callbacks** – List of training callbacks.

Returns
Initialized trainer instance.

Raises
AssertionError – If trainer is None after initialization.

`dicee.trainer.dice_trainer.get_callbacks(args)` → List

Create list of callbacks based on configuration.

Parameters
args – Configuration arguments.

Returns
List of callback instances.

`class dicee.trainer.dice_trainer.DICE_Trainer(args, is_continual_training: bool, storage_path, evaluator=None)`

DICE_Trainer implement
1- Pytorch Lightning trainer (https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html) 2- Multi-GPU Trainer(https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html) 3- CPU Trainer

args

is_continual_training:bool

storage_path:str

evaluator:

report:dict

report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start(*knowledge_graph*)

(1) Initialize training.

(2) Load model

(3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer(*callbacks: List*)
→ lightning.Trainer | *dicee.trainer.model_parallelism.TensorParallel* | *dicee.trainer.torch_trainer.TorchTrainer* | *dicee.t*

Initialize Trainer from input arguments

initialize_or_load_model()

init_dataloader(*dataset: torch.utils.data.Dataset*) → torch.utils.data.DataLoader

init_dataset() → torch.utils.data.Dataset

start(*knowledge_graph: dicee.knowledge_graph.KG* | *numpy.memmap*)
→ Tuple[*dicee.models.base_model.BaseKGE*, str]

Start the training

(1) Initialize Trainer

(2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation(*dataset*) → Tuple[*dicee.models.base_model.BaseKGE*, str]

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
   2.1 initialize trainer and model 2.2. Train model with configuration provided in args. 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.
3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns
model

dicee.trainer.model_parallelism

Tensor Parallelism trainer for ensemble knowledge graph embeddings.

This module implements the tensor parallelism training strategy described in:

"Multiple Run Ensemble Learning with Low-Dimensional Knowledge Graph Embeddings"
https://arxiv.org/abs/2104.05003

The TensorParallel trainer creates an ensemble of models, each trained on a separate GPU, allowing efficient utilization of multi-GPU systems through model parallelism.

Attributes

logger	

Classes

TensorParallel	Abstract base class for KGE model trainers.

Functions

extract_input_outputs(z[, device])	
forward_backward_update_loss(→ float)	

Module Contents

dicee.trainer.model_parallelism.**logger**

dicee.trainer.model_parallelism.**extract_input_outputs**(*z: list*, *device=None*)

dicee.trainer.model_parallelism.**forward_backward_update_loss**(*z: Tuple*, *ensemble_model*) → float

class dicee.trainer.model_parallelism.**TensorParallel**(*args*, *callbacks*)

Bases: *dicee.abstracts.AbstractTrainer*

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (TorchTrainer, TorchDDPTrainer, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered `AbstractCallback` can react.

Parameters

- **args** (*argparse.Namespace or similar*) – Processed configuration object. Must expose at least `random_seed` (int).
- **callbacks** (*list of* AbstractCallback) – Ordered list of callback instances to invoke at each lifecycle hook.

fit(**args*, ***kwargs*)

Train model

dicee.trainer.torch_trainer

Attributes

logger	

Classes

TorchTrainer	TorchTrainer for using single GPU or multi CPUs on a single node

Module Contents

`dicee.trainer.torch_trainer.`**`logger`**

`class` `dicee.trainer.torch_trainer.`**`TorchTrainer`**(*args*, *callbacks*)

Bases: *dicee.abstracts.AbstractTrainer*

TorchTrainer for using single GPU or multi CPUs on a single node

Arguments

callbacks: list of Abstract callback instances

`loss_function = None`

`optimizer = None`

`model = None`

`train_dataloaders = None`

`training_step = None`

`process`

`fit`(**args*, *train_dataloaders*, ***kwargs*) → None

Training starts

Arguments

kwargs:Tuple
empty dictionary

Return type
batch loss (float)

`forward_backward_update`(*x_batch: torch.Tensor*, *y_batch: torch.Tensor*) → torch.Tensor

Compute forward, loss, backward, and parameter update

Arguments

Return type
batch loss (float)

extract_input_outputs_set_device(*batch: list*) → Tuple

Construct inputs and outputs from a batch of inputs with outputs From a batch of inputs and put

Arguments

Return type
(tuple) mini-batch on select device

dicee.trainer.torch_trainer_ddp

Attributes

logger

Classes

TorchDDPTrainer	Abstract base class for KGE model trainers.
NodeTrainer	

Functions

make_iterable_verbose(→ Iterable)

Module Contents

dicee.trainer.torch_trainer_ddp.**logger**

dicee.trainer.torch_trainer_ddp.**make_iterable_verbose**(*iterable_object*, *verbose*, *desc='Default'*, *position=None*, *leave=True*) → Iterable

class dicee.trainer.torch_trainer_ddp.**TorchDDPTrainer**(*args*, *callbacks*)

Bases: *dicee.abstracts.AbstractTrainer*

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (TorchTrainer, TorchDDPTrainer, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered `AbstractCallback` can react.

Parameters

- **args** (*argparse.Namespace or similar*) – Processed configuration object. Must expose at least `random_seed` (int).
- **callbacks** (*list of AbstractCallback*) – Ordered list of callback instances to invoke at each lifecycle hook.

fit(**args*, ***kwargs*)

class dicee.trainer.torch_trainer_ddp.**NodeTrainer**(*trainer*, *model: torch.nn.Module*, *train_dataset_loader: torch.utils.data.DataLoader*, *callbacks*, *num_epochs: int*)

trainer

local_rank

global_rank

optimizer

train_dataset_loader

loss_func

callbacks

model

num_epochs

loss_history = []

ctx

scaler

extract_input_outputs(*z: list*)

train()

dicee.trainer.torch_trainer_fsdp

Multi-GPU trainer: row-wise sharded entity embeddings + FSDP dense parameters.

Entity embeddings are sharded row-wise across ranks via dist.all_to_all_single. Each rank owns a contiguous slice of entity rows; forward and backward use all_to_all to route index requests and gradient returns to the owning rank. A per-rank _LocalSparseAdam updates only the rows that received a non-zero gradient each batch.

Dense parameters (relation embeddings, Clifford coefficients, …) are wrapped with FSDP and updated by the standard dense optimizer.

Attributes

logger	
OptimizedModule	

Classes

TorchFSDPTrainer	Single-node multi-GPU trainer: row-wise sharded entity embeddings + FSDP.

Module Contents

dicee.trainer.torch_trainer_fsdp.**logger**

dicee.trainer.torch_trainer_fsdp.**OptimizedModule = None**

class dicee.trainer.torch_trainer_fsdp.**TorchFSDPTrainer**(*args*, *callbacks*)

Bases: *dicee.abstracts.AbstractTrainer*

Single-node multi-GPU trainer: row-wise sharded entity embeddings + FSDP.

Entity embeddings

Sharded row-wise across GPUs. all_to_all routes each index request to its owning rank; embeddings and gradients travel back the same way. _LocalSparseAdam updates only the rows accessed in each batch — O(active_rows) not O(shard_size). Each entity receives gradient from every rank's batches.

Dense parameters (relation embeddings, Clifford coefficients, ...)

Wrapped with FSDP; updated by the standard dense optimizer step().

Usage

Launched with torchrun:

torchrun –nproc_per_node=4 -u dicee –trainer torchFSDP …

local_rank

global_rank

is_global_zero

device

model = None

raw_model = None

optimizer = None

loss_func = None

train_dataset_loader = None

ptdtype

ctx

scaler

sharding_strategy

gradient_clip_val

use_compile

num_workers

prefetch_factor

entity_optim_device

checkpoint_every_n_epochs

fit(**args*, ***kwargs*)

extract_input_outputs(*z*)

Classes

DICE_Trainer	DICE_Trainer implement

Package Contents

class dicee.trainer.**DICE_Trainer**(*args*, *is_continual_training: bool*, *storage_path*, *evaluator=None*)

DICE_Trainer implement

1- Pytorch Lightning trainer (https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html) 2- Multi-GPU Trainer(https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html) 3- CPU Trainer

args

is_continual_training:bool

storage_path:str

evaluator:

report:dict

report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start(*knowledge_graph*)

(1) Initialize training.

(2) Load model

(3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer(*callbacks: List*)
→ lightning.Trainer | *dicee.trainer.model_parallelism.TensorParallel* | *dicee.trainer.torch_trainer.TorchTrainer* | *dicee.t*

Initialize Trainer from input arguments

initialize_or_load_model()

init_dataloader(*dataset: torch.utils.data.Dataset*) → torch.utils.data.DataLoader

init_dataset() → torch.utils.data.Dataset

start(*knowledge_graph: dicee.knowledge_graph.KG | numpy.memmap*)
→ Tuple[*dicee.models.base_model.BaseKGE*, str]

Start the training

(1) Initialize Trainer

(2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation(*dataset*) → Tuple[*dicee.models.base_model.BaseKGE*, str]

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
   2.1 initialize trainer and model 2.2. Train model with configuration provided in args. 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.
3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns
model

dicee.weight_averaging

Classes

ASWA	Adaptive stochastic weight averaging
SWA	Stochastic Weight Averaging callback.
SWAG	Stochastic Weight Averaging - Gaussian (SWAG).
EMA	Exponential Moving Average (EMA) callback.
TWA	Train with Weight Averaging (TWA) using subspace projection + averaging.

Module Contents

class dicee.weight_averaging.**ASWA**(*num_epochs*, *path*)

Bases: *dicee.abstracts.AbstractCallback*

Adaptive stochastic weight averaging ASWE keeps track of the validation performance and update s the ensemble model accordingly.

path

num_epochs

initial_eval_setting = None

epoch_count = 0

alphas = []

val_aswa = -1

on_fit_end(*trainer*, *model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

static compute_mrr(*trainer*, *model*) → float

get_aswa_state_dict(*model*)

decide(*running_model_state_dict*, *ensemble_state_dict*, *val_running_model*, *mrr_updated_ensemble_model*)

Perform Hard Update, software or rejection

Parameters

- **running_model_state_dict**
- **ensemble_state_dict**
- **val_running_model**
- **mrr_updated_ensemble_model**

on_train_epoch_end(*trainer*, *model*)

Called at the end of each training epoch.

Parameters

- **trainer** (AbstractTrainer *or* pl.Trainer) – The active trainer instance.
- **model** (BaseKGE) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

class dicee.weight_averaging.**SWA**(*swa_start_epoch*, *swa_c_epochs: int = 1*, *lr_init: float = 0.1*, *swa_lr: float = 0.05*, *max_epochs: int = None*)

Bases: *dicee.abstracts.AbstractCallback*

Stochastic Weight Averaging callback.

Initialize SWA callback.

swa_start_epoch: int
The epoch at which to start SWA.

swa_c_epochs: int
The number of epochs to use for SWA.

lr_init: float
The initial learning rate.

swa_lr: float
: The learning rate to use during SWA.

max_epochs: int
: The maximum number of epochs. args.num_epochs

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

swa_model = None

swa_n = 0

current_epoch = -1

static moving_average(*swa_model*, *running_model*, *alpha*)

Update SWA model with moving average of current model. Math: # SWA update: # θ_swa ← (1 - alpha) * θ_swa + alpha * θ # alpha = 1 / (n + 1), where n = number of models already averaged # alpha is tracked via self.swa_n in code

on_train_epoch_start(*trainer*, *model*)

Update learning rate according to SWA schedule.

on_train_epoch_end(*trainer*, *model*)

Apply SWA averaging if conditions are met.

on_fit_end(*trainer*, *model*)

Replace main model with SWA model at the end of training.

class dicee.weight_averaging.**SWAG**(*swa_start_epoch*, *swa_c_epochs: int = 1*, *lr_init: float = 0.1*, *swa_lr: float = 0.05*, *max_epochs: int = None*, *max_num_models: int = 20*, *var_clamp: float = 1e-30*)

Bases: *dicee.abstracts.AbstractCallback*

Stochastic Weight Averaging - Gaussian (SWAG). Parameters

swa_start_epoch
: [int] Epoch at which to start collecting weights.

swa_c_epochs
: [int] Interval of epochs between updates.

lr_init
: [float] Initial LR.

swa_lr
: [float] LR in SWA / GSWA phase.

max_epochs
: [int] Total number of epochs.

max_num_models
: [int] Number of models to keep for low-rank covariance approx.

var_clamp
[float] Clamp low variance for stability.

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

max_num_models = 20

var_clamp = 1e-30

mean = None

sq_mean = None

deviations = []

gswa_n = 0

current_epoch = -1

get_mean_and_var()
Return mean + variance (diagonal part).

sample(*base_model*, *scale=0.5*)
Sample new model from SWAG posterior distribution.

Math: # From SWAG, posterior is approximated as: # θ ~ N(mean, Σ) # where Σ ≈ diag(var) + (1/(K-1)) * D D^T # - mean = running average of weights # - var = elementwise variance (sq_mean - mean^2) # - D = [dev_1, dev_2, …, dev_K], deviations from mean (low-rank approx) # - K = number of collected models

Sampling step: # 1. θ_diag = mean + scale * std ⊙ ε, where ε ~ N(0, I) # 2. θ_lowrank = θ_diag + (D z) / sqrt(K-1), where z ~ N(0, I_K) # Final sample = θ_lowrank

on_train_epoch_start(*trainer*, *model*)
Update LR schedule (same as SWA).

on_train_epoch_end(*trainer*, *model*)
Collect Gaussian stats at the end of epochs after swa_start.

on_fit_end(*trainer*, *model*)
Set model weights to the collected SWAG mean at the end of training.

class dicee.weight_averaging.**EMA**(*ema_start_epoch: int*, *decay: float = 0.999*, *max_epochs: int = None*, *ema_c_epochs: int = 1*)

Bases: *dicee.abstracts.AbstractCallback*

Exponential Moving Average (EMA) callback.

Parameters

- **ema_start_epoch** (*int*) – Epoch to start EMA.
- **decay** (*float*) – EMA decay rate (typical: 0.99 - 0.9999) Math: θ_ema <- decay * θ_ema + (1 - decay) * θ

- **max_epochs** (*int*) – Maximum number of epochs.

ema_start_epoch

decay = 0.999

max_epochs = None

ema_c_epochs = 1

ema_model = None

current_epoch = -1

static ema_update(*ema_model*, *running_model*, *decay: float*)

Update EMA model with exponential moving average of current model. Math: # EMA update: # θ_ema ← (1 - alpha) * θ_ema + alpha * θ # alpha = 1 - decay, where decay is the EMA smoothing factor (typical 0.99 - 0.999) # alpha controls how much of the current model θ contributes to the EMA # decay is fixed in code –> can be extended to scheduled

on_train_epoch_start(*trainer*, *model*)

Track current epoch.

on_train_epoch_end(*trainer*, *model*)

Update EMA if past start epoch.

on_fit_end(*trainer*, *model*)

Replace main model with EMA model at the end of training.

class dicee.weight_averaging.**TWA**(*twa_start_epoch: int*, *lr_init: float*, *num_samples: int = 5*, *reg_lambda: float = 0.0*, *max_epochs: int = None*, *twa_c_epochs: int = 1*)

Bases: *dicee.abstracts.AbstractCallback*

Train with Weight Averaging (TWA) using subspace projection + averaging.

Parameters

twa_start_epoch
[int] Epoch to start TWA.

lr_init
[float] Learning rate used for β updates.

num_samples
[int] Number of sampled weight snapshots to build projection subspace.

reg_lambda
[float] Regularization coefficient for β updates.

max_epochs
[int] Total number of training epochs.

twa_c_epochs
[int] Interval of epochs between TWA updates.

twa_start_epoch

num_samples = 5

reg_lambda = 0.0

max_epochs = None

lr_init

twa_c_epochs = 1

current_epoch = -1

weight_samples = []

twa_model = None

base_weights = None

P = None

beta = None

sample_weights(*model*)

Collect sampled weights from the current model and maintain rolling buffer.

build_projection(*weight_samples*, *k=None*)

Build projection subspace from collected weight samples. :param weight_samples: list of flat weight tensors [(D,), …] :param k: number of basis vectors to keep. Defaults to min(N, D).

Returns

(D,) base weight vector (average) P: (D, k) projection matrix with top-k basis directions

Return type

mean_w

on_train_epoch_start(*trainer*, *model*)

Track epoch.

on_train_epoch_end(*trainer*, *model*)

Main TWA logic: build subspace and update in β space.

Math: # TWA weight update: # w_twa = mean_w + P * beta # mean_w = (1/n) * sum_i w_i (SWA baseline) # beta <- (1 - eta * lambda) * beta - eta * P^T * g # g = gradient of training loss w.r.t. full model weights # eta = learning rate, lambda = ridge regularization # P = orthonormal basis spanning sampled checkpoints {w_i}

on_fit_end(*trainer*, *model*)

Replace with TWA model at the end.

14.2 Attributes

__version__

14.3 Classes

Execute	Executor class for training, retraining and evaluating KGE models.
KGE	Knowledge Graph Embedding Class for interactive usage of pre-trained models

continues on next page

Table 97 – continued from previous page

QueryGenerator	
DICE_Trainer	DICE_Trainer implement
Evaluator	Evaluator class for KGE models in various downstream tasks.

14.4 Package Contents

class dicee.**Execute**(*args*, *continuous_training: bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

args

Processed input arguments.

distributed

Whether distributed training is enabled.

rank

Process rank in distributed training.

world_size

Total number of processes.

local_rank

Local GPU rank.

trainer

Training handler instance.

trained_model

The trained model after training completes.

knowledge_graph

The loaded knowledge graph.

report

Dictionary storing training metrics and results.

evaluator

Model evaluation handler.

distributed

rank

world_size

local_rank

args

is_continual_training = False

trainer: ***dicee.trainer.DICE_Trainer*** **| None = None**

trained_model = None

knowledge_graph: *dicee.knowledge_graph.KG* **| None = None**

report: Dict

evaluator: *dicee.evaluator.Evaluator* **| None = None**

start_time: float | None = None

is_local_rank_zero() → bool

is_global_rank_zero() → bool

cleanup()

setup_executor() → None

Set up storage directories for the experiment.

Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

create_and_store_kg() → None

Create knowledge graph and store as memory-mapped file.

Only executed on global rank 0 in distributed training, so exactly one process writes the shared memmap/vocab files regardless of how many nodes are involved. Skips if memmap already exists.

load_from_memmap() → None

Load knowledge graph from memory-mapped file.

save_trained_model() → None

Save a knowledge graph embedding model

(1) Send model to eval mode and cpu.

(2) Store the memory footprint of the model.

(3) Save the model into disk.

(4) Update the stats of KG again ?

Parameter

rtype

None

end(*form_of_labelling: str*) → dict

End training

(1) Store trained model.

(2) Report runtimes.

(3) Eval model if required.

Parameter

rtype

A dict containing information about the training and/or evaluation

write_report() → None

Report training related information in a report.json file

start() → dict

Start training

(1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

Parameter

rtype

A dict containing information about the training and/or evaluation

class dicee.**KGE**(*path=None*, *url=None*, *construct_ensemble=False*, *model_name=None*)

Bases: `dicee.abstracts.BaseInteractiveKGE`, `dicee.abstracts.InteractiveQueryDecomposition`, `dicee.abstracts.BaseInteractiveTrainKGE`

Knowledge Graph Embedding Class for interactive usage of pre-trained models

__str__()

to(*device: str*) → None

get_transductive_entity_embeddings(*indices: torch.LongTensor | List[str]*, *as_pytorch=False*, *as_numpy=False*, *as_list=True*) → torch.FloatTensor | numpy.ndarray | List[float]

create_vector_database(*collection_name: str*, *distance: str*, *location: str = 'localhost'*, *port: int = 6333*)

generate(*h=''*, *r=''*)

eval_lp_performance(*dataset=List[Tuple[str, str, str]]*, *filtered=True*)

predict_missing_head_entity(*relation: List[str] | str*, *tail_entity: List[str] | str*, *within=None*, *batch_size=2*, *topk=1*, *return_indices=False*) → Tuple

Given a relation and a tail entity, return top k ranked head entity.

argmax_{e in E } f(e,r,t), where r in R, t in E.

Parameter

relation: Union[List[str], str]

String representation of selected relations.

tail_entity: Union[List[str], str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_relations(*head_entity: List[str] | str*, *tail_entity: List[str] | str*, *within=None*, *batch_size=2*, *topk=1*, *return_indices=False*) → Tuple

Given a head entity and a tail entity, return top k ranked relations.

argmax_{r in R } f(h,r,t), where h, t in E.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

`predict_missing_tail_entity`(*head_entity: List[str] | str*, *relation: List[str] | str*, *within: List[str] = None*, *batch_size=2*, *topk=1*, *return_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

argmax_{e in E } f(h,r,e), where h in E and r in R.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

Returns: Tuple

scores

`predict`(**, *h: List[str] | str | None = None*, *r: List[str] | str | None = None*, *t: List[str] | str | None = None*, *within: List[str] | None = None*, *logits: bool = True*) → torch.FloatTensor

Predict scores for triples or missing triple elements.

Parameters

- **`h`** – Head entity/entities. None to predict heads.
- **`r`** – Relation/relations. None to predict relations.
- **`t`** – Tail entity/entities. None to predict tails.
- **`within`** – Optional list of entities to restrict predictions to.
- **`logits`** – If True, return raw scores. If False, return sigmoid scores (0-1).

Returns

- Single triple (h, r, t): scalar score
- Missing element: vector of all possible scores

Return type

torch.FloatTensor of scores. Shape depends on the query type

Raises

- **`TypeError`** – If inputs are not strings or lists of strings.

- **ValueError** – If a required argument for the query type is missing.

Examples

```
>>> # Score a specific triple
>>> model.predict(h="Mongolia", r="isLocatedIn", t="Asia", logits=False)
tensor(0.9523)
```

```
>>> # Get scores for all possible tail entities
>>> model.predict(h="Mongolia", r="isLocatedIn", t=None)
tensor([0.21, 0.95, 0.03, ...])  # One score per entity
```

predict_topk(**, h: str | List[str] | None = None, r: str | List[str] | None = None, t: str | List[str] | None = None, topk: int = 10, within: List[str] | None = None, batch_size: int = 1024*) → List[Tuple[str, float]] | List[List[Tuple[str, float]]]

Predict top-k missing items in a given triple pattern.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **topk** – Number of top predictions to return.
- **within** – Optional list of entities to restrict predictions to.
- **batch_size** – Batch size for processing multiple queries.

Returns

List[(item, score), …] of length topk. For batch query: List of such lists, one per query.

Return type

For single query

Raises

- **TypeError** – If h, r, or t is not a str or list of str.
- **ValueError** – If the required arguments for a query type are None.

Examples

```
>>> model.predict_topk(h=["Mongolia"], r=["isLocatedIn"], topk=3)
[('Asia', 0.99), ('Europe', 0.02), ...]
```

```
>>> model.predict_topk(r=["isLocatedIn"], t=["Asia"], topk=5)
[('Mongolia', 0.85), ('China', 0.82), ...]
```

triple_score(*h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False*) → torch.FloatTensor

Predict triple score

Parameter

head_entity: List[str]

String representation of selected entities.

relation: List[str]

String representation of selected relations.

tail_entity: List[str]

String representation of selected entities.

logits: bool

If logits is True, unnormalized score returned

Returns: Tuple

pytorch tensor of triple score

return_multi_hop_query_results(*aggregated_query_for_all_entities*, *k: int*, *only_scores*)

single_hop_query_answering(*query: tuple*, *only_scores: bool = True*, *k: int = None*, *use_logits: bool = True*)

answer_multi_hop_query(*query_type: str | None = None*, *query: Tuple[str | Tuple[str, str], Ellipsis] | None = None*, *queries: List[Tuple[str | Tuple[str, str], Ellipsis]] | None = None*, *tnorm: str = 'prod'*, *neg_norm: str = 'standard'*, *lambda_: float = 0.0*, *k: int = 10*, *only_scores: bool = False*, *use_logits: bool = True*) → List[Tuple[str, torch.Tensor]] | List[List[Tuple[str, torch.Tensor]]]

Answer multi-hop EPFO (Existential Positive First-Order) queries.

Supports 9 query types: 1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up. See docs/guides/multi_hop_queries.md for detailed query patterns.

Parameters

- **query_type** – Query pattern name. One of: - 1p: (e, (r,)) # One-hop - 2p: (e, (r1, r2)) # Two-hop - 3p: (e, (r1, r2, r3)) # Three-hop - 2i: ((e1, (r1,)), (e2, (r2,))) # Two-way intersection - 3i: ((e1, (r1,)), (e2, (r2,)), (e3, (r3,))) # Three-way intersection - ip: (((e1, (r1,)), (e2, (r2,))), (r3,)) # Intersection + projection - pi: ((e, (r1, r2)), (r3,)) # Projection + intersection (2i meets 2p) - 2u: ((e1, (r1,)), (e2, (r2,))) # Two-way union - up: ((e, (r1, r2)), (e, (r3,))) # Union + projection
- **query** – Single query tuple matching the query_type pattern.
- **queries** – Batch of queries. If provided, query must be None.
- **tnorm** – T-norm for intersection/union. Options: "prod", "min".
- **neg_norm** – Negation norm. Options: "standard", "sugeno", "yager".
- **lambda** – Parameter for sugeno and yager negation (0.0-1.0).
- **k** – Number of top answer entities to return.
- **only_scores** – If True, return only scores tensor. If False, return (entity, score) tuples.
- **use_logits** – If True, use raw model logits. If False, use sigmoid probabilities.

Returns

List[(entity, score), …] of top-k answers. For batch queries: List of such lists, one per query.

Return type
For single query

Raises

- **ValueError** – If query_type is not in {1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up}.
- **AssertionError** – If query structure doesn't match query_type pattern.

Examples

```
>>> # 1p: Find entities located in Asia
>>> model.answer_multi_hop_query(
...     query_type="1p",
...     query=("Asia", ("isLocatedIn",)),
...     k=5
... )
[("Mongolia", 0.92), ("China", 0.89), ...]
```

```
>>> # 2p: Two-hop query (e.g., "capital of countries in Europe")
>>> model.answer_multi_hop_query(
...     query_type="2p",
...     query=("Europe", ("isLocatedIn", "hasCapital")),
...     k=3
... )
[("Paris", 0.85), ("Berlin", 0.82), ...]
```

```
>>> # 2i: Intersection query
>>> model.answer_multi_hop_query(
...     query_type="2i",
...     query=(("Asia", ("isLocatedIn",)), ("Mountains", ("hasGeography",))),
...     k=5
... )
[("Nepal", 0.78), ("Tibet", 0.65), ...]
```

See also

- docs/guides/multi_hop_queries.md: Complete guide with all query patterns
- tests/test_answer_multi_hop_query.py: Usage examples

find_missing_triples(*confidence: float*, *entities: List[str] = None*, *relations: List[str] = None*, *topk: int = 10*, *at_most: int = sys.maxsize*) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

orall e in E and orall r in R f(e,r,x)

Return (e,r,x)

otin G and f(e,r,x) > confidence

confidence: float

A threshold for an output of a sigmoid function given a triple.

topk: int

Highest ranked k item to select triples with f(e,r,x) > confidence .

at_most: int

Stop after finding at_most missing triples

{(e,r,x) | f(e,r,x) > confidence land (e,r,x)

otin G

predict_literals(*entity: List[str] | str = None*, *attribute: List[str] | str = None*, *denormalize_preds: bool = True*) → numpy.ndarray

Predicts literal values for given entities and attributes.

Parameters

- **entity** (*Union[List[str], str]*) – Entity or list of entities to predict literals for.
- **attribute** (*Union[List[str], str]*) – Attribute or list of attributes to predict literals for.
- **denormalize_preds** (*bool*) – If True, denormalizes the predictions.

Returns

Predictions for the given entities and attributes.

Return type

numpy ndarray

class dicee.**QueryGenerator**(*train_path*, *val_path: str*, *test_path: str*, *ent2id: Dict = None*, *rel2id: Dict = None*, *seed: int = 1*, *gen_valid: bool = False*, *gen_test: bool = True*)

train_path

val_path

test_path

gen_valid = False

gen_test = True

seed = 1

max_ans_num = 1000000.0

mode

ent2id = None

rel2id: Dict = None

ent_in: Dict

ent_out: Dict

query_name_to_struct

list2tuple(*list_data*)

tuple2list(*x: List | Tuple*) → List | Tuple

Convert a nested tuple to a nested list.

set_global_seed(*seed: int*)
Set seed

construct_graph(*paths: List[str]*) → Tuple[Dict, Dict]
Construct graph from triples Returns dicts with incoming and outgoing edges

fill_query(*query_structure: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*, *answer: int*) → bool
Private method for fill_query logic.

achieve_answer(*query: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*) → set
Private method for achieve_answer logic. @TODO: Document the code

write_links(*ent_out*, *small_ent_out*)

ground_queries(*query_structure: List[str | List]*, *ent_in: Dict*, *ent_out: Dict*, *small_ent_in: Dict*, *small_ent_out: Dict*, *gen_num: int*, *query_name: str*)
Generating queries and achieving answers

unmap(*query_type*, *queries*, *tp_answers*, *fp_answers*, *fn_answers*)

unmap_query(*query_structure*, *query*, *id2ent*, *id2rel*)

generate_queries(*query_struct: List*, *gen_num: int*, *query_type: str*)
Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting queries and answers in return @ TODO: create a class for each single query struct

save_queries(*query_type: str*, *gen_num: int*, *save_path: str*)

abstractmethod load_queries(*path*)

get_queries(*query_type: str*, *gen_num: int*)

static save_queries_and_answers(*path: str*, *data: List[Tuple[str, Tuple[collections.defaultdict]]]*) → None
Save Queries into Disk

static load_queries_and_answers(*path: str*) → List[Tuple[str, Tuple[collections.defaultdict]]]
Load Queries from Disk to Memory

class dicee.**DICE_Trainer**(*args*, *is_continual_training: bool*, *storage_path*, *evaluator=None*)

DICE_Trainer implement
1- Pytorch Lightning trainer (https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html) 2- Multi-GPU Trainer(https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html) 3- CPU Trainer

args

is_continual_training:bool

storage_path:str

evaluator:

report:dict

report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start(*knowledge_graph*)

(1) Initialize training.

(2) Load model

(3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer(*callbacks: List*)
→ lightning.Trainer | *dicee.trainer.model_parallelism.TensorParallel* | *dicee.trainer.torch_trainer.TorchTrainer* | *dicee.t*

Initialize Trainer from input arguments

initialize_or_load_model()

init_dataloader(*dataset: torch.utils.data.Dataset*) → torch.utils.data.DataLoader

init_dataset() → torch.utils.data.Dataset

start(*knowledge_graph: dicee.knowledge_graph.KG | numpy.memmap*)
→ Tuple[*dicee.models.base_model.BaseKGE*, str]

Start the training

(1) Initialize Trainer

(2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation(*dataset*) → Tuple[*dicee.models.base_model.BaseKGE*, str]

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
   2.1 initialize trainer and model 2.2. Train model with configuration provided in args. 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.
3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns
model

class dicee.**Evaluator**(*args*, *is_continual_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab
: Entity-relation to tail vocabulary for filtered ranking.

re_vocab
: Relation-entity (tail) to head vocabulary.

ee_vocab
: Entity-entity to relation vocabulary.

num_entities
: Total number of entities in the knowledge graph.

num_relations
: Total number of relations in the knowledge graph.

args
: Configuration arguments.

report
: Dictionary storing evaluation results.

during_training
: Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation(*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval(*dataset*, *trained_model*, *form_of_labelling: str*, *during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity(*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*) → None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity(*, *train_set=None*, *valid_set=None*, *test_set=None*, *ordered_bpe_entities*, *trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate BytE model with generation.

eval_with_bpe_vs_all(*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all(*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*, *form_of_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all(*model*, *triple_idx*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte(*model*, *triples: List[List[str]]*, *info: str | None = None*) → Dict[str, float]

Evaluate BytE model with text generation.

Parameters

- **model** – BytE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all(*model*, *triples: List[List[str]]*, *info: str | None = None*, *form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp(*model*, *triple_idx*, *info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

dummy_eval(*trained_model*, *form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data(*dataset*, *trained_model*, *triple_idx: numpy.ndarray*, *form_of_labelling: str*) → Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

dicee.**__version__ = '0.3.3'**

Python Module Index

d

Index

Non-alphabetical

A

B

C

D

E

F

G

H

I

K

L

M

N

O

P

Q

R

S

T

U

V

W

X